

# Safe protocols for Substation Automation

## Literate Guide to the Six Table-Driven Filters

`safe.h` & `safe.c`

Arun Babu Puthuparambil, Shivakumar V, Pradish Murukesa Pillay,  
Shailesh Kapoor, Shivendra Sinha, Dinesh J

Central Power Research Institute, Bengaluru  
arunbabu [AT] cpri [DOT] in

## Contents

|          |                                                                            |           |
|----------|----------------------------------------------------------------------------|-----------|
| <b>1</b> | <b>Glossary</b>                                                            | <b>5</b>  |
| <b>2</b> | <b>Overview</b>                                                            | <b>7</b>  |
| 2.1      | What This Library Does . . . . .                                           | 7         |
| 2.2      | The Six Protocols . . . . .                                                | 7         |
| 2.3      | Where Each Protocol Lives in a Substation . . . . .                        | 8         |
| 2.4      | What Is a Filter? . . . . .                                                | 8         |
| 2.5      | Design Philosophy . . . . .                                                | 8         |
| 2.6      | Invariants . . . . .                                                       | 9         |
| <b>3</b> | <b>Getting Started</b>                                                     | <b>9</b>  |
| 3.1      | Files . . . . .                                                            | 9         |
| 3.2      | Compilation . . . . .                                                      | 9         |
| 3.3      | Hello World: Validating a GOOSE Frame . . . . .                            | 9         |
| 3.4      | Integration Patterns . . . . .                                             | 10        |
| 3.5      | Deployment Scenario: Engineering Workstation vs. Operations Zone . . . . . | 10        |
| <b>4</b> | <b><code>safe.h</code> – Constants, Types, API</b>                         | <b>11</b> |
| 4.1      | <code>SAFE_PURE</code> and <code>SAFE_CONST</code> Macros . . . . .        | 11        |
| 4.2      | Result Structs . . . . .                                                   | 11        |
| 4.3      | Rejection Codes . . . . .                                                  | 12        |
| 4.4      | Public API Functions . . . . .                                             | 14        |
| <b>5</b> | <b>Internal Wire-Format Constants</b>                                      | <b>14</b> |
| <b>6</b> | <b>Shared BER Length Decoder: <code>read_length</code></b>                 | <b>15</b> |
| 6.1      | The Complete Function . . . . .                                            | 16        |
| 6.1.1    | What <code>read_length</code> Does (Plain English) . . . . .               | 16        |
| 6.1.2    | The Formal Contract . . . . .                                              | 16        |
| 6.2      | ACSL Contract – Line by Line . . . . .                                     | 17        |
| 6.3      | How the Decoding Works . . . . .                                           | 17        |
| 6.4      | Ghost Variable and Loop Annotations . . . . .                              | 17        |
| <b>7</b> | <b>SafeASN.1 – Whitelist Array + Recursive TLV Walker</b>                  | <b>17</b> |
| 7.1      | What is ASN.1 and Why Does It Matter? . . . . .                            | 17        |
| 7.2      | Worked Example: Decoding a DER Message . . . . .                           | 17        |
| 7.3      | Implementation . . . . .                                                   | 18        |
| 7.4      | BER/DER TLV Wire Format . . . . .                                          | 18        |
| 7.5      | <code>ASN_TAG_WHITELIST</code> – The Whitelist Array . . . . .             | 18        |
| 7.6      | Blocked ASN.1 tags: rationale . . . . .                                    | 19        |
| 7.7      | <code>asn_filter_recursive</code> – The Recursive TLV Walker . . . . .     | 20        |
| 7.7.1    | ACSL Contract . . . . .                                                    | 20        |
| 7.7.2    | Ghost Variables and Loop Annotations . . . . .                             | 21        |
| 7.7.3    | Algorithm, Step by Step . . . . .                                          | 21        |

|         |                                                                         |    |
|---------|-------------------------------------------------------------------------|----|
| 7.7.4   | Termination Proof . . . . .                                             | 22 |
| 7.8     | <b>safe_asn_filter</b> – Entry Point . . . . .                          | 22 |
| 7.8.1   | Contract . . . . .                                                      | 23 |
| 7.8.2   | Behavior . . . . .                                                      | 23 |
| 7.9     | All 10 validation checks: what, why, and what error code . . . . .      | 23 |
| 7.9.1   | Check 1 – Null pointer (ASN_REJECT_NULL) . . . . .                      | 23 |
| 7.9.2   | Check 2 – Message size (ASN_REJECT_SIZE) . . . . .                      | 23 |
| 7.9.3   | Check 3 – Tag whitelist (ASN_REJECT_TAG) . . . . .                      | 23 |
| 7.9.4   | Check 4 – Indefinite length (ASN_REJECT_INDEF) . . . . .                | 23 |
| 7.9.5   | Check 5 – Long-form length byte count (ASN_REJECT_LENGTH) . . . . .     | 23 |
| 7.9.6   | Check 6 – Truncation / boundary (ASN_REJECT_TRUNCATE) . . . . .         | 24 |
| 7.9.7   | Check 7 – Nesting depth (ASN_REJECT_DEPTH) . . . . .                    | 24 |
| 7.9.8   | Check 8 – INTEGER content (ASN_REJECT_BADINT) . . . . .                 | 24 |
| 7.9.9   | Check 9 – BOOLEAN and NULL content (ASN_REJECT_CONTENT) . . . . .       | 24 |
| 7.9.10  | Check 10 – Node count (ASN_REJECT_TOO_MANY_NODES) . . . . .             | 24 |
| 8       | <b>SafeASN.1 Parser – Filter + Zero-Copy Node Extraction</b> . . . . .  | 24 |
| 8.1     | Types . . . . .                                                         | 24 |
| 8.2     | Behavior . . . . .                                                      | 25 |
| 9       | <b>SafeMMS – PDU Tag + Service Whitelists</b> . . . . .                 | 25 |
| 9.1     | What is MMS? . . . . .                                                  | 25 |
| 9.2     | Worked Example: MMS Read Request . . . . .                              | 25 |
| 9.3     | Implementation . . . . .                                                | 26 |
| 9.4     | MMS PDU Types . . . . .                                                 | 26 |
| 9.5     | MMS PDU Wire Format . . . . .                                           | 26 |
| 9.6     | MMS_PDU_WHITELIST – PDU Tag Whitelist Array . . . . .                   | 26 |
| 9.7     | MMS_SERVICE_WHITELIST – Service Tag Whitelist Array . . . . .           | 27 |
| 9.8     | Disabled MMS services: rationale . . . . .                              | 28 |
| 9.9     | <b>safe_mms_filter</b> – Entry Point . . . . .                          | 28 |
| 9.9.1   | ACSL Contract . . . . .                                                 | 28 |
| 9.9.2   | Algorithm . . . . .                                                     | 29 |
| 9.10    | All 7 validation checks: what, why, and what error code . . . . .       | 30 |
| 9.10.1  | Check 1 – PDU size (MMS_REJECT_SIZE) . . . . .                          | 30 |
| 9.10.2  | Check 2 – PDU tag whitelist (MMS_REJECT_PDU_TAG) . . . . .              | 30 |
| 9.10.3  | Check 3 – Truncation (MMS_REJECT_TRUNCATE) . . . . .                    | 30 |
| 9.10.4  | Check 4 – Inner SEQUENCE tag (MMS_REJECT_INNER_SEQ) . . . . .           | 30 |
| 9.10.5  | Check 5 – invokeId (MMS_REJECT_INVOKE_ID) . . . . .                     | 30 |
| 9.10.6  | Check 6 – Service tag whitelist (MMS_REJECT_SERVICE) . . . . .          | 30 |
| 9.10.7  | Check 7 – Service body content (MMS_REJECT_BODY) . . . . .              | 31 |
| 10      | <b>SafeDNP3 – FC/Group/Variation/Qualifier Whitelists</b> . . . . .     | 31 |
| 10.1    | What is DNP3? . . . . .                                                 | 31 |
| 10.2    | Worked Example: DNP3 Read Request . . . . .                             | 32 |
| 10.3    | Implementation . . . . .                                                | 32 |
| 10.4    | DNP3 Application Layer Wire Format . . . . .                            | 32 |
| 10.5    | DNP3_FUNCTION_CODE_WHITELIST – Function Code Whitelist Array . . . . .  | 32 |
| 10.6    | DNP3_GROUP_WHITELIST – Object Group Whitelist Array . . . . .           | 33 |
| 10.7    | DNP3_POINT_SIZE – 2D Point Size Lookup . . . . .                        | 34 |
| 10.7.1  | How each size is derived . . . . .                                      | 35 |
| 10.8    | DNP3_RANGE_SIZE – Qualifier Range Size . . . . .                        | 36 |
| 10.9    | DNP3_PREFIX_SIZE – Object Prefix Size . . . . .                         | 37 |
| 10.10   | Disabled DNP3 function codes: rationale . . . . .                       | 37 |
| 10.11   | <b>safe_dnp3_filter</b> – Entry Point . . . . .                         | 38 |
| 10.11.1 | ACSL Contract . . . . .                                                 | 38 |
| 10.11.2 | Algorithm . . . . .                                                     | 38 |
| 10.12   | Worked Examples: Object Size Counting . . . . .                         | 39 |
| 10.12.1 | Example 1 – analog input (g30 v1) . . . . .                             | 39 |
| 10.12.2 | Example 2 – packed binary input (g1 v1) . . . . .                       | 39 |
| 10.12.3 | Example 3 – time and interval (g50 v2), a corrected cell . . . . .      | 40 |
| 10.13   | All 7 validation checks: what, why, and what error code . . . . .       | 40 |
| 10.13.1 | Check 1 – Application layer size (DNP3_REJECT_SIZE) . . . . .           | 40 |
| 10.13.2 | Check 2 – Function code whitelist (DNP3_REJECT_FUNCTION_CODE) . . . . . | 40 |

|           |                                                                              |           |
|-----------|------------------------------------------------------------------------------|-----------|
| 10.13.3   | Check 3 – Object group whitelist (DNP3_REJECT_GROUP)                         | 40        |
| 10.13.4   | Check 4 – Variation validity (DNP3_REJECT_VARIATION)                         | 40        |
| 10.13.5   | Check 5 – Qualifier code validity (DNP3_REJECT_QUALIFIER)                    | 40        |
| 10.13.6   | Check 6 – Object header count (DNP3_REJECT_OBJ_COUNT)                        | 41        |
| 10.13.7   | Check 7 – Truncation (DNP3_REJECT_TRUNCATE)                                  | 41        |
| <b>11</b> | <b>SafeIEC104 – APCI Header + TypeID/VSQ Whitelists</b>                      | <b>41</b> |
| 11.1      | What is IEC 60870-5-104?                                                     | 41        |
| 11.2      | Worked Example: IEC 104 I-frame (M_SP_NA_1)                                  | 42        |
| 11.3      | Implementation                                                               | 42        |
| 11.4      | IEC 60870-5-104 APDU Wire Format                                             | 42        |
| 11.5      | IEC104_ELEMENT_SIZE – TypeID Element Size Array                              | 42        |
| 11.6      | IEC104_TYPEID_WHITELIST – TypeID Permission Array                            | 44        |
| 11.7      | IEC104_UFRAME_WHITELIST – U-Frame Control Field Whitelist                    | 44        |
| 11.8      | Disabled IEC 104 TypeIDs: rationale                                          | 44        |
| 11.9      | safe_iec104_filter – Entry Point                                             | 45        |
| 11.9.1    | ACSL Contract                                                                | 45        |
| 11.9.2    | Algorithm                                                                    | 46        |
| 11.10     | All 8 validation checks: what, why, and what error code                      | 46        |
| 11.10.1   | Check 1 – APDU size (IEC104_REJECT_SIZE)                                     | 46        |
| 11.10.2   | Check 2 – Start byte (IEC104_REJECT_START)                                   | 46        |
| 11.10.3   | Check 3 – L field consistency (IEC104_REJECT_LEN)                            | 46        |
| 11.10.4   | Check 4 – Frame type (IEC104_REJECT_UFRAME)                                  | 47        |
| 11.10.5   | Check 5 – TypeID whitelist (IEC104_REJECT_TYPEID)                            | 47        |
| 11.10.6   | Check 6 – VSQ validity (IEC104_REJECT_OBJECTS)                               | 47        |
| 11.10.7   | Check 7 – Body size (IEC104_REJECT_TRUNCATE)                                 | 47        |
| 11.10.8   | Check 8 – Minimum ASDU size (IEC104_REJECT_LEN)                              | 47        |
| 11.11     | Error Code Reference                                                         | 47        |
| <b>12</b> | <b>SafeGOOSE – State Machine + Data Value Validation</b>                     | <b>48</b> |
| 12.1      | What is GOOSE?                                                               | 48        |
| 12.2      | Worked Example: GOOSE Frame                                                  | 48        |
| 12.3      | Implementation                                                               | 48        |
| 12.4      | GOOSE Ethernet Frame Wire Format                                             | 49        |
| 12.5      | GOOSE_NEXT_STATE – State Transition Table                                    | 49        |
| 12.6      | GOOSE_FIELD_BOUNDS – Field Length Bounds                                     | 50        |
| 12.7      | GOOSE_DATA_VALUE_WHITELIST – Data Value Tag Whitelist Array                  | 50        |
| 12.8      | Disabled GOOSE data types: rationale                                         | 51        |
| 12.9      | goose_validate_alldata – Data Value Validator                                | 52        |
| 12.9.1    | ACSL Contract                                                                | 52        |
| 12.9.2    | Main Loop Annotations                                                        | 52        |
| 12.9.3    | Algorithm                                                                    | 52        |
| 12.10     | safe_goose_filter – Entry Point                                              | 53        |
| 12.10.1   | ACSL Contract                                                                | 53        |
| 12.10.2   | Algorithm                                                                    | 53        |
| 12.11     | All 14 validation checks: what, why, and what error code                     | 54        |
| 12.11.1   | Check 1 – Frame size (GOOSE_REJECT_SIZE)                                     | 54        |
| 12.11.2   | Check 2 – EtherType (GOOSE_REJECT_ETH_TYPE)                                  | 54        |
| 12.11.3   | Check 3 – Header length consistency (GOOSE_REJECT_HDR_LEN)                   | 54        |
| 12.11.4   | Check 4 – Outer APPLICATION 1 tag (GOOSE_REJECT_OUTER_TAG)                   | 54        |
| 12.11.5   | Check 5 – Field tag state machine (GOOSE_REJECT_FIELD_TAG)                   | 54        |
| 12.11.6   | Check 6 – Field length bounds (GOOSE_REJECT_FIELD_SIZE)                      | 54        |
| 12.11.7   | Check 7 – Field boundary / truncation (GOOSE_REJECT_TRUNCATE)                | 55        |
| 12.11.8   | Check 8 – Data value tag whitelist (GOOSE_REJECT_DATA_VALUE_TAG)             | 55        |
| 12.11.9   | Check 9 – Data value count (GOOSE_REJECT_DATA_VALUE_COUNT)                   | 55        |
| 12.11.10  | Check 10 – Bit-string padding byte (GOOSE_REJECT_DATA_VALUE_PADDING)         | 55        |
| 12.11.11  | Check 11 – Simulation / test frame (GOOSE_REJECT_TEST)                       | 55        |
| 12.11.12  | Check 12 – allData integer and unsigned widths (GOOSE_REJECT_DATA_VALUE_LEN) | 55        |
| 12.11.13  | Check 13 – Unsigned32 header field value range (GOOSE_REJECT_FIELD_VALUE)    | 56        |
| 12.11.14  | Check 14 – allData unsigned value range (GOOSE_REJECT_DATA_VALUE_RANGE)      | 56        |

|                                                                                          |           |
|------------------------------------------------------------------------------------------|-----------|
| <b>13 SafeSV – Sampled Values Filter</b>                                                 | <b>56</b> |
| 13.1 What are Sampled Values, and why does security matter?                              | 56        |
| 13.2 SV on the wire: the complete frame layout                                           | 56        |
| 13.3 All 13 validation checks: what, why, and what error code                            | 57        |
| 13.3.1 Check 1 – Frame size (SV_REJECT_SIZE)                                             | 57        |
| 13.3.2 Check 2 – EtherType (SV_REJECT_ETH_TYPE)                                          | 58        |
| 13.3.3 Check 3 – APPID range (SV_REJECT_APPID)                                           | 58        |
| 13.3.4 Check 4 – Header length consistency (SV_REJECT_HDR_LEN)                           | 58        |
| 13.3.5 Check 5 – Outer savPDU tag (SV_REJECT_OUTER_TAG)                                  | 59        |
| 13.3.6 Check 6 – Truncation (throughout: SV_REJECT_TRUNCATE)                             | 59        |
| 13.3.7 Check 7 – ASDU sequence tag (SV_REJECT_ASDU_TAG)                                  | 59        |
| 13.3.8 Check 8 – ASDU count limit (SV_REJECT_ASDU_COUNT)                                 | 59        |
| 13.3.9 Checks 9–10 – ASDU field tag and size (SV_REJECT_FIELD_TAG, SV_REJECT_FIELD_SIZE) | 59        |
| 13.3.10 Check 11 – smpSynch and smpMod content (SV_REJECT_CONTENT)                       | 59        |
| 13.3.11 Check 12 – noASDU consistency (SV_REJECT_NOASDU)                                 | 60        |
| 13.3.12 Check 13 – Trailing bytes (SV_REJECT_TRAILING)                                   | 60        |
| 13.4 The SV field descriptor array and sequential validation                             | 61        |
| 13.4.1 Fixed-position array versus dynamic tag dispatch                                  | 61        |
| 13.4.2 Why these specific bounds?                                                        | 61        |
| 13.4.3 How the sequential walk handles optional fields                                   | 62        |
| 13.4.4 ACSL annotations for <code>sv_validate_asdu</code>                                | 62        |
| 13.5 Complete error code reference                                                       | 62        |
| 13.6 ACSL contract for <code>safe_sv_filter</code>                                       | 63        |
| 13.7 Worked example: annotated step-by-step validation                                   | 63        |
| <b>14 ACSL Annotations – Complete Reference</b>                                          | <b>64</b> |
| 14.1 Function Contracts                                                                  | 64        |
| 14.2 Loop Annotations                                                                    | 65        |
| 14.3 Ghost Variables                                                                     | 65        |
| 14.4 Inline Assertions                                                                   | 65        |
| 14.5 The <code>decreases</code> Clause (Recursive Termination)                           | 66        |
| 14.6 The <code>\separated</code> Clause in Detail                                        | 66        |
| <b>15 Frama-C Verification</b>                                                           | <b>66</b> |
| 15.1 Why Formal Verification Matters                                                     | 66        |
| 15.2 Running the Verification                                                            | 66        |
| 15.3 What the Command-Line Flags Mean                                                    | 67        |
| 15.4 What the Verification Covers                                                        | 67        |
| <b>16 Lookup Table Catalog</b>                                                           | <b>67</b> |
| <b>17 Summary of All Functions</b>                                                       | <b>67</b> |

# 1 Glossary

|                        |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |
|------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>ACSL</b>            | ANSI/ISO C Specification Language. A formal annotation language used with Frama-C to write function contracts ( <b>requires</b> , <b>ensures</b> , <b>assigns</b> ).                                                                                                                                                                                                                                                                                                                                                              |
| <b>APCI</b>            | Application Protocol Control Information. The fixed 6-byte header prepended to every IEC 60870-5-104 message: start byte <b>0x68</b> , length byte L, and four control-field bytes CF1–CF4. The APCI identifies the frame type (I/S/U) and carries send/receive sequence numbers for flow control.                                                                                                                                                                                                                                |
| <b>APDU</b>            | Application Protocol Data Unit. The application-layer message (e.g., a GOOSE APDU contains the 12 fields of a GOOSE message).                                                                                                                                                                                                                                                                                                                                                                                                     |
| <b>APPID</b>           | Application Identifier. A 2-byte field in the GOOSE and SV Ethernet headers (bytes 14–15) that identifies the logical device publishing the data. GOOSE APPIDs occupy <b>0x0000–0x3FFF</b> ; SV APPIDs occupy <b>0x4000–0x7FFF</b> . SafeGOOSE and SafeSV reject frames with APPIDs outside the assigned range.                                                                                                                                                                                                                   |
| <b>ASN.1</b>           | Abstract Syntax Notation One. A standard for describing data structures that can be serialised into binary bytes. Used by X.509 certificates, SNMP, LDAP, and IEC 61850.                                                                                                                                                                                                                                                                                                                                                          |
| <b>ASDU</b>            | Application Service Data Unit. In SV, one set of sampled measurements from one instant in time.                                                                                                                                                                                                                                                                                                                                                                                                                                   |
| <b>BER</b>             | Basic Encoding Rules. A flexible binary encoding for ASN.1 values. Allows multiple valid encodings for the same value (e.g., indefinite-length, non-minimal integers).                                                                                                                                                                                                                                                                                                                                                            |
| <b>CA</b>              | Common Address. A 2-byte little-endian field in the IEC 104 ASDU header that identifies the remote station (outstation) being addressed. All information objects in one ASDU share the same CA.                                                                                                                                                                                                                                                                                                                                   |
| <b>CF1</b>             | Control Field byte 1 in an IEC 60870-5-104 APCI header (the third byte of the 6-byte header, immediately after the start byte and length byte). The low two bits of CF1 identify the frame type: bit 0 = 0 means an I-frame (information transfer); <b>CF1 = 0x01</b> (bit 0 = 1, bit 1 = 0, other bits zero) means an S-frame (supervisory, acknowledgement only); bit 0 = 1 with bit 1 = 1 means a U-frame (unnumbered, connection management). SafeIEC104 reads CF1 first to classify the frame before any further validation. |
| <b>Circuit breaker</b> | A high-voltage switch that can disconnect a faulty section of the power grid under load. Opening a circuit breaker isolates the fault and protects equipment from damage. Protection relays send trip commands to breakers via GOOSE or DNP3 Direct Operate.                                                                                                                                                                                                                                                                      |
| <b>COT</b>             | Cause of Transmission. A 1-byte (or 2-byte) field in the IEC 104 ASDU header that explains why the message was sent. Examples: 3 = spontaneous (event-driven), 5 = requested, 20 = station interrogation response. SafeIEC104 does not whitelist COT values; it only checks structural fields.                                                                                                                                                                                                                                    |
| <b>COTP</b>            | Connection-Oriented Transport Protocol (ISO 8073). A thin session-management layer immediately above TPKT in the MMS stack. COTP has two roles: connection setup, using a CR (Connection Request, code <b>0xE0</b> ) / CC (Connection Confirm, code <b>0xD0</b> ) handshake, and data transfer using DT PDUs (code <b>0xF0</b> ). Only DT PDUs carry MMS payload; CR and CC are handled by the IED transport stack and never reach SafeMMS. All COTP framing is stripped before SafeMMS sees the MMS PDU.                         |
| <b>DER</b>             | Distinguished Encoding Rules. A strict subset of BER that allows exactly one encoding per value. Our filters require DER.                                                                                                                                                                                                                                                                                                                                                                                                         |
| <b>DNP3</b>            | Distributed Network Protocol 3 (IEEE 1815). A SCADA protocol for communication between control centres and remote terminal units.                                                                                                                                                                                                                                                                                                                                                                                                 |
| <b>EtherType</b>       | A 2-byte field at bytes 12–13 of an Ethernet frame that identifies the Layer 3 protocol carried in the payload. GOOSE uses <b>0x88B8</b> ; SV uses <b>0x88BA</b> ; regular IP traffic uses <b>0x0800</b> . The filters reject any frame whose EtherType is not the expected value.                                                                                                                                                                                                                                                |
| <b>Frama-C</b>         | A static analysis platform for C code. We use its WP (Weakest Precondition) plugin to prove properties about the filters.                                                                                                                                                                                                                                                                                                                                                                                                         |
| <b>GOOSE</b>           | Generic Object Oriented Substation Event. A fast Layer 2 Ethernet protocol for protection signalling between relays. Latency budget: 3 ms.                                                                                                                                                                                                                                                                                                                                                                                        |
| <b>I-frame</b>         | Information frame in IEC 104. Carries an ASDU with TypeID, VSQ, COT, CA, and information objects. Identified by CF1 bit 0 = 0.                                                                                                                                                                                                                                                                                                                                                                                                    |

## IEC 60870-5-104

An extension of the IEC 60870-5-101 telecontrol standard that maps the ASDU (Application Service Data Unit) onto TCP/IP using a fixed 6-byte APCI header (start byte 0x68, length byte, four control-field bytes CF1–CF4). Commonly abbreviated IEC 104 or T104. Used for SCADA communication between control centres and substations.

**IEC 61850** The international standard for communication in electrical substations. Defines GOOSE, SV, and MMS protocols.

**IED** Intelligent Electronic Device. A microprocessor-based controller in a substation (e.g., protection relay, bay controller).

**IOA** Information Object Address. A 3-byte little-endian address inside each IEC 104 information object that identifies the specific data point (e.g., breaker position sensor 5 at the remote station). Every information object in an I-frame begins with its 3-byte IOA.

**lib60870** An open-source C library (MZ Automation) implementing IEC 60870-5-101 and IEC 60870-5-104. The `IEC104_ELEMENT_SIZE` and `IEC104_TYPEID_WHITELIST` tables in `SafeIEC104` were cross-referenced against the lib60870 TypeID definitions.

## Merging Unit (MU)

A device installed close to high-voltage instrument transformers. It digitises the analogue current and voltage signals at up to 4000 samples per second and broadcasts the digital measurements as SV frames over the process-bus Ethernet. Protection relays subscribe to these frames to perform fast fault calculations.

**MMS** Manufacturing Message Specification (ISO 9506). A client-server protocol over TCP for reading/writing data on IEDs (station bus).

**Multicast** A network addressing scheme where one sender transmits a single packet that is delivered to a group of interested receivers simultaneously. GOOSE and SV use Layer 2 multicast (not broadcast) so that only subscribed devices process the frame; the Ethernet switch delivers it only to ports in the multicast group.

**PDU** Protocol Data Unit. A generic term for a unit of data at any protocol layer.

**Process bus** The substation-internal Ethernet network that carries time-critical Layer 2 traffic: GOOSE (protection signalling, 3 ms latency) and SV (continuous measurement streams, 250  $\mu$ s latency). Merging units and protection relays are directly connected to the process bus. No routers; pure switched Ethernet.

## Protection relay

An IED that continuously monitors electrical quantities (current, voltage, frequency) and automatically trips a circuit breaker within milliseconds when it detects a fault such as a short circuit or overload. Relays receive real-time measurements via SV and exchange protection signals via GOOSE.

**RTU** Remote Terminal Unit. A field device that connects sensors and actuators at a remote site (substation or feeder) to a SCADA control centre. RTUs communicate upstream via DNP3 or IEC 104 over TCP or serial links. Equivalent to “outstation” in DNP3 terminology.

**S-frame** Supervisory frame in IEC 104. Contains only receive-sequence acknowledgement numbers (CF2–CF4); no ASDU follows. Identified by `CF1 = 0x01`.

**SCADA** Supervisory Control and Data Acquisition. A system for monitoring and controlling industrial processes remotely.

**SCL** Substation Configuration Language. An XML format (IEC 61850-6) describing the complete configuration of a substation.

**State machine** A model with numbered states and transitions. Used by SafeGOOSE to enforce strict field ordering.

**Station bus** The substation Ethernet network for engineering and monitoring traffic. Engineering workstations connect to IEDs over the station bus using MMS/TCP to read measurements, configure settings, and retrieve disturbance records. Not time-critical; TCP/IP is used.

**SV** Sampled Values. A Layer 2 Ethernet protocol carrying digitised current/voltage measurements from instrument transformers. Latency budget: 250  $\mu$ s.

|                        |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
|------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>TLV</b>             | Tag–Length–Value. The fundamental encoding unit: a tag byte identifying the data type, a length field, and the content bytes.                                                                                                                                                                                                                                                                                                                                                                                                                          |
| <b>TPKT</b>            | A simple 4-byte framing header (RFC 1006) that wraps OSI/ISO data over TCP. Used as the transport layer beneath COTP in the MMS protocol stack. TPKT byte 0 is always 3; bytes 2–3 give the total packet length including the TPKT header.                                                                                                                                                                                                                                                                                                             |
| <b>TypeID</b>          | A 1-byte field in the IEC 104 ASDU that identifies the data type of every information object in the message. IEC 60870-5-101 defines 127 TypeIDs. Names follow the convention: prefix <b>M_</b> = monitored data (from outstation), <b>C_</b> = command (to outstation); then the type ( <b>SP</b> = single-point, <b>DP</b> = double-point, <b>ME</b> = measured value); then time-tag suffix ( <b>NA</b> = no time tag, <b>TA/TB</b> = with time tag). Example: <b>M_SP_NA_1</b> = monitored single-point information without time tag, variation 1. |
| <b>U-frame</b>         | Unnumbered frame in IEC 104. Used for connection management commands: STARTDT act/con, STOPDT act/con, TESTFR act/con. No ASDU. Identified by CF1 bits <b>0b11</b> . SafeIEC104 whitelists the six permitted CF1 values (0x07, 0x0B, 0x43, 0x83, 0x13, 0x23).                                                                                                                                                                                                                                                                                          |
| <b>Whitelist array</b> | A lookup table where <code>table[value] == 1</code> means accept and 0 means reject. Provides O(1) constant-time decisions.                                                                                                                                                                                                                                                                                                                                                                                                                            |

## 2 Overview

### 2.1 What This Library Does

This library provides six input filters for industrial communication protocols used in electrical substations. Each filter answers one question: “**Is this network packet structurally safe to process?**” The filter reads raw bytes from the network, checks them against a set of rules (allowed tags, field sizes, nesting depth), and returns a verdict: accept or reject with a specific reason code and byte offset.

The filters do *not* modify the input, do *not* allocate memory, and have *no* side effects. They are pure functions: the same input always produces the same output.

### 2.2 The Six Protocols

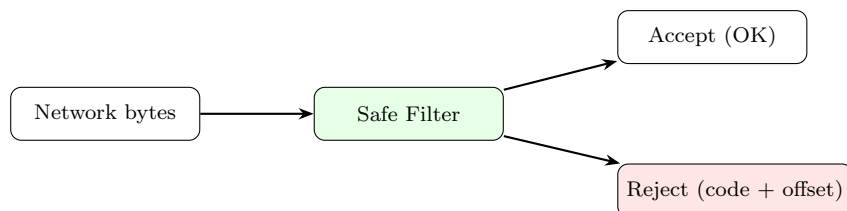


Figure 1: Filter architecture: bytes in, verdict out

An IEC 61850 substation uses multiple communication protocols at different layers:

- **GOOSE** (process bus): Fast protection signalling between relays. A relay detecting a fault sends a GOOSE frame over Ethernet multicast within 3 ms. Think of it as a digital “trip wire”: when a breaker needs to open, GOOSE delivers the signal.
- **Sampled Values (SV)** (process bus): Digitised current and voltage measurements from instrument transformers, streamed 80 times per power cycle (4000 samples/s at 50 Hz). Merging units send SV frames over Ethernet multicast with 250  $\mu$ s timing precision.
- **MMS** (station bus): A client-server protocol over TCP for engineering workstations to read and write data on IEDs. Used for configuration, monitoring, and control, not time-critical like GOOSE or SV.
- **DNP3** (SCADA link): A polling protocol between a control centre and remote substations. Carries status readings (binary inputs, analog values) and control commands (breaker open/close) over TCP or serial.
- **IEC 60870-5-104** (telecontrol link): The TCP/IP variant of the IEC 60870-5 telecontrol standard. Carries monitoring data (single-point, double-point, analog) and optionally commands between a control centre and RTUs. The APCI uses a fixed 6-byte header (start byte 0x68) and three frame types: I-frames (data), S-frames (supervisory), and U-frames (connection management).
- **ASN.1 DER** (encoding layer): The binary encoding used by GOOSE, SV, and MMS. Every field in these protocols is encoded as a Tag–Length–Value (TLV) triplet. SafeASN.1 is the structural validator shared by all three.

Table 1: Protocol placement in a substation

| Zone              | Protocol | Transport        | Port / Layer     | Typical traffic                             |
|-------------------|----------|------------------|------------------|---------------------------------------------|
| Process bus       | GOOSE    | Raw Ethernet     | EtherType 0x88B8 | Relay protection signals, breaker trips     |
| Process bus       | SV       | Raw Ethernet     | EtherType 0x88BA | Current/voltage samples from merging units  |
| Station bus       | MMS      | TCP/IP           | TCP 102          | Engineering workstation reads/writes        |
| WAN / telecontrol | DNP3     | TCP/IP or serial | TCP 20000        | SCADA polling, commands from control centre |
| WAN / telecontrol | IEC 104  | TCP/IP           | TCP 2404         | SCADA polling, commands (European standard) |

## 2.3 Where Each Protocol Lives in a Substation

A useful mental model: a modern digital substation has three communication zones.

**Key point for filter deployment.** GOOSE and SV filters see complete Ethernet frames (including the 14-byte Ethernet header). The MMS filter sees only the MMS PDU after TCP/TPKT/COTP/ISO layers are stripped. The DNP3 filter sees only the application-layer bytes after data-link and transport headers are removed. The IEC 104 filter sees the raw 6-byte APCI plus any following ASDU, directly off the TCP byte stream.

## 2.4 What Is a Filter?

A filter is a C function that takes a buffer of raw bytes and its length, and returns a result struct:

```

1 // Example: validate a GOOSE Ethernet frame
2 SafeGooseResult r = safe_goose_filter(frame_bytes, frame_length);
3
4 if (! r.error) {
5     // Frame is valid. Process it.
6     printf("Valid GOOSE, %u data values\n", r.data_value_count);
7 } else {
8     // Frame rejected. Log and discard.
9     printf("Rejected: code %u at byte %u\n", r.error, r.reject_at);
10 }

```

Key properties:

- **Read-only:** The filter never modifies the input buffer.
- **No allocation:** No `malloc`, no `free`, no heap. All working memory is on the stack.
- **Deterministic:** Same input always produces same output.
- **Bounded time:**  $O(n)$  where  $n$  is the input length. No backtracking, no exponential blowup.
- **Formally verified:** Memory safety and key properties proved by Frama-C/WP.

## 2.5 Design Philosophy

The table-driven approach is inspired by McConnell’s *Code Complete*, which advocates replacing complex conditional logic with data stored in tables. Instead of chains of `if/else` if testing individual values, each decision is a single array lookup: `TABLE[value]`. This has several advantages:

- **Readability.** The accept set is visible as data, not buried in conditionals.
- **Constant time.** Every lookup is  $O(1)$  regardless of the number of accepted values.
- **Verifiability.** Frama-C can reason about array accesses more easily than about long chains of Boolean conditions.
- **Maintainability.** Adding or removing an accepted value is a one-line change to a table initializer, not a restructuring of branching logic.

There are three kinds of lookup tables in the codebase:

- **Whitelist arrays.** `TABLE[value] == 1` means “accept this value.” A value of 0 (the default for uninitialized entries) means reject. This pattern is used for ASN.1 tags (`ASN_TAG_WHITELIST`), GOOSE data-value tags (`GOOSE_DATA_VALUE_WHITELIST`), MMS PDU tags (`MMS_PDU_WHITELIST`), MMS service tags (`MMS_SERVICE_WHITELIST`), DNP3 function codes (`DNP3_FUNCTION_CODE_WHITELIST`), DNP3 object groups (`DNP3_GROUP_WHITELIST`), IEC 104 TypeIDs (`IEC104_TYPEID_WHITELIST`), and IEC 104 U-frame subtypes (`IEC104_UFRAME_WHITELIST`).
- **State transition tables.** `GOOSE_NEXT_STATE[state][tag]` yields the next state. Zero means reject.
- **Direct lookup tables.** `DNP3_POINT_SIZE[group][variation]` yields the point size in bytes. Sentinel values `DNP3_VARIATION_INVALID` (0x00) and `DNP3_VARIATION_PACKED` (0xFF) encode “invalid” and “packed bits,” while `DNP3_VARIATION_ANY` (0xFD) marks variation 0/class objects with no fixed size (short aliases PKD/ANY are used in the generating macro for readability). A struct array (`SV_FIELDS[10]`, with tag/max/min/optional per entry) describes sequential SV ASDU fields. Additional lookup tables (`DNP3_RANGE_SIZE`, `DNP3_PREFIX_SIZE`, `GOOSE_FIELD_BOUNDS`) provide bounds and sizes indexed by wire-format codes.

In total there are **22 lookup tables** in the implementation. Every one is declared `static const`, meaning it lives in the read-only data segment and cannot be modified at runtime.



## 2.6 Invariants

These hold across the entire codebase:

- **Zero libc calls.** No standard headers are included. Fixed-width types (u8, u16, u32, u64) are `typedef`'d directly in `safe.h` from `unsigned char/int`, with compile-time size assertions. There is no `printf`, no `memcpy`, no `strlen`, no `malloc`, nothing.
- **No dynamic memory.** Every result is returned by value on the stack. No `malloc`, no `free`, no heap.
- **Single include.** A project that wants to use the library includes `safe.h` and compiles `safe.c`. That is the entire integration story.
- **Const correctness.** All input buffers are `const u8 *const`. All lookup tables are `static const`. No function modifies the input.
- **Pure functions.** Every public filter function is annotated with `SAFE_PURE`. It reads from the input buffer and returns a result struct. There are no side effects, no global state, and no function pointers at runtime.
- **Bounded recursion depth** (ASN.1 only). Protocol headers and standalone ASN.1 are bounded at 8 levels; MMS service bodies at 32 levels. All other functions are non-recursive.
- **Every loop has a proved variant.** Frama-C/WP verifies that every loop terminates.
- **ACSL contracts on every function.** All functions carry pre/postconditions verified by Frama-C/WP with Alt-Ergo and Z3.

## 3 Getting Started

### 3.1 Files

The entire library consists of two files:

- `safe.h` – public API: types, constants, function declarations
- `safe.c` – implementation: all six filters, lookup tables, ACSL annotations

There are no external dependencies. Fixed-width integer types (u8, u32, etc.) are `typedef`'d directly in `safe.h` using `unsigned char` and `unsigned int`, so the library is self-contained on any C11 compiler.

### 3.2 Compilation

```
gcc safe.c my_app.c -o my_app
```

No special flags, no `-l` libraries, no build system required. `safe.c` and `my_app.c` compile together in a single step.

### 3.3 Hello World: Validating a GOOSE Frame

Listing 1: Minimal example: validate a GOOSE frame

```
1 #include "safe.h"
2 #include <stdio.h>
3
4 int main(void) {
5     /* A minimal valid GOOSE frame (from test suite) */
6     u8 frame[128];
7     u32 len = build_goose_frame(frame); /* your code */
8
9     SafeGooseResult r = safe_goose_filter(frame, len);
10
11     if (! r.error) {
12         printf("ACCEPTED: %u data values\n",
13               r.data_value_count);
14     } else {
15         printf("REJECTED: code %u at byte %u\n",
16               r.error, r.reject_at);
17     }
18     return 0;
19 }
```

**Wiring to a real network interface.** Pass the complete Ethernet frame exactly as received from the OS or MAC driver. Using a raw socket on Linux:

Listing 2: Wiring `safe_goose_filter` to a raw Linux socket

```
1 #include "safe.h"
2 #include <sys/socket.h>
3 #include <linux/if_packet.h>
4 #include <net/ethernet.h>
5
6 void rx_loop(int sock) {
7     u8 buf[1600];
8     ssize_t n;
9     while ((n = recv(sock, buf, sizeof(buf), 0)) > 0) {
10         SafeGooseResult r = safe_goose_filter(buf, (u32)n);
11         if (! r.error)
12             forward_to_application(buf, (u32)n);
13         else
14             log_reject("GOOSE", r.error, r.reject_at);
15     }
16 }
```

For `libpcap`, pass `pkthdr->caplen` and `pkt.data` directly to the filter. No header stripping is needed for GOOSE or SV; the filter expects the full frame starting at the destination MAC byte.

### 3.4 Integration Patterns

Three ways to deploy the filters:

1. **Filter-then-parse:** Call the filter first. If it returns OK, pass the buffer to your existing parser (e.g., `libiec61850`). If it rejects, discard the buffer. The existing parser never sees invalid input.
2. **Conditional filter:** Apply filtering only on critical paths (e.g., process-bus GOOSE) while allowing full protocol on engineering ports.
3. **Replace parser:** Use `safe_asn_parse` as the sole ASN.1 parser. It validates structure and extracts zero-copy TLV node pointers into a stack-allocated array (no heap allocation).

Which pattern to use.

- **Protection IED, bay unit, or merging unit:** use **Filter-then-parse** on all process-bus traffic (GOOSE, SV). The filter adds negligible latency ( $< 2\mu s$ ) and ensures the downstream parser never sees malformed frames.
- **RTU or concentrator (DNP3/IEC 104):** use **Filter-then-parse**. No latency budget is tighter than the 3-second poll cycle.
- **New IED firmware with no existing parser:** use **Replace parser** for ASN.1 (`safe_asn_parse`) and the protocol filter for everything else.
- **Gateway or inline appliance enforcing a zone boundary:** use **Filter-then-parse** with a separate instance per protocol on each ingress interface.

### 3.5 Deployment Scenario: Engineering Workstation vs. Operations Zone

The filters are content-only: they inspect PDU bytes and have no awareness of source IP addresses. Per-source policy therefore requires two separate filter instances, each compiled with a different `safe_config.h`.

**Operations profile (default, no extra defines).** SCADA masters and HMI clients use only normal operation services: Read, Write, InformationReport, and connection management. The default `safe_config.h` enables exactly these; no additional `#define` is needed.

**Engineering workstation (EWS) profile.** Commissioning tools additionally need identity queries, data-set management, and file services. Create a second configuration header that extends the default:

```
1 /* safe_config_ews.h -- include before safe.h for EWS instances */
2 #define SAFE_ENABLE_MMS_IDENTIFY
3 #define SAFE_ENABLE_MMS_STATUS
4 #define SAFE_ENABLE_MMS_DELETE_NVL
5 #include "safe_config.h"
```

Note: `fileRename` [75] and `fileDelete` [76] are unconditionally removed from the safe subset in the current `safe.c`; there is no compile-time macro to re-enable them, so they are absent from every profile.

**Network-layer enforcement.** The firewall or inline security appliance routes traffic to the appropriate filter instance based on source IP:

| Source               | Filter instance compiled with        | Action on reject |
|----------------------|--------------------------------------|------------------|
| Engineering WS (/32) | <code>safe_config_ews.h</code>       | drop + log       |
| All other hosts      | <code>safe_config.h</code> (default) | drop + log       |

The EWS filter still enforces all structural checks (tag whitelist, depth, node count, service body). Only the service whitelist is wider. If the EWS source IP is spoofed, the operations-profile filter on the default path still rejects any commissioning-only service code.

**Complete list of service-enable flags.** The default `safe_config.h` disables every risky or rarely-used service; a deployment enables them individually at compile time. Table 2 lists all of them.

Table 2: Compile-time configuration flags for optional services

| Configuration flag                       | Effect when enabled                                      |
|------------------------------------------|----------------------------------------------------------|
| <i>SafeGOOSE optional data values</i>    |                                                          |
| SAFE_ENABLE_GOOSE_MMS_STRING             | Permit mMSString (0x90) in allData                       |
| <i>SafeMMS optional services</i>         |                                                          |
| SAFE_ENABLE_MMS_CANCEL                   | Permit cancelRequest/Response/Error PDU tags [0xA5–0xA7] |
| SAFE_ENABLE_MMS_IDENTIFY                 | Permit Identify [2]                                      |
| SAFE_ENABLE_MMS_STATUS                   | Permit Status [0]                                        |
| SAFE_ENABLE_MMS_DELETE_NVL               | Permit DeleteNamedVariableList [13]                      |
| <i>SafeDNP3 optional function codes</i>  |                                                          |
| SAFE_ENABLE_DNP3_DIRECT_OPERATE          | Permit FC 0x05                                           |
| SAFE_ENABLE_DNP3_DIRECT_OPERATE_NR       | Permit FC 0x06                                           |
| SAFE_ENABLE_DNP3_COUNTER_FREEZE          | Permit FC 0x07–0x0C                                      |
| SAFE_ENABLE_DNP3_COLD_RESTART            | Permit FC 0x0D                                           |
| SAFE_ENABLE_DNP3_WARM_RESTART            | Permit FC 0x0E                                           |
| SAFE_ENABLE_DNP3_DISABLE_UNSOLICITED     | Permit FC 0x15                                           |
| SAFE_ENABLE_DNP3_ASSIGN_CLASS            | Permit FC 0x16                                           |
| SAFE_ENABLE_DNP3_FILE_TRANSFER           | Permit FC 0x19–0x1E                                      |
| <i>SafeDNP3 optional object groups</i>   |                                                          |
| SAFE_ENABLE_DNP3_SECURE_AUTH             | Permit groups 120–122                                    |
| <i>SafeIEC104 optional TypeID groups</i> |                                                          |
| SAFE_ENABLE_IEC104_COMMANDS              | Permit TypeIDs 45–51, 58–64 (command/control direction)  |
| SAFE_ENABLE_IEC104_CLOCK_SYNC            | Permit TypeID 103 (clock synchronisation)                |

## 4 safe.h – Constants, Types, API

The public header defines five protocol modules. Each module provides:

1. A set of `#define` constants for the OK code (always 0) and rejection codes (always nonzero).
2. A result struct whose first field is `error` (0 = accepted) and second field is `reject_at` (byte offset of first violation).
3. A single entry-point function taking `(const uint8_t *buffer, const uint32_t length)`.

### 4.1 SAFE\_PURE and SAFE\_CONST Macros

Listing 3: Compiler hint macros

```
1 /* safe.h */
2 #define SAFE_PURE    __attribute__((pure))
3 #define SAFE_CONST   __attribute__((const)) /* scalar-only functions only */
```

`SAFE_CONST` is retained for functions whose output depends only on scalar arguments (no memory reads), where CSE is acceptable.

Every public filter function is marked `SAFE_PURE` in its declaration, but since the macro is empty the functions compile without any purity attribute attached. They are semantically pure (no side effects, no global state) but the compiler is not told so.

### 4.2 Result Structs

Each protocol defines its own result struct. They all share the same first two fields:

Listing 4: SafeAsnResult – the simplest result struct

```
1 typedef struct {
2     u32 error;
3     u32 reject_at;
4 } SafeAsnResult;
```

`result` is 0 on acceptance (`ASN_OK`), or a nonzero rejection code. On rejection, `reject_at` gives the byte offset of the first offending byte. Protocol-specific structs add extra fields:

Listing 5: Protocol-specific result structs

```
1 typedef struct {
2     u32 error;
3     u32 reject_at;
4     u32 field; /* which GOOSE field failed */
5     u32 data_value_count; /* number of data values parsed */
6 } SafeGooseResult;
7
8 typedef struct {
```

```

9   u32 error;
10  u32 reject_at;
11  u32 asdu_count;      /* number of ASDUs parsed */
12 } SafeSvResult;
13
14 typedef struct {
15     u32 error;
16     u32 reject_at;
17     u32 service_tag;   /* MMS service tag number */
18 } SafeMmsResult;
19
20 typedef struct {
21     u32 error;
22     u32 reject_at;
23     u8  function_code; /* DNP3 FC byte */
24     u32 object_count;  /* number of object headers */
25 } SafeDnp3Result;
26
27 typedef struct {
28     u32 error;
29     u32 reject_at;
30     u8  frame_type;    /* 0=I-frame, 1=S-frame, 2=U-frame */
31     u8  typeid;        /* IEC 104 TypeID (I-frames only) */
32     u32 object_count;  /* number of information objects */
33 } SafeIec104Result;

```

These extra fields give the caller useful diagnostic information even on acceptance. For example, after a successful GOOSE parse, `data_value_count` tells the caller how many data values were inside the `allData` field.

### 4.3 Rejection Codes

Each protocol defines its own rejection codes. When a filter rejects a packet, `error` contains a nonzero code identifying the problem, and `reject_at` identifies the byte offset where the violation was detected.

**Common codes (you will see these often):**

- **REJECT\_SIZE:** The frame is too small (e.g., <22 bytes for GOOSE) or exceeds the protocol maximum.
- **REJECT\_TAG / REJECT\_PDU\_TAG / REJECT\_FIELD\_TAG:** A tag byte is not in the whitelist. This means the packet contains a data type or PDU type that the safe subset does not permit.
- **REJECT\_TRUNCATE:** The packet is incomplete: a length field says “expect *N* bytes” but the frame ends before those bytes.

**Rare codes (usually indicate malformed or adversarial input):**

- **ASN\_REJECT\_INDEF:** Indefinite-length encoding (byte 0x80 in the length position). This is valid BER but forbidden in DER and our safe subset.
- **ASN\_REJECT\_DEPTH:** Nesting exceeds the configured depth limit (8 for protocol headers via `safe_asn_filter`; 32 for MMS service bodies via `asn_filter_top`). Normal IEC 61850 GOOSE/SV messages nest at most 7 levels; MMS service bodies from substation IEDs reach up to depth 28.
- **ASN\_REJECT\_BADINT:** An INTEGER is zero-length, not minimally encoded, or has a length not in {1, 2, 4, 8} (non-machine-word width). *Minimal encoding* means no redundant leading bytes: positive values use no leading 0x00 unless the most-significant bit would otherwise be 1 (which would signal a negative number). For example, value 1 encodes as 02 01 01, not 02 02 00 01; value 128 encodes as 02 02 00 80 (the leading 0x00 is necessary to prevent misreading as -128).
- **DNP3.REJECT\_FUNCTION\_CODE:** A DNP3 function code outside the safe whitelist (e.g., Cold Restart 0x0D).

**Example:** A truncated GOOSE frame:

... 61 10 80 03 41 42 (6 bytes of APDU, but length says 16)

Result: **GOOSE\_REJECT\_TRUNCATE** at the byte where content runs out.

**Using reject\_at for security logging.** `reject_at` is the byte offset of the first violation. Log it with the rejection code so network operators can trace the exact injection point in packet captures:

Listing 6: Logging a rejection with byte offset

```

1 SafeGooseResult r = safe_goose_filter(frame, len);
2 if (r.error) {
3     /* Log to syslog / IEC 62351-8 security event */
4     log_security_event(
5         "GOOSE rejected: code=%u at byte=%u src_mac=%s",
6         r.error, r.reject_at, mac_str(frame + 6));
7     drop_frame(frame);
8 }

```

The same pattern applies to all six filters; each result struct has the same `error` / `reject_at` layout.

Here is the complete table of all rejection codes:

Table 3: All rejection codes across the six filters

| Protocol | Code                                 | Meaning                                                                                                                    |
|----------|--------------------------------------|----------------------------------------------------------------------------------------------------------------------------|
| ASN.1    | ASN_REJECT_TAG (1)                   | Tag byte not in whitelist array                                                                                            |
|          | ASN_REJECT_LENGTH (2)                | Long-form length > 4 bytes                                                                                                 |
|          | ASN_REJECT_DEPTH (3)                 | Nesting exceeds <code>SAFE_ASN_MAX_DEPTH</code> (8)                                                                        |
|          | ASN_REJECT_INDEF (4)                 | Indefinite-length encoding (0x80)                                                                                          |
|          | ASN_REJECT_TRUNCATE (5)              | Content extends past buffer                                                                                                |
|          | ASN_REJECT_BADINT (6)                | INTEGER: zero length, non-minimal encoding, or length not in {1, 2, 4, 8}                                                  |
|          | ASN_REJECT_SIZE (7)                  | Message exceeds 65535 bytes                                                                                                |
|          | ASN_REJECT_CONTENT (8)               | BOOLEAN length $\neq$ 1 or NULL length $\neq$ 0                                                                            |
|          | ASN_REJECT_NULL (9)                  | Null buffer pointer passed to entry point                                                                                  |
|          | ASN_REJECT_TOO_MANY_NODES (10)       | Node count exceeds <code>SAFE_ASN_MAX_NODES</code>                                                                         |
|          | ASN_REJECT_SET_MULTI (11)            | SET contains more than <code>SAFE_MAX_SET_MEMBERS</code> members (default: 1) in relaxed mode. SET is not used in IEC61850 |
| GOOSE    | GOOSE_REJECT_SIZE (1)                | Frame too small or APDU too large                                                                                          |
|          | GOOSE_REJECT_ETH_TYPE (2)            | EtherType not 0x88B8                                                                                                       |
|          | GOOSE_REJECT_HDR_LEN (3)             | Header length exceeds frame                                                                                                |
|          | GOOSE_REJECT_OUTER_TAG (4)           | Missing APPLICATION 1 tag (0x61)                                                                                           |
|          | GOOSE_REJECT_FIELD_TAG (5)           | Unexpected field tag / wrong order                                                                                         |
|          | GOOSE_REJECT_FIELD_SIZE (6)          | Field length out of bounds                                                                                                 |
|          | GOOSE_REJECT_TRUNCATE (7)            | Field extends past APDU                                                                                                    |
|          | GOOSE_REJECT_DATA_VALUE_TAG (8)      | Data value tag not in whitelist array                                                                                      |
|          | GOOSE_REJECT_DATA_VALUE_COUNT (9)    | Data value count > 64 or < 1                                                                                               |
|          | GOOSE_REJECT_DATA_VALUE_PADDING (10) | Padding byte $\neq$ 0 in a bit-string data value                                                                           |
|          | GOOSE_REJECT_TEST (11)               | simulation field (0x87) is TRUE: test/simulation frame (always rejected; unconditional)                                    |
|          | GOOSE_REJECT_DATA_VALUE_LEN (12)     | allData integer (0x85) length not in {1, 2, 4, 8}, or unsigned (0x86) length not in {1, 2, 4}                              |
|          | GOOSE_REJECT_FIELD_VALUE (13)        | Unsigned32 header field (stNum, sqNum, confRev, numDatSetEntries) value outside $[0, 2^{32} - 1]$                          |
|          | GOOSE_REJECT_DATA_VALUE_RANGE (14)   | allData unsigned (0x86) value outside $[0, 2^{32} - 1]$                                                                    |
| SV       | SV_REJECT_SIZE (1)                   | Frame too small or too large                                                                                               |
|          | SV_REJECT_ETH_TYPE (2)               | EtherType not 0x88BA                                                                                                       |
|          | SV_REJECT_HDR_LEN (3)                | Header length exceeds frame                                                                                                |
|          | SV_REJECT_OUTER_TAG (4)              | Missing APPLICATION 0 tag (0x60)                                                                                           |
|          | SV_REJECT_TRUNCATE (5)               | Field extends past APDU                                                                                                    |
|          | SV_REJECT_FIELD_TAG (6)              | Unexpected ASDU field tag                                                                                                  |
|          | SV_REJECT_FIELD_SIZE (7)             | ASDU field length out of bounds                                                                                            |
|          | SV_REJECT_ASDU_TAG (8)               | Expected SEQUENCE (0x30) for ASDU                                                                                          |
|          | SV_REJECT_ASDU_COUNT (9)             | More than 8 ASDUs                                                                                                          |
|          | SV_REJECT_APPID (10)                 | APPID bytes 14–15 outside 0x4000–0x7FFF                                                                                    |
|          | SV_REJECT_CONTENT (11)               | smpSynch (0x85, INT8U) is a reserved value {3, 4, 255}, or smpMod (0x88, INT16U) value > 2                                 |
|          | SV_REJECT_NOASDU (12)                | Declared noASDU count $\neq$ actual ASDU count                                                                             |
|          | SV_REJECT_TRAILING (13)              | Bytes remain inside savPDU after all TLVs consumed                                                                         |
| MMS      | MMS_REJECT_SIZE (1)                  | PDU exceeds 65535 bytes                                                                                                    |
|          | MMS_REJECT_PDU_TAG (2)               | PDU tag not in whitelist array                                                                                             |
|          | MMS_REJECT_TRUNCATE (3)              | PDU extends past buffer                                                                                                    |
|          | MMS_REJECT_INNER_SEQ (4)             | Missing inner SEQUENCE (0x30) in unconfirmed PDU (0xA3)                                                                    |
|          | MMS_REJECT_INVOKE_ID (5)             | Missing/oversized invokeId                                                                                                 |
|          | MMS_REJECT_SERVICE (6)               | Service tag not in whitelist array                                                                                         |
|          | MMS_REJECT_BODY (7)                  | SafeASN.1 rejected service body content                                                                                    |
|          | MMS_REJECT_DER (8)                   | ASN.1 structural (DER) check failed                                                                                        |
| DNP3     | DNP3_REJECT_SIZE (1)                 | Application layer < 2 or > 2048                                                                                            |
|          | DNP3_REJECT_FUNCTION_CODE (2)        | Function code not in whitelist array                                                                                       |
|          | DNP3_REJECT_GROUP (3)                | Object group not in whitelist array                                                                                        |
|          | DNP3_REJECT_VARIATION (4)            | Variation invalid for group                                                                                                |
|          | DNP3_REJECT_QUALIFIER (5)            | Qualifier code invalid                                                                                                     |
|          | DNP3_REJECT_OBJ_COUNT (6)            | More than 64 object headers                                                                                                |

Table 3: All rejection codes (continued)

| Protocol | Code                       | Meaning                                                                     |
|----------|----------------------------|-----------------------------------------------------------------------------|
|          | DNP3_REJECT_TRUNCATE (7)   | Object extends past buffer                                                  |
| IEC 104  | IEC104_REJECT_SIZE (1)     | APDU < 6 bytes or L field > 253                                             |
|          | IEC104_REJECT_START (2)    | Byte 0 is not 0x68                                                          |
|          | IEC104_REJECT_LEN (3)      | L field inconsistent with frame length                                      |
|          | IEC104_REJECT_UFRAME (4)   | U-frame CF1 value not in whitelist                                          |
|          | IEC104_REJECT_TYPEID (5)   | TypeID not in IEC104_TYPEID_WHITELIST, or ASDU too short for header         |
|          | IEC104_REJECT_OBJECTS (6)  | Object count $N$ is 0 or > 127                                              |
|          | IEC104_REJECT_TRUNCATE (7) | Body bytes implied by $N \times \text{ELEM\_SIZE}$ exceed available payload |

## 4.4 Public API Functions

Listing 7: Public API – seven filters (plus a relaxed ASN.1 variant) + one parser

```

1 SAFE_PURE SafeAsnResult safe_asn_filter (const u8 *asn_data, const u32 length);
2 SAFE_PURE SafeAsnResult safe_asn_filter_relaxed(const u8 *asn_data, const u32 length);
3 u32 safe_asn_parse (const u8 *asn_data, const u32 length, SafeAsnParseResult *
  out);
4 SAFE_PURE SafeGooseResult safe_goose_filter(const u8 *ethernet_frame, const u32 length);
5 SAFE_PURE SafeSvResult safe_sv_filter (const u8 *ethernet_frame, const u32 length);
6 SAFE_PURE SafeMmsResult safe_mms_filter (const u8 *mms_pdu, const u32 length);
7 SAFE_PURE SafeDnp3Result safe_dnp3_filter (const u8 *apdu, const u32 length);
8 SAFE_PURE SafeIec104Result safe_iec104_filter(const u8 *apdu, const u32 length);

```

All filter functions are pure: they read from `param[0..length-1]` and return a result struct by value. No side effects, no heap allocation, no global state. `safe_asn_filter_relaxed` is the same strict ASN.1 filter with the `ASN.TAG.WHITELIST_RELAXED` tags (and the contiguous application/context range 0x40–0xBF) additionally accepted, for substation PKI traffic (X.509/OCSP/Kerberos/LDAP); see Section 7. The parser (`safe_asn_parse`) writes zero-copy TLV node pointers into a caller-supplied `SafeAsnParseResult` struct and returns the error code as a `u32` (Section 8).

**What does each parameter contain?** The parameter names describe what the caller must pass:

- **safe\_goose\_filter**, **safe\_sv\_filter** (`ethernet_frame`) — `ethernet_frame[0]` is the first byte of the Ethernet frame (destination MAC address). The 14-byte Ethernet header, EtherType, and protocol header are all present and checked by the filter. If the frame arrives VLAN-tagged (0x8100), strip the 4-byte VLAN tag before calling the filter, so that the EtherType appears at bytes 12–13 as expected. IEEE 802.3 frames shorter than 64 bytes are padded by the MAC driver to the minimum. Both filters determine true payload extent from the APDU length field in the protocol header (bytes 16–17 of the frame), not from `eth_frame_len`; trailing pad bytes beyond the APDU are harmless and are not validated.
- **safe\_mms\_filter** (`mms_pdu`) — `mms_pdu[0]` is the first byte of the MMS PDU. The TCP, TPKT, COTP, ISO Session, and Presentation layers must be stripped before calling the filter. Only the MMS application-layer payload is passed.
- **safe\_dnp3\_filter** (`apdu`) — `apdu[0]` is the Application Control byte of the DNP3 application-layer PDU. The data-link frame (start bytes, CRC, addresses) and the 1-byte transport header must be removed before calling the filter. For multi-segment messages, reassemble all transport frames (FIR to FIN) into a single contiguous buffer before calling the filter.
- **safe\_asn\_filter**, **safe\_asn\_parse** (`asn_data`) — `asn_data[0]` is the first ASN.1/BER tag byte. No network headers of any kind are present.

## 5 Internal Wire-Format Constants

These `#define` constants at the top of `safe.c` describe the wire layout of Ethernet frames, protocol headers, and per-protocol limits. Every constant is an unsigned literal (suffixed with `U`) to avoid signed/unsigned comparison warnings.

Listing 8: Wire-format constants

```

1 #define ETH_HEADER_LENGTH 14U
2 #define ETH_ETHERTYPE_OFFSET 12U
3 #define PROTOCOL_HEADER_LENGTH_OFFSET 16U
4 #define APDU_OFFSET 22U /* ETH(14) + AppId(2)+Len(2)+Res(4) */

```



GOOSE and SV frames share the same Ethernet header structure: a 14-byte Ethernet header (6 bytes destination MAC, 6 bytes source MAC, 2 bytes EtherType), followed by a 2-byte Application Identifier, 2-byte Length, and 4 bytes of reserved fields. The APDU therefore begins at byte offset 22.

Listing 9: ASN.1 limits

```
1 #define SAFE_ASN_MAX_DEPTH      8U    /* safe_config.h -- protocol headers */
2 #define SAFE_ASN_MMS_BODY_MAX_DEPTH 32U /* safe_config.h -- MMS service bodies */
3 #define SAFE_ASN_MAX_MESSAGE    65535U /* safe_config.h */
4 #define ASN_MAX_LENGTH_BYTES    4U    /* safe.h */
5 #define ASN_TAG_UTF8_STRING     0x0CU /* safe.h: UTF8String */
```

`SAFE_ASN_MAX_DEPTH` bounds the recursion depth for protocol headers and standalone ASN.1 (called via `safe_asn_filter`). `SAFE_ASN_MMS_BODY_MAX_DEPTH` is the higher limit used by `asn_filter_top` for MMS service body content: real substation IED traffic nests `InformationReport` data up to depth 28, so  $32 = 28$  (max observed) + 4 (headroom) accommodates all conformant operational traffic. `SAFE_ASN_MAX_MESSAGE` is the largest message the ASN.1 filter will accept (one full `uint16_t` range). `ASN_MAX_LENGTH_BYTES` caps the number of subsequent bytes in a BER long-form length encoding at 4, which is enough to encode lengths up to  $2^{32} - 1$ .

Listing 10: GOOSE limits

```
1 #define GOOSE_MIN_FRAME        22U    /* safe.h */
2 #define SAFE_GOOSE_MAX_APDU    1400U  /* safe_config.h */
3 #define GOOSE_OUTER_TAG        0x61U  /* safe.h: APPLICATION 1 */
4 #define GOOSE_DATA_MMS_STRING_TAG 0x90U /* safe.h: allData mms-string (opt-in) */
```

**All bounds are tunable.** Every numeric bound in `safe_config.h` is wrapped in an `#ifndef` guard, so you can tighten or relax any value by defining it before the `#include`:

Listing 11: Overriding a bound per deployment

```
1 /* tighten GOOSE data depth for this SCL deployment */
2 #define SAFE_GOOSE_MAX_DATA_DEPTH 2U
3 #include "safe_config.h"
4 #include "safe.h"
```

**Warning.** The default values in `safe_config.h` define the safe protocol subsets (SafeASN.1, SafeGOOSE, SafeSV, SafeMMS, SafeDNP3, SafeIEC104). Changes to these values are for exceptional circumstances only and may cause the filter to accept messages outside the validated subset, making it incompatible with the corresponding Safe protocol specification. Re-run Frama-C/WP and the full test suite after any modification.

`GOOSE_MIN_FRAME` is the minimum frame size (must at least contain the Ethernet and protocol headers). `SAFE_GOOSE_MAX_APDU` caps the APDU at 1400 bytes (the IEC 61850 practical maximum for a single Ethernet frame). `GOOSE_OUTER_TAG` is the `APPLICATION 1` constructed tag (0x61) that wraps the GOOSE PDU body. `GOOSE_DATA_MMS_STRING_TAG` (0x90) is the `allData mms-string` tag; it is accepted only when `SAFE_ENABLE_GOOSE_MMS_STRING` is defined (off by default).

Listing 12: SV limits

```
1 #define SAFE_SV_MIN_FRAME      22U    /* safe.h */
2 #define SAFE_SV_MAX_FRAME      1600U  /* safe_config.h */
3 #define SV_OUTER_TAG           0x60U  /* APPLICATION 0 */
4 #define SV_NOASDU_TAG          0x80U
5 #define SV_SEQASDU_TAG         0xA2U
6 #define SV_ASDU_SEQ_TAG        0x30U
```

SV frames are similar to GOOSE but use a maximum frame size of 1600 bytes and a different outer tag (`APPLICATION 0 = 0x60`). The three inner tags identify the `noASDU` count field (0x80), the `seqASDU` sequence wrapper (0xA2), and the `SEQUENCE` tag (0x30) expected at the start of each individual ASDU.

Listing 13: MMS and DNP3 limits

```
1 #define SAFE_MMS_MAX_PDU       65535U /* safe_config.h */
2 #define SAFE_ASN_MMS_BODY_MAX_DEPTH 32U /* safe_config.h */
3
4 #define SAFE_DNP3_MIN_APP      2U
5 #define SAFE_DNP3_MAX_APP      2048U
6 #define SAFE_DNP3_MAX_OBJECTS  64U
```

MMS operates over TCP/TPKT, so its PDUs can be up to 65535 bytes. The `safe_mms_filter` calls `asn_filter_top` (not `safe_asn_filter`) on the service body content, using `SAFE_ASN_MMS_BODY_MAX_DEPTH = 32` instead of `SAFE_ASN_MAX_DEPTH = 8` to accommodate the deep nesting in real IED traffic. DNP3 application-layer fragments are at least 2 bytes (one control byte and one function code byte) and at most 2048 bytes. The filter will process up to 64 object headers per fragment.

## 6 Shared BER Length Decoder: `read_length`

Before we dive into individual filters, let us look at the one shared helper function. Every filter that handles BER-encoded data needs to decode length fields, and they all call `read_length`.

## 6.1 The Complete Function

### 6.1.1 What read\_length Does (Plain English)

This function decodes a BER length field from the buffer at the current offset and advances the offset past it. It is shared by the ASN.1-based filters (SafeASN.1, SafeGOOSE, SafeSV, SafeMMS).

- **Input:** `buffer` (byte array), `buffer_length` (size), `*offset` (current read position).
- **Output:** Returns the decoded length as a `uint32_t`. Advances `*offset` past the length bytes.
- **Short form** (first byte  $< 0x80$ ): The byte itself is the length. Example: `0x07` means 7 bytes of content.
- **Long form** (first byte `0x81–0x84`): The low bits say how many subsequent bytes encode the length. Example: `0x82 01 00` means “2 length bytes follow” → length = 256.
- **Invalid** (first byte `0x80` or  $> 0x84$ ): Returns 0 and does not advance offset. The caller treats 0 as an error.

### 6.1.2 The Formal Contract

The ACSL contract below formally specifies the guarantees. The `requires` clauses state what the caller must provide; the `ensures` clauses state what the function guarantees:

Listing 14: `read_length` – BER definite-length decoder

```
1  /*@
2  requires \valid_read(buffer + (0 .. buffer_length - 1));
3  requires \valid(offset);
4  requires \separated(offset, buffer + (0 .. buffer_length - 1));
5  requires *offset <= buffer_length;
6  requires buffer_length <= 65535;
7
8  assigns *offset;
9  ensures *offset <= buffer_length;
10 ensures *offset >= \old(*offset);
11 */
12 static u32 read_length(
13     const u8 *const buffer,
14     const u32 buffer_length,
15     const u32 *const offset)
16 {
17     /*@ ghost u32 offset_init = *offset;
18
19     if (*offset >= buffer_length)
20         return 0U;
21
22     const u8 first = buffer[(*offset)++];
23
24     /*@ assert *offset >= offset_init; */
25
26     /* Short form: bit 7 clear, byte is the length directly */
27     if ((first & 0x80U) == 0U)
28         return (u32) first;
29
30     /* Long form: need at least one more byte */
31     if (*offset >= buffer_length)
32         return 0U;
33
34     const u32 num_length_bytes = (u32)(first & 0x7FU);
35
36     /* Reject indefinite-length (0x80 -> num_length_bytes==0)
37        and limit to 4 length bytes (defensive) */
38     if (num_length_bytes == 0U || num_length_bytes > 4U)
39         return 0U;
40
41     /* Reject non-minimal encoding: a leading zero byte means
42        the length could have been expressed in fewer bytes */
43     if (num_length_bytes > 1U && buffer[*offset] == 0x00U)
44         return 0U;
45
46     /* Overflow-safe check: buffer must hold all length bytes */
47     const U32 bytes_required = ADD(*offset, num_length_bytes);
48     if (bytes_required.overflowed || bytes_required.value > buffer_length)
49         return 0U;
50
51     u32 length = 0U;
52
53     /*@
54     loop invariant 0 <= i <= num_length_bytes;
55     loop invariant *offset <= buffer_length;
56     loop invariant *offset >= offset_init;
57     loop assigns i, length, *offset;
58     loop variant num_length_bytes - i;
59     */
60     for (u32 i = 0U;
61         (i < num_length_bytes) && (*offset < buffer_length);
62         ++i)
63     {
64         length = (length << 8U) | (u32)buffer[*offset];
65         (*offset)++;
66     }
67
68     /*@ assert *offset >= offset_init; */
69 }
```



```

70 |     return length;
71 | }

```

## 6.2 ACSL Contract – Line by Line

- **requires** `\valid_read(buffer + (0 .. buffer_length - 1))` – the caller promises that the entire buffer is readable. This is how Frama-C knows every `buffer[*offset]` access in the function body is safe.
- **requires** `\valid(offset)` – the offset pointer itself is valid (we can read and write through it).
- **requires** `\separated(offset, buffer + (...))` – the offset pointer does not alias any byte of the buffer. This is critical: without it, writing to `*offset` could change the buffer contents, and the prover could not reason about buffer reads after offset updates.
- **requires** `*offset <= buffer_length` – we start within bounds.
- **requires** `buffer_length <= 65535` – caps the buffer size so arithmetic cannot overflow `uint32_t`.
- **assigns** `*offset` – the function modifies only the offset. It does not modify the buffer or any global state.
- **ensures** `*offset <= buffer_length` – offset stays in bounds after the call.
- **ensures** `*offset >= \old(*offset)` – offset never moves backward. This is essential for callers that rely on monotonic progress to guarantee loop termination.

## 6.3 How the Decoding Works

BER length encoding comes in two forms:

1. **Short form.** Bit 7 is clear. The byte itself is the length (0–127).
2. **Long form.** Bit 7 is set. The low 7 bits give the number of subsequent bytes that hold the length in big-endian. We allow at most 4 length bytes (enough for lengths up to  $2^{32} - 1$ ). If `num_length_bytes > 4U`, we return 0 and let the caller detect the problem via its own bounds check.

Indefinite-length encoding is rejected inside `read_length` itself: after stripping the long-form flag bit, a residual value of zero means the original byte was exactly `0x80`, the BER indefinite-length marker. The check `num_length_bytes == 0U` catches this and returns 0. Non-minimal encoding (a long-form length that could be expressed in fewer bytes) is caught by checking for a leading zero byte. An overflow-safe `ADD()` call verifies that the buffer holds all announced length bytes before the accumulation loop runs.

## 6.4 Ghost Variable and Loop Annotations

The ghost variable `offset_init` captures the initial offset for use in the loop invariant and the trailing assertion. It exists only in the specification world; it is not compiled into the binary.

The loop has five annotations:

1. **loop invariant** `0 <= i <= num_length_bytes` – the loop counter is bounded.
2. **loop invariant** `*offset <= buffer_length` – the offset stays within the buffer at all times.
3. **loop invariant** `*offset >= offset_init` – the offset never goes below its initial value (monotonicity).
4. **loop assigns** `i, length, *offset` – only these three variables change in the loop body.
5. **loop variant** `num_length_bytes - i` – this expression is non-negative and strictly decreases with each iteration, proving termination.

# 7 SafeASN.1 – Whitelist Array + Recursive TLV Walker

## 7.1 What is ASN.1 and Why Does It Matter?

ASN.1 (Abstract Syntax Notation One) is a way to encode structured data as binary bytes. Think of it as binary JSON: where JSON uses `{"name": "Alice", "age": 30}`, ASN.1 uses bytes like `30 0B 0C 05 41 6C 69 63 65 02 01 1E`.

Every value is encoded as a **TLV triplet**: Tag (what type?), Length (how many bytes?), Value (the content). For example, the integer 42 is encoded as:

|              |                |           |
|--------------|----------------|-----------|
| 02           | 01             | 2A        |
| Tag: INTEGER | Length: 1 byte | Value: 42 |

**BER** (Basic Encoding Rules) is the flexible encoding: it allows multiple valid ways to encode the same value. **DER** (Distinguished Encoding Rules) is stricter: exactly one valid encoding per value. Our filter requires DER.

**Why filtering matters:** ASN.1 parsers have a long history of security vulnerabilities. The flexible encoding rules of BER create a large attack surface: indefinite-length fields, optional fields, recursive nesting, and open types all give an attacker room to craft inputs that confuse or crash the parser. The SafeASN.1 filter rejects these dangerous constructs before any parser sees them.

## 7.2 Worked Example: Decoding a DER Message

Consider this 9-byte DER message (a SEQUENCE containing an INTEGER and an OCTET STRING):

30 07 02 01 2A 04 02 68 69

The filter processes it byte by byte:

1. **Offset 0:** Tag = 0x30 (SEQUENCE). Whitelist lookup: `ASN_TAG_WHITELIST[0x30] == 1` → accepted. SEQUENCE is a constructed type → will recurse into children.
2. **Offset 1:** Length = 0x07 (short form, 7 bytes of content). Content region: bytes 2–8.
3. **Recurse into SEQUENCE children (depth 1):**
  - (a) **Offset 2:** Tag = 0x02 (INTEGER). Whitelist: accepted.
  - (b) **Offset 3:** Length = 0x01 (1 byte).
  - (c) **Offset 4:** Content = 0x2A (42). Minimal encoding check: length > 0, no leading zero followed by a byte with bit 7 clear → pass.
  - (d) **Offset 5:** Tag = 0x04 (OCTET STRING). Whitelist: accepted.
  - (e) **Offset 6:** Length = 0x02 (2 bytes).
  - (f) **Offset 7–8:** Content = 68 69 (“hi”). Advance past.
4. **Offset 9:** End of SEQUENCE content. All children valid.
5. **Result:** ASN\_OK.

### 7.3 Implementation

The ASN.1 filter validates that a DER-encoded message uses only an approved set of tag types, has properly encoded lengths, has minimally encoded INTEGERS, and does not nest more than 8 levels deep (or 32 for MMS service bodies, via `asn_filter_top`).

### 7.4 BER/DER TLV Wire Format

Every ASN.1 value is encoded as a Tag–Length–Value (TLV) triplet. The filter walks these triplets left-to-right, recursing into constructed types (SEQUENCE).

**TLV structure** (every ASN.1 value):

| Tag    | Length    | Value (content) |
|--------|-----------|-----------------|
| 1 byte | 1–5 bytes | 0–65 535 bytes  |

**Tag byte layout** (bits 7–0):

| b7                     | b6 | b5            | b4                                  | b3 | b2 | b1 | b0 |
|------------------------|----|---------------|-------------------------------------|----|----|----|----|
| Class                  |    | P/C           | Tag number                          |    |    |    |    |
| 00=Univ, 01=App        |    | 0=Primitive   | 0x01 = BOOLEAN, 0x02 = INTEGER, ... |    |    |    |    |
| 10=Context, 11=Private |    | 1=Constructed | 0x30 = SEQUENCE, 0x31 = SET         |    |    |    |    |

*P/C (Primitive/Constructed): 0 means the value bytes are raw data (leaf node, no recursion); 1 means the value bytes contain nested TLV elements — `asn_filter_recursive` recurses into them. SEQUENCE (0x30) is the only constructed type accepted by the strict filter. SET (0x31) is additionally accepted by `safe_asn_filter_relaxed` for X.509 parsing; the number of members is limited to `SAFE_MAX_SET_MEMBERS` (default: 1, matching RFC 5280 RDNs; raise in `safe_config.h` if multi-valued RDNs are required).*

*Length encoding: single byte if < 0x80; multi-byte if 0x81–0x84 (1–4 following bytes); 0x80 (indefinite-length) is **rejected**.*

### 7.5 ASN\_TAG\_WHITELIST – The Whitelist Array

Listing 15: ASN\_TAG\_WHITELIST – direct whitelist array (192 entries)

```

1 #define ASN_TAG_WHITELIST_SIZE 192U
2
3 static const u8 ASN_TAG_WHITELIST[ASN_TAG_WHITELIST_SIZE] = {
4     [0x01] = 1, /* BOOLEAN */           [0x02] = 1, /* INTEGER */
5     [0x03] = 1, /* BIT STRING */        [0x04] = 1, /* OCTET STRING */
6     [0x05] = 1, /* NULL */              [0x06] = 1, /* OBJECT IDENTIFIER */
7     [0x0C] = 1, /* UTF8String */        [0x16] = 1, /* IA5String */
8 #ifdef SAFE_ENABLE_ASN_VISIBLE_STRING
9     [0x1A] = 1, /* VisibleString */      /* MMS filenames; enabled by default */
10 #endif
11     [0x30] = 1, /* SEQUENCE */
12 };
13
14 /* Tags accepted by safe_asn_filter_relaxed but not by safe_asn_filter. */
15 static const u8 ASN_TAG_WHITELIST_RELAXED[ASN_TAG_WHITELIST_SIZE] = {
16     [0x0A] = 1, /* ENUMERATED (OCSP response status) */
17     [0x13] = 1, /* PrintableString (X.509 DNs) */
18     [0x17] = 1, /* UTCTime (X.509 validity dates <= 2049) */
19     [0x18] = 1, /* GeneralizedTime (X.509 validity dates > 2049) */
20     [0x1B] = 1, /* GeneralString (Kerberos) */
21     [0x31] = 1, /* SET (X.509 DNs; single member) */
22     /* 0x40–0xBF Application/Context: passed unconditionally by relaxed filter. */
23 };

```

This is a 192-entry array where `ASN_TAG_WHITELIST[tag] == 1` means the tag is allowed. All uninitialized entries default to 0, which means reject. The check is simple:

Listing 16: Accept check pattern

```

1 if (tag >= ASN_TAG_WHITELIST_SIZE || !ASN_TAG_WHITELIST[tag])
2     return ASN_REJECT_TAG;

```

The bounds check `tag >= ASN_TAG_WHITELIST_SIZE` ensures that tags `0xC0–0xFF` are automatically rejected because they exceed the array size. These are *private-class* tags (bits 7:6 = 11): vendor-specific extensions with no defined structure in any public standard. Neither IEC 61850 nor X.509 defines any private-class tag, so there is no legitimate traffic that requires accepting them (see Table 5 for full rationale).

Table 4: ASN.1 tag whitelist: accepted and rejected tags

| Tag       | Name                  | Status                                                                                           |
|-----------|-----------------------|--------------------------------------------------------------------------------------------------|
| 0x00      | EOC (End-of-Contents) | <b>rejected</b> (terminator for indefinite-length BER encoding)                                  |
| 0x01      | BOOLEAN               | accepted                                                                                         |
| 0x02      | INTEGER               | accepted (content validated for minimal encoding)                                                |
| 0x03      | BIT STRING            | accepted                                                                                         |
| 0x04      | OCTET STRING          | accepted                                                                                         |
| 0x05      | NULL                  | accepted                                                                                         |
| 0x06      | OBJECT IDENTIFIER     | accepted                                                                                         |
| 0x0A      | ENUMERATED            | <b>relaxed only</b> (OCSP response status; rejected by strict filter)                            |
| 0x0C      | UTF8String            | accepted                                                                                         |
| 0x13      | PrintableString       | <b>relaxed only</b> (X.509 issuer/subject names; rejected by strict filter)                      |
| 0x16      | IA5String             | accepted                                                                                         |
| 0x17      | UTCTime               | <b>relaxed only</b> (X.509 validity dates $\leq 2049$ ; rejected by strict filter)               |
| 0x1A      | VisibleString         | accepted if <code>SAFE_ENABLE_ASN_VISIBLE_STRING</code> (MMS filenames)                          |
| 0x30      | SEQUENCE              | accepted (recurse into children)                                                                 |
| 0x31      | SET                   | <b>relaxed mode only</b> (X.509 DNs; up to <code>SAFE_MAX_SET_MEMBERS</code> members, default 1) |
| 0x40–0x7F | Application           | <b>relaxed only</b> (rejected by strict filter)                                                  |
| 0x80–0xBF | Context-specific      | <b>relaxed only</b> and MMS body via <code>ASN_FLAG_ALLOW_CONTEXT</code>                         |
| 0xC0–0xFF | Private class         | <b>rejected</b> (vendor-specific; no defined structure in IEC 61850)                             |

## 7.6 Blocked ASN.1 tags: rationale

The strict filter (`safe_asn_filter`) accepts 10 universal tags by default (up to 12 with `SAFE_ENABLE_ASN_UTCTIME` and `SAFE_ENABLE_ASN_VISIBLE_STRING`). Application (0x40–0x7F), context-specific (0x80–0xBF), PrintableString (0x13), and SET (0x31) are *not* in the whitelist array and are rejected by the strict filter. The relaxed filter (`safe_asn_filter_relaxed`) additionally accepts the entire 0x40–0xBF range plus 0x13, 0x18, 0x1B, and 0x31 for X.509/Kerberos certificate parsing. `safe_mms_filter` passes `ASN_FLAG_ALLOW_CONTEXT` to the recursive walker so that MMS service body fields (context-specific 0x80–0xBF) are accepted while the strict universal-class whitelist is still enforced. The table below explains why the remaining tag space is blocked.

Table 5: Blocked ASN.1 tags and rationale

| Tag  | Name                  | Reason blocked                                                                                                                                                                                                                                                                                                                                                                                               |
|------|-----------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 0x00 | EOC (End-of-Contents) | The EOC tag (0x00, length 0) is the terminator for indefinite-length BER encoding. SafeASN requires DER (definite lengths only). Accepting EOC would require supporting indefinite-length parsing, which introduces a second, more complex length-decoding code path. CVE-2006-2937 (OpenSSL) and multiple other vulnerabilities stem from indefinite-length edge cases. Rejected at the whitelist boundary. |
| 0x07 | OBJECT_DESCRIPTOR     | A character string type not used in IEC 61850 or X.509. No defined use in the target protocols; accepting it expands the attack surface of the string-handling path.                                                                                                                                                                                                                                         |
| 0x08 | EXTERNAL              | EXTERNAL/EmbeddedPDV encapsulates an arbitrary external encoding inside an ASN.1 structure. Accepting EXTERNAL would allow an attacker to embed an opaque binary blob inside a valid ASN.1 frame that passes structural validation. The blob is then presented as legitimate content to the application layer.                                                                                               |

Table 5: Blocked ASN.1 tags (continued)

| Tag                     | Name                                                    | Reason blocked                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |
|-------------------------|---------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 0x09                    | REAL                                                    | Floating-point BER encoding with three distinct forms (binary, decimal, special values). The decimal-encoding form is variable-length text-encoded floating point – an unusual code path not exercised by any IEC 61850 or X.509 data type. Reject in favour of IEC 61850 floating-point in GOOSE (tag 0x87, context-specific).                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |
| 0x0B                    | EMBEDDED PDV                                            | Same reasoning as EXTERNAL (tag 0x08): encapsulates opaque content. Not used in target protocols.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
| 0x0D                    | RELATIVE-OID                                            | A relative Object Identifier. Not used in IEC 61850 or X.509 certificates. No established use case in substation communication.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |
| 0x0E, 0x0F              | TIME / DATE                                             | ASN.1 2008 time types with complex variable-length encoding rules. Not used in IEC 61850 (which uses CP56Time2a in GOOSE/SV, not BER-encoded time). Accepting these types would require a separate length validation path.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
| 0x11–0x15,<br>0x17–0x29 | String types (T61, VideotexString, GraphicString, etc.) | Only UTF8String (0x0C) and IA5String (0x16) are used in IEC 61850 and X.509 common names. The remaining string types (NumericString, PrintableString, TeletexString, VisibleString, GeneralString, UniversalString, CharacterString, BMPString) are obsolete, ambiguous in their character encoding, or not used by the target protocols. Accepting them widens the string-handling attack surface without operational benefit.                                                                                                                                                                                                                                                                                                                                                                                                                                                   |
| 0xC0–0xFF               | PRIVATE class                                           | The private-class tag range (bits 7:6 = 11) is used for vendor-specific extensions. Private tags are rejected because their structure and length encoding are undefined in any public standard. An attacker can craft arbitrary content inside a private-class TLV and be confident that no standard decoder will flag it – it is a general-purpose opaque carrier. The array bounds check ( <code>tag &gt;= sizeof(ASN_TAG_WHITELIST)</code> with the array sized at 192 = 0xC0) automatically rejects all private-class tags without any explicit check. Operators who must interoperate with non-standard equipment that uses specific private tags can extend <code>ASN_TAG_WHITELIST_SIZE</code> to 256 and set the required entries to 1 in <code>ASN_TAG_WHITELIST</code> ; this is a deliberate per-deployment decision to widen the safe subset for those specific tags. |

## 7.7 asn\_filter\_recursive – The Recursive TLV Walker

This is the heart of the ASN.1 filter. It walks sibling TLV (Tag-Length-Value) elements in `buffer[offset..end)` and recurses into SEQUENCE children.

### 7.7.1 ACSL Contract

Listing 17: `asn_filter_recursive` contract

```

1  /*@
2  requires \valid_read(buffer + (0 .. buffer_length - 1));
3  requires offset <= end <= buffer_length;
4  requires buffer_length <= 65535;
5  requires depth <= max_depth;
6  requires max_depth <= SAFE_ASN_MMS_BODY_MAX_DEPTH;
7  requires \valid(node_count);
8  requires *node_count <= SAFE_ASN_MAX_NODES;
9  decreases end - offset;
10 assigns *node_count;
11 ensures *node_count <= SAFE_ASN_MAX_NODES;
12 ensures *node_count >= \old(*node_count);
13 */
14 static SafeAsnResult asn_filter_recursive(
15     const u8 *const buffer,
16     const u32 buffer_length,
17     const u32 offset,

```

```

18  const    u32          end,
19  const    u32          depth,
20  const    u32          max_depth,
21  const    u32          *const node_count,
22  const    u32          flags)

```

Let us read each clause:

- **requires** `offset <= end <= buffer_length` – we walk a sub-range of the buffer. The offset starts at the beginning of the sub-range, and `end` marks the boundary we must not cross.
- **requires** `depth <= max_depth` – recursion depth is bounded by the caller-supplied limit. `safe_asn_filter` passes `SAFE_ASN_MAX_DEPTH = 8`; `asn_filter_top` passes `SAFE_ASN_MMS_BODY_MAX_DEPTH = 32`.
- **requires** `max_depth <= SAFE_ASN_MMS_BODY_MAX_DEPTH` – the upper bound on any depth limit, ensuring the recursion stack is always bounded.
- **decreases** `end - offset` – this is the termination measure for the recursive function. Each recursive call operates on a strictly smaller range (the child content is always a proper subset of the parent range), so this value decreases with every call. Frama-C/WP uses this to prove that the recursion terminates.
- **assigns** `*node_count` – the function increments the shared node counter once per TLV visited (both filter and parser use the same counter to enforce the `SAFE_ASN_MAX_NODES` cap). The `offset` parameter is passed by value, so advancing it inside the function does not modify any caller-visible state. No other memory is written.

The `flags` parameter controls per-call policy overrides. Currently one flag is defined: `ASN_FLAG_ALLOW_ZERO_PRIMITIVE` (bit 0). When set, primitive context-specific tags with length 0 are accepted; MMS uses these for `IMPLICIT NULL` choices (e.g. `ObjectScope::vmdSpecific [0]` encodes as `80 00`). The flag must not be set for GOOSE, SV, or standalone ASN.1 parsing, where zero-length primitive fields indicate the class of vulnerability reported by IEDFuRL. Callers propagate `flags` unchanged into every recursive call.

### 7.7.2 Ghost Variables and Loop Annotations

Listing 18: Main loop of the recursive walker

```

1  /*@ ghost u32 offset_entry = offset; */
2  /*@ ghost u32 nc_entry    = *node_count; */
3
4  /*@
5   loop invariant offset      <= end;
6   loop invariant end        <= buffer_length;
7   loop invariant offset     >= offset_entry;
8   loop invariant *node_count <= SAFE_ASN_MAX_NODES;
9   loop invariant *node_count >= nc_entry;
10  loop assigns  offset, *node_count;
11  loop variant  end - offset;
12  */
13  while (offset < end)
14  {
15      /*@ ghost u32 offset_top = offset;
16      ...
17      /*@ assert offset > offset_top; */
18  }

```

Three ghost variables are used:

1. `offset_entry` captures the offset at function entry. The loop invariant `offset >= offset_entry` tells the prover that the offset never moves backward across loop iterations.
2. `nc_entry` captures `*node_count` at function entry. The loop invariant `*node_count >= nc_entry` tells the prover the counter never decreases.
3. `offset_top` captures the offset at the start of each loop iteration. The assertion `offset > offset_top` at the bottom confirms that every iteration makes strict forward progress. This is what proves the loop variant `end - offset` strictly decreases.

### 7.7.3 Algorithm, Step by Step

Each loop iteration processes one sibling TLV element:

1. **Tag check.** Read the tag byte. Look it up in `ASN_TAG_WHITELIST`. If the tag is out of range or the entry is 0, return `ASN_REJECT_TAG`.
2. **Multi-byte tag.** If the low 5 bits of the tag byte are all 1s (`0x1F` mask), BER uses a multi-byte “high-tag” encoding for tag numbers greater than 30. The tag number is encoded in one or more *continuation bytes* that follow the initial tag byte: each continuation byte has bit 7 set (meaning “more bytes follow”); the final byte has bit 7 clear (meaning “this is the last byte”). For example, MMS service 77 (`fileDirectory`) is encoded as a single continuation byte `0x4D` (77 decimal, bit 7 = 0, so it terminates immediately). `SAFE_MAX_CONTINUATION_BYTES = 3` is the rejection threshold: the loop counter (`continuation_bytes`) counts every extension byte, including the terminator, so reaching 3 causes rejection regardless of whether a terminator was found. In practice this permits at most **two** extension bytes, covering tag numbers 0–16,383 ( $2^{14} - 1$ ). No IEC 61850 context tag exceeds approximately 77 (one extension byte suffices for all known services), so the limit is ample. The inner loop has its own ACSL annotations:

Listing 19: Multi-byte tag continuation loop

```

1  /*@

```



```

2      loop invariant offset <= buffer_length;
3      loop invariant offset > offset_top;
4      loop invariant 0 <= continuation_bytes <= SAFE_MAX_CONTINUATION_BYTES;
5      loop assigns   offset, continuation_bytes;
6      loop variant   (int)(SAFE_MAX_CONTINUATION_BYTES - continuation_bytes);
7
8  /*
9  while (offset < buffer_length &&
10         continuation_bytes < SAFE_MAX_CONTINUATION_BYTES)
11  {
12      const u8 b = buffer[offset];
13      offset++;
14      continuation_bytes++;
15      if ((b & 0x80U) == 0U) break;
16  }

```

After the high-tag loop, if `offset > end` (the tag header bytes extend past the current TLV boundary), return `ASN_REJECT_TRUNCATE`. The assertion `/*@ assert offset <= end; */` after this check is a prover cut-point that re-establishes the loop invariant for subsequent goal generation.

3. **Length byte.** Read the first length byte. If it is `0x80` (indefinite length), return `ASN_REJECT_INDEF`: DER requires definite lengths.
4. **Short-form length.** If the high bit is clear, the byte itself is the length.
5. **Long-form length.** If the high bit is set, the low 7 bits give `num_length_bytes`. If this exceeds `ASN_MAX_LENGTH_BYTES` (4), return `ASN_REJECT_LENGTH`. Otherwise, read that many bytes in big-endian order. The inner loop has its own annotations:

Listing 20: Long-form length decoding loop

```

1  /*@
2      loop invariant 0 <= i <= num_length_bytes;
3      loop invariant offset <= buffer_length;
4      loop invariant offset > offset_top;
5      loop assigns   i, length, offset;
6      loop variant   num_length_bytes - i;
7
8  /*
9  for (u32 i = 0;
10      (i < num_length_bytes) && (offset < buffer_length);
11      ++i)
12  {
13      length = (length << 8U) | (u32)buffer[offset];
14      offset++;
15  }

```

After the length loop, if `offset > end` (the length header bytes extend past the current TLV boundary), return `ASN_REJECT_TRUNCATE`. Again, `/*@ assert offset <= end; */` re-establishes the invariant as a prover cut-point.

6. **Size check.** If `length > ASN_MAX_MESSAGE` (65535), return `ASN_REJECT_SIZE`.
7. **Bounds check.** If `length > end` or `offset + length > end`, the content extends past the current element's boundary. Return `ASN_REJECT_TRUNCATE`. Two inline assertions help the prover:

```

1  /*@ assert offset + length <= end; */
2  /*@ assert offset + length <= buffer_length; */

```

8. **INTEGER validation.** If `tag == 0x02U`, validate minimal encoding per DER rules:
  - Zero-length INTEGER: return `ASN_REJECT_BADINT`.
  - Leading `0x00` when the next byte has bit 7 clear (unnecessary positive padding): return `ASN_REJECT_BADINT`.
  - Leading `0xFF` when the next byte has bit 7 set (unnecessary negative padding): return `ASN_REJECT_BADINT`.
9. **SEQUENCE recursion.** If `tag == 0x30U`, recurse into the content bytes `[offset, offset+length)` with `depth + 1U`. If `depth >= max_depth`, return `ASN_REJECT_DEPTH` instead.
10. **Advance.** Skip past the content: `offset += length`.

#### 7.7.4 Termination Proof

Each loop iteration advances `offset` by at least 2 bytes (one for the tag, one for the length byte). The assertion `offset > offset_top` confirms strict progress, so the loop variant `end - offset` strictly decreases. For SEQUENCE recursion, the sub-range `[offset, offset+length)` is strictly smaller than `[offset_entry, end)`, so the decreases `end - offset` measure decreases across recursive calls.

## 7.8 safe\_asn\_filter – Entry Point

Listing 21: `safe_asn_filter` – ASN.1 filter entry point

```

1  /*@
2      requires length <= 65535;
3      requires length == 0 || \valid_read(asn_data + (0 .. length - 1));
4      assigns   \nothing;
5
6      ensures   asn_data != \null &&
7                length > SAFE_ASN_MAX_MESSAGE
8                ==> \result.error == ASN_REJECT_SIZE;
9
10     ensures   asn_data != \null &&
11                length > 0

```

```

12         length <= SAFE_ASN_MAX_MESSAGE OK
13         asn_data[0] < ASN_TAG_WHITELIST_SIZE OK
14         ASN_TAG_WHITELIST[asn_data[0]] == 0
15         ==> \result.error != ASN_OK;
16     */
17     SAFE_PURE SafeAsnResult safe_asn_filter(
18         const u8 * const asn_data,
19         const u32 length)

```

### 7.8.1 Contract

- **Requires:** `length <= 65535`; `asn_data` readable if nonempty.
- **Assigns:** nothing.
- **First ensures:** oversized messages yield `ASN_REJECT_SIZE`. This is a functional correctness guarantee: any caller can rely on the fact that a message longer than 65535 bytes will always be rejected with exactly this code.
- **Second ensures:** if the message is non-empty, within the size limit, and the first tag byte maps to 0 in `ASN_TAG_WHITELIST`, the result is guaranteed to be non-OK. This ensures that a bad leading tag is always caught.
- **Accepts:** empty `asn_data` (vacuously valid; length 0 returns OK).

### 7.8.2 Behavior

1. If `length == 0U`, return OK (empty message is vacuously valid).
2. If `length > ASN_MAX_MESSAGE`, set `result = ASN_REJECT_SIZE` and return.
3. Pre-check: if `asn_data[0]` is not in the whitelist array, set `result = ASN_REJECT_TAG` and return.
4. Delegate to `asn_filter_top(asn_data, length, SAFE_ASN_MAX_DEPTH)` – performs null/size/tag pre-checks then calls `asn_filter_recursive` at offset 0, depth 0, depth limit `SAFE_ASN_MAX_DEPTH = 8`.
5. Return the result code from the recursive walker.

## 7.9 All 10 validation checks: what, why, and what error code

### 7.9.1 Check 1 – Null pointer (`ASN_REJECT_NULL`)

**What is checked.** If `asn_data` is NULL, return `ASN_REJECT_NULL` immediately before any dereference.

**Why.** The ACSL contract requires `length == 0 || \valid_read(asn_data + (0 .. length-1))` but a defensive null check at the C level prevents a crash in environments that do not enforce the precondition at compile time.

**Attack prevented.** A caller that forwards a null pointer from a malformed packet header gets a clean error code instead of a segmentation fault.

### 7.9.2 Check 2 – Message size (`ASN_REJECT_SIZE`)

**What is checked.** `length > ASN_MAX_MESSAGE` (65 535) is rejected before any byte is accessed.

**Why.** BER uses a 2-byte definite length field at the top level (as imposed by the TPKT transport in MMS). A length field above 65 535 cannot be represented in 2 bytes and is therefore malformed. Capping the message size here also bounds all downstream arithmetic on `offset` and `length` to values that fit in `uint32_t`.

**Attack prevented.** Passing a gigabyte-sized synthetic buffer to trigger integer overflow in length arithmetic is blocked here.

### 7.9.3 Check 3 – Tag whitelist (`ASN_REJECT_TAG`)

**What is checked.** Every tag byte is looked up in the 192-entry `ASN_TAG_WHITELIST` array. Tags not listed (entry 0) are rejected immediately.

**Why.** BER defines hundreds of tag values; IEC 61850 ACSI uses only a small subset (INTEGER, BOOLEAN, OCTET STRING, SEQUENCE, and a handful of context tags). Any tag outside this subset is either a protocol mismatch or an injection attempt. Using a flat array indexed by tag value makes the lookup  $O(1)$  and branch-free.

**Attack prevented.** Injecting obscure BER types (e.g., REAL, EXTERNAL, EMBEDDED PDV) that a downstream parser might mishandle is blocked at the whitelist boundary.

### 7.9.4 Check 4 – Indefinite length (`ASN_REJECT_INDEF`)

**What is checked.** A length byte of exactly `0x80` signals BER indefinite-length encoding and is rejected.

**Why.** DER (the subset of BER used by IEC 61850) mandates definite-length encoding. Indefinite-length requires scanning for an end-of-contents marker (two zero bytes), which creates unbounded loop behaviour and has historically been the source of parser vulnerabilities (e.g., the OpenSSL ASN.1 bugs of 2002 and 2014).

**Attack prevented.** An attacker who injects a BER stream with indefinite-length encoding hoping to exhaust the loop with a missing end-of-contents marker is blocked immediately.

### 7.9.5 Check 5 – Long-form length byte count (`ASN_REJECT_LENGTH`)

**What is checked.** In long-form length encoding, the first byte encodes how many subsequent bytes carry the length value. If this count exceeds `ASN_MAX_LENGTH_BYTES` (4), the message is rejected.

**Why.** A 4-byte length value can encode up to  $2^{32} - 1$ . Since the message itself is bounded to 65 535 bytes (Check 2), any element claiming a multi-gigabyte content length is malformed. Accepting more than 4 length bytes would also allow integer overflow in the accumulated `length` variable.

**Attack prevented.** Crafting a “length-of-length” byte of `0x87` (7 subsequent bytes) to overflow a 32-bit length accumulator is rejected here.

### 7.9.6 Check 6 – Truncation / boundary (ASN\_REJECT\_TRUNCATE)

**What is checked.** After decoding a TLV element’s length field, the filter verifies that `offset + element_length <= end` before recursing into the element’s content. This check is applied throughout the recursive walk, including after multi-byte tag decoding and after long-form length decoding.

**Why.** A truncated element is either a malformed frame or a deliberate attempt to make the parser read beyond the buffer. Because the filter uses pointer arithmetic relative to `end`, a single failed boundary check prevents all downstream out-of-bounds accesses within the current nesting level.

**Attack prevented.** Crafting a SEQUENCE with a claimed content length of 1000 bytes inside a 50-byte frame is rejected here. This is the most common BER parsing vulnerability class (“length-before-buffer-end” attacks).

### 7.9.7 Check 7 – Nesting depth (ASN\_REJECT\_DEPTH)

**What is checked.** The recursion depth is tracked and rejected if it reaches the caller-supplied `max_depth` limit. `safe_asn_filter` uses `SAFE_ASN_MAX_DEPTH = 8` (for protocol headers, GOOSE/SV, and standalone ASN.1). `asn_filter_top` uses `SAFE_ASN_MMS_BODY_MAX_DEPTH = 32` (for MMS service body content, called only from `safe_mms_filter`).

**Why.** IEC 61850 GOOSE and SV use at most 4 levels of nesting; outer MMS PDU headers use at most 7. The limit of 8 allows all legitimate header traffic while bounding the call stack depth. MMS `InformationReport` service bodies contain nested `Data { structure { Data { ... } } }` structures that reach depth 28 in real substation IED traffic; the body limit of 32 accommodates this. Without these checks, deeply-nested BER structures could cause a stack overflow on embedded devices with 4–8 kB stacks.

**Attack prevented.** The “billion laughs” BER variant (deeply-nested empty SEQUENCES) is blocked after 8 levels of recursion for headers (32 for MMS service bodies).

### 7.9.8 Check 8 – INTEGER content (ASN\_REJECT\_BADINT)

**What is checked.** For INTEGER elements, three constraints are enforced:

1. Content length  $\geq 1$  (DER does not allow zero-length integers).
2. Encoding is minimal: no leading `0x00` byte unless the second byte has bit 7 set.
3. Content length is a machine-word width: exactly 1, 2, 4, or 8 bytes. Any other length (e.g., 3, 5, 6, 7) returns `ASN_REJECT_BADINT`.

**Why.** The first two constraints enforce DER uniqueness. The machine-word width constraint is specific to IEC 61850: the standard defines INTEGER as mapping to `INT8`, `INT16`, `INT32`, or `INT64`. No conformant IEC 61850 message carries a 3-byte or 5-byte INTEGER. Accepting non-word-width integers would require the filter to handle values that IED software cannot represent and that have no defined semantics in the standard.

**Attack prevented.** Injecting a non-minimal integer encoding (e.g., `0x02 0x02 0x00 0x01` for the value 1) that might bypass a signature check is rejected here. A crafted 3-byte INTEGER (e.g., `0x02 0x03 0x01 0x00 0x00`) is also rejected before reaching any IED application code that would attempt to parse it.

### 7.9.9 Check 9 – BOOLEAN and NULL content (ASN\_REJECT\_CONTENT)

**What is checked.** BOOLEAN elements must have length exactly 1. NULL elements must have length exactly 0.

**Why.** DER specifies these lengths precisely. A BOOLEAN with length 2 or a NULL with length 1 is malformed and has no legitimate use. Parsing past such elements risks misaligning the TLV offset pointer, causing subsequent tags to be mis-decoded.

**Attack prevented.** Encoding a BOOLEAN with a 4-byte content field to push the parser’s offset forward by 3 extra bytes (potentially skipping a mandatory field) is blocked.

### 7.9.10 Check 10 – Node count (ASN\_REJECT\_TOO\_MANY\_NODES)

**What is checked.** Both `asn_filter_recursive` and `asn_parse_recursive` maintain a shared `node_count` counter. If the count reaches `SAFE_ASN_MAX_NODES = 256` before processing is complete, the function rejects with `ASN_REJECT_TOO_MANY_NODES`.

**Why.** The filter uses the counter to bound the total work done on a single message. The parser additionally uses it to prevent writes past the end of the fixed-size `SafeAsnNode` output array. In both cases the limit caps the number of TLV elements processed regardless of message content.

**Attack prevented.** Sending a frame with 10000 small INTEGER elements is blocked: the counter hits 256 and the frame is rejected before the 257th element is processed. The check applies to both `safe_asn_filter` and `safe_asn_parse`.

## 8 SafeASN.1 Parser – Filter + Zero-Copy Node Extraction

The safe parser (`safe_asn_parse`) performs the same table-driven validation as `safe_asn_filter` but additionally extracts zero-copy TLV node pointers into a fixed-size output array. It is intended for deployments that want to replace a general-purpose ASN.1 parser entirely.

### 8.1 Types

Listing 22: Parser types in `safe.h`

```
1 /* Defined in safe_config.h (default 256) */
2
3 typedef struct {
4     u8      tag;
```



```

5     u32      length;
6     const u8 *content; /* zero-copy pointer into caller's buffer */
7     u32      depth;
8 } SafeAsnNode;
9
10 typedef struct {
11     u32      error;
12     u32      reject_at;
13     u32      node_count;
14     SafeAsnNode nodes[SAFE_ASN_MAX_NODES];
15 } SafeAsnParseResult;

```

Each `SafeAsnNode` records a tag byte, content length, a pointer directly into the caller's buffer (no copy, no allocation), and the nesting depth. The 512-node array accommodates the deepest conformant IEC 61850 messages (typically under 100 TLV elements).

## 8.2 Behavior

`safe_asn_parse` mirrors `safe_asn_filter` exactly:

1. Same size check, same first-tag pre-check.
2. Delegates to `asn_parse_recursive`, which is structurally identical to `asn_filter_recursive`: same switch, same loop, same node counter. The only addition is that after each TLV is validated, the parser records the node (tag, length, content pointer, depth) into the output array.
3. On reject, returns the same error codes as the filter.
4. On accept, `node_count` contains the number of extracted nodes.

Node recording (`nodes[i].tag, .length, .content, .depth`) is an API output, not a security property, and is not covered by the Frama-C/WP proof.

The parser is faster than *libiec61850* and OpenSSL because it avoids heap allocation entirely: every node descriptor is written into the stack-allocated `nodes[]` array.

## 9 SafeMMS – PDU Tag + Service Whitelists

### 9.1 What is MMS?

MMS (Manufacturing Message Specification, ISO 9506) is the client-server protocol used on the IEC 61850 **station bus**. An engineering workstation connects to an IED over TCP (port 102) and uses MMS to:

- **Read** measurements and status (`GetDataValues`)
- **Write** settings and control commands (`Write`)
- **Browse** the IED's data model (`GetNameList`, `GetVariableAccessAttributes`)
- **Transfer files** (`FileOpen`, `FileRead`, `FileClose`)
- **Receive reports** (`InformationReport`)

MMS runs over a deep protocol stack:

TCP → TPKT → COTP → ISO Session → ISO Presentation → MMS PDU

Each layer wraps the one above it in a small header:

- **TPKT** (RFC 1006): 4-byte framing wrapper. Byte 0 = 3 (version), byte 1 = 0 (reserved), bytes 2–3 = total length including this header. Exists solely to carry ISO-over-TCP.
- **COTP** (ISO 8073): Connection-oriented transport. Connection Request (CR, 0xE0) and Connection Confirm (CC, 0xD0) PDUs establish the session; Data Transfer PDUs (DT, 0xF0) carry the MMS payload. Only DT PDUs reach SafeMMS; CR and CC are consumed by the transport stack. COTP adds 3 bytes (length, code, EOT/sequence) per DT segment.
- **ISO Session / Presentation**: Session tokens and ASN.1 data-format negotiation. In practice a fixed-layout set of headers that are the same on every MMS connection.

The SafeMMS filter operates on the **MMS PDU** only, after all these layers have been stripped by the receiving stack. The caller must pass only the MMS application-layer payload.

### 9.2 Worked Example: MMS Read Request

A confirmed-Request PDU for a Read service (service tag [4]):

A0 09 02 01 01 A4 04 30 02 02 00

Per ISO 9506-2, `ConfirmedRequestPDU` is [0] IMPLICIT SEQUENCE. The [0] IMPLICIT means the SEQUENCE tag 0x30 is *replaced* by the context tag A0 on the wire; there is no inner 0x30 wrapper.

1. **Byte 0**: PDU tag = 0xA0 (confirmed-Request). Whitelist: `MMS_PDU_WHITELIST[0xA0] == 1` → accepted.
2. **Byte 1**: Length = 0x09 (9 bytes of content).
3. **Byte 2**: `invokeId` tag = 0x02 (INTEGER). Directly after the PDU length (no inner SEQUENCE tag, because [0] IMPLICIT absorbs it).

4. **Byte 3:** invokeId length = 0x01 (1 byte). Length  $\leq 4 \rightarrow$  accepted.
5. **Byte 4:** invokeId value = 0x01 = 1. Skip.
6. **Byte 5:** Service tag = 0xA4. Low 5 bits = 4 (Read). Whitelist: MMS\_SERVICE\_WHITELIST[4] == 1  $\rightarrow$  accepted.
7. **Result:** MMS\_OK, service\_tag = 4.

### 9.3 Implementation

The MMS filter validates Manufacturing Message Specification PDUs used in IEC 61850 for SCADA communications. It checks the PDU type tag and, for confirmed and unconfirmed PDUs, drills into the inner structure to validate the service tag.

### 9.4 MMS PDU Types

ISO 9506 defines eleven PDU types relevant to IEC 61850 operation. The terminology “confirmed” and “unconfirmed” comes from the standard:

- confirmed-Request (0xA0)** A request from a client (HMI or SCADA) to an IED for a specific service that *requires an explicit reply*. Examples: Read, Write, GetNameList. Every confirmed-Request carries an *invokeId* (a sequence number) so the client can match the response to its request.
- confirmed-Response (0xA1)** The IED’s successful reply to a confirmed-Request. Contains the same invokeId as the original request plus the result data.
- confirmed-ErrorPDU (0xA2)** An error reply to a confirmed-Request when the service itself failed. Body validated as ASN.1 (shallow depth).
- unconfirmed (0xA3)** A one-way notification from the IED to the client – *no reply is expected*. Used exclusively for **InformationReport**: spontaneous data pushed by the IED (periodic measurements, event notifications).
- rejectPDU (0xA4)** Sent by a peer to refuse a request it could not process (e.g. unsupported service). Body validated as ASN.1 (shallow depth).
- initiate-Request / initiate-Response (0xA8 / 0xA9)** MMS connection setup. The client announces its capabilities (max PDU size, max outstanding requests); the IED replies with its own. Sent exactly once at connection establishment; contains no operational service data.
- initiate-ErrorPDU (0xAA)** Error response during MMS connection negotiation. Body validated as ASN.1 (shallow depth).
- conclude-Request / conclude-Response (0x8B / 0x8C)** Graceful connection teardown. No service payload; accepted by SafeMMS on PDU tag alone.
- conclude-ErrorPDU (0xAD)** Error response during connection teardown. Body validated as ASN.1 (shallow depth).

SafeMMS accepts all eleven types. The inner service tag is parsed only for confirmed-Request, confirmed-Response, and unconfirmed PDUs. Error and reject PDUs (0xA2, 0xA4, 0xAA, 0xAD) have their bodies validated as ASN.1 at shallow depth. Initiate PDUs (0xA8/0xA9) have their bodies validated as ASN.1 at depth 32. Conclude PDUs (0x8B/0x8C) carry no payload and are accepted on the PDU tag alone; a non-empty conclude body returns MMS\_REJECT.TRUNCATE.

### 9.5 MMS PDU Wire Format

MMS operates over ISO 9506 / IEC 61850-8-1 on TCP port 102, framed by TPKT/COTP/ISO session/presentation layers. The filter operates on the MMS PDU after COTP delivery.

**MMS PDU layout** (confirmed-Request 0xA0 / confirmed-Response 0xA1):

| B 0     | B 1     | B 2       | B 3       | B 4       | B 5          | B 6+            |
|---------|---------|-----------|-----------|-----------|--------------|-----------------|
| PDU tag | PDU len | invId tag | invId len | invId val | Service tag  | Service content |
| 0xA0    |         | 0x02      | 1–4 B     | integer   | [0], [4], .. | ...             |

*Unconfirmed (0xA3): inner SEQUENCE (0x30) before service tag. Initiate (0xA8/0xA9): body validated as ASN.1 (depth 32). Conclude (0x8B/0x8C): accepted on PDU tag alone (must be empty).*

### 9.6 MMS\_PDU\_WHITELIST – PDU Tag Whitelist Array

Listing 23: MMS\_PDU\_WHITELIST – 176-entry whitelist array

```

1 static const u8 MMS_PDU_WHITELIST[176] = {
2     [MMS_PDU_CONFIRMED_REQUEST] = 1, /* confirmed-Request */
3     [MMS_PDU_CONFIRMED_RESPONSE] = 1, /* confirmed-Response */
4     [MMS_PDU_CONFIRMED_ERROR] = 1, /* confirmed-ErrorPDU */
5     [MMS_PDU_UNCONFIRMED] = 1, /* unconfirmed */
6     [MMS_PDU_REJECT] = 1, /* rejectPDU */
7 #ifdef SAFE_ENABLE_MMS_CANCEL
8     [MMS_PDU_CANCEL_REQUEST] = 1, /* cancelRequestPDU */
9     [MMS_PDU_CANCEL_RESPONSE] = 1, /* cancelResponsePDU */
10    [MMS_PDU_CANCEL_ERROR] = 1, /* cancelErrorPDU */
11 #endif
12    [MMS_PDU_INITIATE_REQUEST] = 1, /* initiate-Request */
13    [MMS_PDU_INITIATE_RESPONSE] = 1, /* initiate-Response */

```

```

14 [MMS_PDU_INITIATE_ERROR ] = 1, /* initiate-Error */
15 [MMS_PDU_CONCLUDE_REQUEST ] = 1, /* conclude-Request */
16 [MMS_PDU_CONCLUDE_RESPONSE ] = 1, /* conclude-Response */
17 [MMS_PDU_CONCLUDE_ERROR ] = 1, /* conclude-Error */
18 };

```

This array has 176 entries. By default, 11 PDU types are accepted. With `SAFE_ENABLE_MMS_CANCEL` defined, three cancel PDU types (0xA5–0xA7) are also accepted, giving 14 in total. Everything else maps to 0 and is rejected.

The symbolic constants are defined in `safe.h`:

Listing 24: MMS\_PDU\_\* constants (safe.h)

```

1 #define MMS_PDU_CONFIRMED_REQUEST 0xA0U
2 #define MMS_PDU_CONFIRMED_RESPONSE 0xA1U
3 #define MMS_PDU_CONFIRMED_ERROR 0xA2U
4 #define MMS_PDU_UNCONFIRMED 0xA3U
5 #define MMS_PDU_REJECT 0xA4U
6 #define MMS_PDU_CANCEL_REQUEST 0xA5U /* opt-in: SAFE_ENABLE_MMS_CANCEL */
7 #define MMS_PDU_CANCEL_RESPONSE 0xA6U /* opt-in: SAFE_ENABLE_MMS_CANCEL */
8 #define MMS_PDU_CANCEL_ERROR 0xA7U /* opt-in: SAFE_ENABLE_MMS_CANCEL */
9 #define MMS_PDU_INITIATE_REQUEST 0xA8U
10 #define MMS_PDU_INITIATE_RESPONSE 0xA9U
11 #define MMS_PDU_INITIATE_ERROR 0xAAU
12 #define MMS_PDU_CONCLUDE_REQUEST 0x8BU
13 #define MMS_PDU_CONCLUDE_RESPONSE 0x8CU
14 #define MMS_PDU_CONCLUDE_ERROR 0xADU

```

Table 6: MMS PDU tag whitelist

| Tag  | MMS PDU Type          | Status                            |
|------|-----------------------|-----------------------------------|
| 0xA0 | confirmed-RequestPDU  | accepted                          |
| 0xA1 | confirmed-ResponsePDU | accepted                          |
| 0xA2 | confirmed-ErrorPDU    | accepted                          |
| 0xA3 | unconfirmed-PDU       | accepted                          |
| 0xA4 | rejectPDU             | accepted                          |
| 0xA5 | cancel-RequestPDU     | accepted (SAFE_ENABLE_MMS_CANCEL) |
| 0xA6 | cancel-ResponsePDU    | accepted (SAFE_ENABLE_MMS_CANCEL) |
| 0xA7 | cancel-ErrorPDU       | accepted (SAFE_ENABLE_MMS_CANCEL) |
| 0xA8 | initiate-RequestPDU   | accepted                          |
| 0xA9 | initiate-ResponsePDU  | accepted                          |
| 0xAA | initiate-ErrorPDU     | accepted                          |
| 0x8B | conclude-RequestPDU   | accepted                          |
| 0x8C | conclude-ResponsePDU  | accepted                          |
| 0xAD | conclude-ErrorPDU     | accepted                          |

## 9.7 MMS\_SERVICE\_WHITELIST – Service Tag Whitelist Array

Listing 25: MMS\_SERVICE\_WHITELIST – 80-entry whitelist array

```

1 static const u8 MMS_SERVICE_WHITELIST[80] = {
2 #ifdef SAFE_ENABLE_MMS_STATUS
3 [0] = 1, /* status */
4 #endif
5 [1] = 1, /* getNameList */
6 #ifdef SAFE_ENABLE_MMS_IDENTIFY
7 [2] = 1, /* identify */
8 #endif
9 [4] = 1, /* read */
10 [5] = 1, /* write */
11 [6] = 1, /* getVarAccessAttr */
12 [11] = 1, /* defineNVL */ /* required: IEDs use this to set up data sets */
13 [12] = 1, /* getNVLAttributes */
14 #ifdef SAFE_ENABLE_MMS_DELETE_NVL
15 [13] = 1, /* deleteNVL */
16 #endif
17 [46] = 1, /* obtainFile */
18 [72] = 1, /* fileOpen */
19 [73] = 1, /* fileRead */
20 [74] = 1, /* fileClose */
21 [77] = 1, /* fileDirectory */
22 /* [75] fileRename and [76] fileDelete unconditionally removed from SafeMMS */
23 };

```

When a confirmed-RequestPDU (0xA0) or confirmed-ResponsePDU (0xA1) is received, the filter extracts the service tag number from inside the PDU and checks it against this array. Eleven MMS services are accepted by default; three more are configurable (Status, Identify, DeleteNVL). `fileRename` [75] and `fileDelete` [76] are unconditionally removed from the safe subset (no compile option re-enables them). `defineNVL` [11] is always enabled because IEDs use it during commissioning to create named variable lists (data sets); disabling it would break normal IED configuration workflows.

Table 7: MMS service tag whitelist

| Service Tag | MMS Service                    | IEC 61850 Usage                         |
|-------------|--------------------------------|-----------------------------------------|
| 1           | getNameList                    | Discovery                               |
| 4           | read                           | Data access                             |
| 5           | write                          | Data access                             |
| 6           | getVariableAccessAttributes    | Discovery                               |
| 11          | defineNamedVariableList        | NVL management (always on)              |
| 12          | getNamedVariableListAttributes | NVL management                          |
| 13          | deleteNamedVariableList        | NVL management (configurable)           |
| 46          | obtainFile                     | File transfer                           |
| 72          | fileOpen                       | File transfer                           |
| 73          | fileRead                       | File transfer                           |
| 74          | fileClose                      | File transfer                           |
| 75          | fileRename                     | File transfer (unconditionally removed) |
| 76          | fileDelete                     | File transfer (unconditionally removed) |
| 77          | fileDirectory                  | File transfer                           |

## 9.8 Disabled MMS services: rationale

The table below explains why each rejected or conditionally disabled MMS PDU type or service is blocked by default. Security reason and operational context are given for each item.

Table 8: MMS constructs disabled by default and rationale

| Tag / Service      | Name                                         | Reason disabled                                                                                                                                                                                                                                                                                                                                                                         |
|--------------------|----------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 0xA5-0xA7          | cancel-PDU variants                          | Transaction cancellation messages. Rarely used in IEC 61850 deployments. An asymmetric cancel attack (sending a cancel for a transaction that was never initiated) can abort legitimate control sessions without knowledge of the outstanding invokeId.                                                                                                                                 |
| Service 13         | deleteNamedVariableList                      | Deletes an existing NVL. Deletion is destructive and cannot be reversed without IED configuration reload. A compromised HMI could delete all NVLs used by SCADA to read real-time data, causing a loss of visibility. Block by default.                                                                                                                                                 |
| Service 75         | fileRename                                   | Renames a file on the IED filesystem (unconditionally removed; no compile option re-enables it). A protection relay stores its configuration as files; renaming a settings file to a backup name and uploading a malicious replacement is the classic firmware-swap attack path.                                                                                                        |
| Service 76         | fileDelete                                   | Deletes a file on the IED filesystem (unconditionally removed; no compile option re-enables it). Deleting a relay's trip-curve table or protection settings file is a denial-of-protection attack.                                                                                                                                                                                      |
| Services 72-74, 46 | fileOpen / fileRead / fileClose / obtainFile | File upload and download services. Although accepted by default (IEC 61850 disturbance recording and log retrieval use fileRead), these services represent the main firmware/-configuration injection path. If the protected IED has no disturbance recorder or log retrieval requirement, disable these services in <code>safe_config.h</code> to eliminate the file channel entirely. |

## 9.9 safe\_mms\_filter – Entry Point

### 9.9.1 ACSL Contract

Listing 26: safe\_mms\_filter ACSL contract

```

1 /*@
2  requires length <= 65535;

```

```

3  requires length == 0 || \valid_read(mms_pdu + (0 .. length - 1));
4  assigns \nothing;
5
6  ensures length > 65535
7  ==> \result.error == MMS_REJECT_SIZE;
8
9  ensures length == 0
10 ==> \result.error == MMS_REJECT_TRUNCATE;
11
12 ensures length > 0
13 length <= 65535
14 mms_pdu[0] < sizeof(MMS_PDU_WHITELIST)
15 MMS_PDU_WHITELIST[mms_pdu[0]] == 0
16 ==> \result.error == MMS_REJECT_PDU_TAG;
17
18 SAFE_PURE SafeMmsResult safe_mms_filter(
19     const u8 * const mms_pdu,
20     const u32 length)

```

The contract provides three functional correctness guarantees:

- Oversize messages always yield `MMS_REJECT_SIZE`.
- Empty messages always yield `MMS_REJECT_TRUNCATE`.
- If the first byte is within the whitelist array range but maps to 0, the result is guaranteed to be `MMS_REJECT_PDU_TAG`.

### 9.9.2 Algorithm

1. **Size guard.** Reject if `length == 0U` (truncated) or `length > MMS_MAX_PDU` (65535).
2. **PDU tag check.** Look up `buffer[0]` in `MMS_PDU_WHITELIST`. If the tag is out of range or maps to 0, reject with `MMS_REJECT_PDU_TAG`.
3. **Decode PDU length** using `read_length`. Verify that the PDU content fits within the buffer. Inline assertions help the prover:

```

1  /*@ assert \separated(&offset, buffer + (0 .. length - 1)); */
2  const u32 pdu_len = read_length(buffer, length, &offset);
3  /*@ assert offset <= length; */

```

4. **Confirmed-RequestPDU (0xA0) or confirmed-ResponsePDU (0xA1):**
  - (a) **No inner SEQUENCE.** Per ISO 9506-2, [0] IMPLICIT SEQUENCE means the SEQUENCE tag (0x30) is suppressed on the wire. The `invokeId` INTEGER appears immediately after the PDU length.
  - (b) Expect INTEGER (tag 0x02) for `invokeId`. Reject with `MMS_REJECT_INVOKE_ID` if missing or if its length exceeds 3 bytes.
  - (c) Skip past the `invokeId` value bytes.
  - (d) Read the service tag byte. If the low 5 bits are all 1s (0x1F), decode a multi-byte tag number using base-128 encoding:

Listing 27: High-tag-number (base-128) decoding in MMS

```

1  const u8 service_byte = mms_pdu[offset];
2  offset++;
3  u32 service = (u32)(service_byte & 0x1FU);
4  if (service == 0x1FU)
5  {
6      service = 0U;
7      /*@
8      loop invariant offset <= length;
9      loop assigns offset, service;
10     loop variant length - offset;
11     */
12     while (offset < length)
13     {
14         const u8 b = mms_pdu[offset];
15         offset++;
16         service = (service << 7U) | (u32)(b & 0x7FU);
17         if ((b & 0x80U) == 0U)
18             break;
19     }
20 }

```

When the low 5 bits are 0x1F, it means “tag number  $\geq 31$ .” The actual tag number follows in base-128 encoding where each byte’s high bit is a continuation flag. Many MMS services (like `fileDirectory` at 46, `getDataValues` at 72) have tag numbers above 30, so this decoding is essential.

- (e) Look up the decoded `service` in `MMS_SERVICE.WHITELIST`. Store it in `out.service_tag`. Reject with `MMS_REJECT_SERVICE` if not found.
5. **Unconfirmed-PDU (0xA3):**
    - (a) Expect inner SEQUENCE (0x30).
    - (b) Read service tag byte. Extract low 5 bits as service number.
    - (c) Only service 0 (informationReport) is accepted. Anything else is rejected with `MMS_REJECT_SERVICE`.
  6. **Error/reject PDUs** (0xA2 confirmed-Error, 0xA4 reject, 0xAA initiate-Error, 0xAD conclude-Error): body validated as ASN.1 at shallow depth (8 levels); rejected with `MMS_REJECT_BODY` if malformed.
  7. **Initiate PDUs** (0xA8 initiate-Request, 0xA9 initiate-Response): body validated with a full SafeASN.1 walk (`asn_filter_top`, depth 32). A first-byte context-tag assertion is made before the walk.

8. **Conclude PDUs** (0x8B conclude-Request, 0x8C conclude-Response): IEC 61850 conclude frames carry no payload; accepted on PDU tag alone. A non-empty conclude body (`length`  $\neq$  0) returns `MMS_REJECT_TRUNCATE`.

## 9.10 All 7 validation checks: what, why, and what error code

### 9.10.1 Check 1 – PDU size (`MMS_REJECT_SIZE`)

**What is checked.** `length == 0` (empty) or `length > MMS_MAX_PDU` (65 535) is rejected immediately before any byte access.

**Why.** MMS is carried over TPKT, which uses a 16-bit length field. A PDU larger than 65 535 bytes cannot exist on the wire; any buffer claiming to be larger is either a driver error or an injection attempt. The empty-length case is rejected because there is no valid MMS PDU in zero bytes.

**Attack prevented.** A synthetic buffer claiming to be 4GB large – passed to trigger integer overflow in offset arithmetic – is stopped here.

### 9.10.2 Check 2 – PDU tag whitelist (`MMS_REJECT_PDU_TAG`)

**What is checked.** The first byte of the PDU is looked up in `MMS_PDU_WHITELIST`. Tags not in the whitelist (entry 0) are rejected.

**Why.** ISO 9506 defines a range of PDU types; SafeMMS accepts the 11 that appear in normal IEC 61850 operation: confirmed request/response/error (0xA0–0xA2), reject (0xA4), unconfirmed (0xA3), initiate request/response/error (0xA8/0xA9/0xAA), and conclude request/response/error (0x8B/0x8C/0xAD). The whitelist excludes cancel (0xA5–0xA7) and other unused types that expand attack surface without serving any legitimate IEC 61850 function.

**Attack prevented.** Injecting a cancel-PDU (0xA5–0xA7) to disrupt an ongoing MMS transaction is blocked at the PDU tag check.

### 9.10.3 Check 3 – Truncation (`MMS_REJECT_TRUNCATE`)

**What is checked.** After decoding the PDU length field, the filter verifies that the declared content length does not extend past the available buffer bytes.

**Why.** The PDU length field is decoded from the BER TLV length bytes immediately following the PDU tag byte. If the declared length would require reading beyond the end of the buffer, the frame is either truncated at the network layer or deliberately crafted to read out-of-bounds.

**Attack prevented.** A CONFIRMED-REQUEST PDU with a length field claiming 500 bytes of content inside a 20-byte buffer is caught here, before any inner field decode begins.

### 9.10.4 Check 4 – Inner SEQUENCE tag (`MMS_REJECT_INNER_SEQ`)

**What is checked.** For unconfirmed PDUs (tag 0xA3), the first byte inside the PDU content must be either 0x30 (SEQUENCE-wrapped encoding) or 0xA0 (direct [0] InformationReport encoding). Any other first byte is rejected.

**Why.** ISO 9506-2 defines `ConfirmedRequestPDU` as [0] IMPLICIT SEQUENCE: the [0] IMPLICIT replaces the outer 0x30 SEQUENCE tag entirely, so the content follows directly after the 0xA0 PDU tag – no inner 0x30 appears. `UnconfirmedPDU` is [3] IMPLICIT SEQUENCE enclosing an `UnconfirmedService CHOICE`. IEC 61850-8-1 implementations encode this in two ways on the wire: (a) a SEQUENCE wrapper (0x30) followed by the [0] InformationReport tag, or (b) the [0] tag directly with no wrapper. Both are seen in production captures; the filter accepts either. A first byte outside {0x30, 0xA0} means the PDU is malformed.

**Attack prevented.** An unconfirmed PDU without the expected inner SEQUENCE tag would cause the subsequent service-tag extraction to read from the wrong offset, potentially decoding garbage as a valid service tag. The check prevents this misparse.

### 9.10.5 Check 5 – `invokeId` (`MMS_REJECT_INVOKE_ID`)

**What is checked.** For confirmed-request (0xA0) and confirmed-response (0xA1) PDUs, an INTEGER tag (0x02) must appear immediately after the PDU length bytes, with a content length of at most 4 bytes.

**Why.** The `invokeId` INTEGER is mandatory in all confirmed transactions (ISO 9506-2 clause 5.3.3). ISO 9506-2 defines `invokeId` as `Unsigned32`, allowing values up to  $2^{32} - 1$ . The filter accepts up to 4 DER content bytes ( $\leq 2^{31} - 1 = 2,147,483,647$ ), covering the full range reached by long-running IEDs. An `invokeId` longer than 4 bytes is rejected.

**Attack prevented.** Crafting an `invokeId` with a claimed length of 100 bytes to advance the offset by 97 extra bytes, causing the service tag decoder to read an attacker-controlled byte as a whitelisted service, is blocked here.

### 9.10.6 Check 6 – Service tag whitelist (`MMS_REJECT_SERVICE`)

**What is checked.** The service tag decoded from the PDU (using base-128 multi-byte encoding if the low 5 bits are 0x1F) is looked up in `MMS_SERVICE_WHITELIST`. Services not in the whitelist are rejected.

**Why.** ISO 9506 defines more than 80 service types. IEC 61850 normal operation uses only a subset: read (1), write (2), getNameList (3), getNamedVariableListAttributes (12), getDataValues (72/73), informationReport (0), and a handful of file-access services. The whitelist excludes administrative and file-manipulation services unless explicitly re-enabled via `safe_config.h`.

**Attack prevented.** Sending a `fileRename` service request (tag 75) to rename a protection relay's configuration file is always blocked: `fileRename` (75) and `fileDelete` (76) are unconditionally removed from the safe subset and no compile option re-enables them.

### 9.10.7 Check 7 – Service body content (MMS.REJECT.BODY)

**What is checked.** After the service tag is accepted, the remaining service body bytes (following the service tag and its length field) are passed to `asn_filter_top` with `max_depth = SAFE_ASN_MMS_BODY_MAX_DEPTH`. This is a structural ASN.1 DER check using the higher depth limit `SAFE_ASN_MMS_BODY_MAX_DEPTH = 32` (instead of `SAFE_ASN_MAX_DEPTH = 8`), to accommodate the deep nesting found in `InformationReport` service bodies from real substation IED deployments (max observed depth: 28). If any ASN.1 structural violation is found (wrong tag, indefinite length, bad integer, depth  $\geq 32$ , etc.), the PDU is rejected with `MMS.REJECT.BODY`.

**Why.** MMS service bodies are themselves ASN.1-encoded. Passing unvalidated body bytes to the downstream MMS parser (e.g., `libiec61850`) is what leads to CVE-class vulnerabilities: a crafted domain ID exceeding the buffer limit (`SAFE_ASN_MAX_IA5_STRING = 129`), or a deeply-nested structure that triggers a stack overflow. The body check provides a second structural gate before any semantic parsing occurs.

**Why depth 32 instead of 8.** Real substation relay `InformationReport` bodies (captured from production substations) reach nesting depth 28 due to `Data { structure { Data { structure { ... } } } }` encoding of multi-attribute logical-node objects. Using depth 8 would reject this legitimate operational traffic (false positives). The default `32 = 28 + 4` headroom accepts all observed real-world MMS traffic. Operators with simpler deployments may reduce `SAFE_ASN_MMS_BODY_MAX_DEPTH` in `safe_config.h` for a stricter policy. If an IED in your network legitimately produces bodies deeper than 32 (e.g., a non-standard implementation with unusually nested structures), increase `SAFE_ASN_MMS_BODY_MAX_DEPTH` and re-run Frama-C/WP and the full test suite to verify the new limit.

**Attack prevented.** A crafted MMS service body with 10-level nested `SEQUENCE` elements that would overflow a fixed-size buffer in a downstream parser is checked here. For depth limits  $\leq 32$ : rejected at `MMS.REJECT.BODY` before the body reaches any semantic parser.

## 10 SafeDNP3 – FC/Group/Variation/Qualifier Whitelists

### 10.1 What is DNP3?

DNP3 (Distributed Network Protocol 3, IEEE 1815) is the dominant SCADA protocol in North America for communication between control centres and remote substations. A **master station** polls **outstations** (remote terminal units) for status data and sends control commands.

Key concepts:

- **Function codes:** what operation? Read (0x01), Write (0x02), Select (0x03), Operate (0x04), Response (0x81), etc. **Select-Before-Operate (SBO)** is a two-step command handshake: the master first sends a Select to arm the output (outstation responds to confirm the point exists and is ready); within a short window (typically 5s) the master sends Operate to execute the action. A single stale or replayed Operate without a prior Select is rejected by a conformant outstation. Direct Operate (FC 0x05) skips the Select step entirely and must be disabled in high-security installations.
- **Object groups:** what data type? A *group* identifies a class of physical signal. Group 1 = Binary Input (switch or contact status), Group 10 = Binary Output (relay command), Group 30 = Analog Input (voltage, current, power), Group 20 = Counter (pulse count), and so on. Each group has a number from 0 to 255; the filter checks it against `DNP3_GROUP_WHITELIST`.
- **Points:** a *point* is one individual signal within a group. Points are addressed by a zero-based integer index. For example, a substation might have Binary Inputs 0–9 (breaker statuses) and Analog Inputs 0–3 (phase voltages). A single DNP3 object header can carry a *range* of consecutive points (e.g., “Binary Input, indices 0–9”). The filter must know the byte size of each point to skip correctly from one object header to the next; this is what `DNP3_POINT_SIZE` encodes.
- **Variations:** how is each point encoded on the wire? Within a group, a *variation* selects the binary format. Group 30 (Analog Input) offers six variations: Var 1 = 32-bit integer with quality flags (5 bytes per point), Var 2 = 16-bit integer with flags (3 bytes), Var 3 = 32-bit integer without flags (4 bytes), Var 4 = 16-bit integer without flags (2 bytes), Var 5 = single-precision float with flags (5 bytes). If the filter does not know the byte width of the chosen variation it cannot find the next object header, opening a buffer-overread path. `DNP3_POINT_SIZE[group][variation]` provides the byte width; an INV entry means the variation is not permitted.
- **Qualifier:** how many points and how are they addressed? The qualifier byte encodes two sub-fields packed into one byte. The high nibble (bits 7:4) is the *object prefix code*: it says whether each object is prefixed with an index, a size, or nothing (0x0). The low nibble (bits 3:0) is the *range code*: it says how many points are being referenced and how they are numbered. Common range codes: 0x00 = 8-bit start/stop indices, 0x01 = 16-bit start/stop, 0x06 = all objects (no range bytes), 0x07 = 8-bit count, 0x08 = 16-bit count. The filter checks the qualifier byte against `DNP3_RANGE_SIZE` to know how many range bytes follow the object header, and rejects unknown codes with `DNP3_REJECT_QUALIFIER`.

DNP3 has a data-link layer (with CRC protection) and a transport layer (for fragment reassembly). The SafeDNP3 filter operates on the **application layer** only, after the lower layers have been processed.

**DNP3 layer summary (for context).** The full DNP3 stack from wire to filter:



1. **Data-link frame:** 10-byte header (start bytes 0x05 0x64, length, control, destination/source addresses) + data bytes + CRC per 16 bytes. The data-link layer handles CRC verification and delivers data-link frames to the transport layer.
2. **Transport segment:** 1-byte transport header (FIR/FIN flags + sequence number). Multiple transport segments may be reassembled into one application-layer fragment.
3. **Application layer:** 1-byte Application Control byte + 1-byte Function Code + zero or more object headers. **This is what safe\_dnp3\_filter receives.**

## 10.2 Worked Example: DNP3 Read Request

A Read request for Class 1, 2, 3 data:

C0 01 3C 02 06 3C 03 06 3C 04 06

1. **Byte 0:** Application control = 0xC0 (FIR=1, FIN=1, sequence=0). This is a single-fragment request.
2. **Byte 1:** Function code = 0x01 (Read). Whitelist: DNP3\_FUNCTION\_CODE\_WHITELIST[0x01] == 1 → accepted.
3. **Bytes 2–4:** Object header 1: group=60 (0x3C), variation=2, qualifier=0x06 (all objects, no range field). Group 60 = Class Data → DNP3\_GROUP\_WHITELIST[60] == 1 → accepted.
4. **Bytes 5–7:** Object header 2: group=60, variation=3, qualifier=0x06 → accepted.
5. **Bytes 8–10:** Object header 3: group=60, variation=4, qualifier=0x06 → accepted.
6. **Result:** DNP3\_OK, function\_code = 0x01, object\_count = 3.

## 10.3 Implementation

The DNP3 filter validates the application layer of DNP3 messages. It checks the function code, then parses each object header and validates the group, variation, and qualifier against lookup tables. This section covers all six tables and the main filter function.

## 10.4 DNP3 Application Layer Wire Format

DNP3 (IEEE 1815) uses a data-link layer with CRC protection, a transport layer for fragment reassembly, and an application layer. The filter operates on the reassembled application layer only (after data-link CRC stripping and transport reassembly).

**DNP3 application-layer fragment layout:**

| Byte 0                                      | Byte 1                                               | Bytes 2+                                                                                        |
|---------------------------------------------|------------------------------------------------------|-------------------------------------------------------------------------------------------------|
| App Control<br><br>FIR/FIN/CON/UNS<br>+ Seq | Function Code<br><br>0x01=Read,<br>0x81=Resp,<br>... | Object headers (up to 64)<br><br>Group (1B) + Variation (1B) + Qualifier (1B) +<br>Range + Data |

**Application Control byte (bits 7–0):**

| b7                             | b6  | b5  | b4  | b3                               | b2 | b1 | b0 |
|--------------------------------|-----|-----|-----|----------------------------------|----|----|----|
| FIR                            | FIN | CON | UNS | Sequence (0–15)                  |    |    |    |
| FIR=first frag, FIN=final frag |     |     |     | CON=confirm-req, UNS=unsolicited |    |    |    |

## 10.5 DNP3\_FUNCTION\_CODE\_WHITELIST – Function Code Whitelist Array

Listing 28: DNP3\_FUNCTION\_CODE\_WHITELIST – 136-entry whitelist array

```

1 static const u8 DNP3_FUNCTION_CODE_WHITELIST[136] = {
2     [0x00] = 1, /* confirm */
3     [0x01] = 1, /* read */
4     [0x02] = 1, /* write */
5     [0x03] = 1, /* select */
6     [0x04] = 1, /* operate */
7     #ifdef SAFE_ENABLE_DNP3_DIRECT_OPERATE
8     [0x05] = 1, /* direct-operate */
9     #endif
10    #ifdef SAFE_ENABLE_DNP3_DIRECT_OPERATE_NR
11    [0x06] = 1, /* direct-operate-nr */
12    #endif
13    #ifdef SAFE_ENABLE_DNP3_COUNTER_FREEZE
14    [0x07] = 1, /* immed-freeze */
15    [0x08] = 1, /* immed-freeze-nr */
16    [0x09] = 1, /* freeze-clear */
17    [0x0A] = 1, /* freeze-clear-nr */
18    [0x0B] = 1, /* freeze-at-time */
19    [0x0C] = 1, /* freeze-at-time-nr */
20    #endif
21    #ifdef SAFE_ENABLE_DNP3_COLD_RESTART
22    [0x0D] = 1, /* cold-restart */

```

```

23 #endif
24 #ifdef SAFE_ENABLE_DNP3_WARM_RESTART
25     [0x0E] = 1, /* warm-restart */
26 #endif
27     [0x14] = 1, /* enable-unsol */
28 #ifdef SAFE_ENABLE_DNP3_DISABLE_UNSOLICITED
29     [0x15] = 1, /* disable-unsol */
30 #endif
31 #ifdef SAFE_ENABLE_DNP3_ASSIGN_CLASS
32     [0x16] = 1, /* assign-class */
33 #endif
34     [0x17] = 1, /* delay-measure */
35 #ifdef SAFE_ENABLE_DNP3_FILE_TRANSFER
36     [0x19] = 1, /* open-file */
37     [0x1A] = 1, /* close-file */
38     [0x1B] = 0, /* delete-file is ALWAYS disabled */
39     [0x1C] = 1, /* get-file-info */
40     [0x1D] = 1, /* authenticate */
41     [0x1E] = 1, /* abort-file */
42 #endif
43     [0x81] = 1, /* response */
44     [0x82] = 1, /* unsol-response */
45 };

```

Nine function codes are accepted by default. Eight additional groups are configurable via compile-time flags in `safe_config.h` (direct operate, counter freeze, cold/warm restart, disable unsolicited, assign class, file transfer).

Table 9: DNP3 function code whitelist

| FC   | Function             | Direction              |
|------|----------------------|------------------------|
| 0x00 | Confirm              | Request                |
| 0x01 | Read                 | Request                |
| 0x02 | Write                | Request                |
| 0x03 | Select               | Request                |
| 0x04 | Operate              | Request                |
| 0x14 | Enable Unsolicited   | Request                |
| 0x15 | Disable Unsolicited  | Request (configurable) |
| 0x16 | Assign Class         | Request (configurable) |
| 0x17 | Delay Measurement    | Request                |
| 0x81 | Response             | Response               |
| 0x82 | Unsolicited Response | Response               |

## 10.6 DNP3\_GROUP\_WHITELIST – Object Group Whitelist Array

Listing 29: DNP3\_GROUP\_WHITELIST – 123-entry whitelist array

```

1 static const u8 DNP3_GROUP_WHITELIST[123] = {
2     [ 1] = 1, /* binary input */
3     [ 2] = 1, /* binary input event */
4     [ 3] = 1, /* double-bit binary input */
5     [ 4] = 1, /* double-bit binary input event */
6     [10] = 1, /* binary output */
7     [11] = 1, /* binary output event */
8     [12] = 1, /* CROB */
9     [20] = 1, /* binary counter */
10    [21] = 1, /* frozen counter */
11    [22] = 1, /* binary counter event */
12    [23] = 1, /* frozen counter event */
13    [30] = 1, /* analog input */
14    [31] = 1, /* frozen analog input */
15    [32] = 1, /* analog input event */
16    [33] = 1, /* frozen analog input event */
17    [40] = 1, /* analog output */
18    [41] = 1, /* analog output block */
19    [42] = 1, /* analog output event */
20    [50] = 1, /* time and date */
21    [51] = 1, /* time CTO */
22    [52] = 1, /* time delay */
23    [60] = 1, /* class objects */
24    [80] = 1, /* internal ind */
25 #ifdef SAFE_ENABLE_DNP3_SECURE_AUTH
26     [120] = 1, /* authentication */
27     [121] = 1, /* auth aggressive mode */
28     [122] = 1, /* auth certificate */
29 #endif
30 };

```

Twenty-three object groups are accepted by default. These cover the core SCADA data types: binary inputs and outputs (including event reports), counters and frozen counters (including event variants), analog inputs and frozen analogs (including event variants), analog outputs, time objects, class data, and internal indications. Groups 120–122 (Secure Authentication) are configurable via `SAFE_ENABLE_DNP3_SECURE_AUTH`.

## 10.7 DNP3\_POINT\_SIZE – 2D Point Size Lookup

The point-size table is generated from a single source-of-truth X-macro list, DNP3\_POINTS, so the permitted (group, variation) set and the size table can never drift apart. Each X(group, variation, octet-size) entry is one accepted combination; any (group, variation) not listed defaults to DNP3\_VARIATION\_INVALID (0x00) and is rejected (fail-closed).

**Why an X-macro.** DNP3\_POINTS is a *list macro*: it takes one argument, conventionally named X, and expands to a run of X(group, variation, octet-size) invocations, one per accepted combination. The list stores data, not code. Each site that consumes it first #defines X to emit whatever that site needs from each entry, expands DNP3\_POINTS(X), then #undefs X. The size table defines X(g, v, sz) as the designated initializer [g][v] = (u8)(sz), so DNP3\_POINTS(X) expands to one initializer per entry and fills DNP3\_POINT\_SIZE directly.

We use the pattern for three reasons. *Single source of truth*: the permitted (group, variation) pairs and their on-wire sizes are written once, in one list, so the membership test and the size lookup read the same array and cannot disagree. *Fail-closed by construction*: any pair absent from the list keeps the array's zero default (DNP3\_VARIATION\_INVALID) and is rejected, so an omission denies an object rather than admitting it unchecked. *One-line maintenance*: adding or removing a single X(...) line updates every table generated from the list at once. The IEC-104 size and permission tables (Section 11.5) apply the same technique more visibly, expanding one list set twice under two different definitions of X.

Listing 30: DNP3\_POINT\_SIZE – point size by group and variation, generated from DNP3\_POINTS

```

1  #define DNP3_VARIATION_INVALID 0x00U /* not permitted; also the unlisted default */
2  #define DNP3_VARIATION_PACKED 0xFFU /* packed bits: ceil(count/8) */
3  #define DNP3_VARIATION_ANY 0xFDU /* variation 0 / class: no fixed size */
4
5  /* Short aliases used inside the DNP3_POINTS source list below */
6  #define PKD DNP3_VARIATION_PACKED
7  #define ANY DNP3_VARIATION_ANY
8
9  /*
10 * Single source of truth for permitted DNP3 objects: each X(group, variation,
11 * octet-size) is one accepted (group, variation), its fixed on-wire object size
12 * verified against IEEE Std 1815-2012 Annex A. DNP3_POINT_SIZE is generated
13 * from this list so the table and the permitted set can never drift. Any
14 * (group, variation) NOT listed defaults to DNP3_VARIATION_INVALID (0) and is
15 * rejected -- fail-closed.
16 *
17 * Size building blocks: flag/control octet (BSTR8) = 1, UINT16/INT16 = 2,
18 * UINT32/INT32/FLT32 = 4, DNP3TIME (UINT48) = 6, FLT64 = 8.
19 */
20 #define DNP3_POINTS(X) \
21     /* g1 Binary Input (A.2) */ \
22     X( 1,0,ANY) X( 1,1,PKD) X( 1,2, 1) \
23     /* g2 Binary Input Event (A.3) */ \
24     X( 2,0,ANY) X( 2,1, 1) X( 2,2, 7) X( 2,3, 3) \
25     /* g3 Double-bit Binary Input (A.4) */ \
26     X( 3,0,ANY) X( 3,1,PKD) X( 3,2, 1) \
27     /* g4 Double-bit Binary In Evt (A.5) */ \
28     X( 4,0,ANY) X( 4,1, 1) X( 4,2, 7) X( 4,3, 3) \
29     /* g10 Binary Output (A.6) */ \
30     X(10,0,ANY) X(10,1,PKD) X(10,2, 1) \
31     /* g11 Binary Output Event (A.7: only v1,v2 defined) */ \
32     X(11,0,ANY) X(11,1, 1) X(11,2, 7) \
33     /* g12 CROB (A.8.1) */ \
34     X(12,1,11) \
35     /* g20 Counter (A.10) */ \
36     X(20,0,ANY) X(20,1, 5) X(20,2, 3) X(20,5, 4) X(20,6, 2) \
37     /* g21 Frozen Counter (A.11: v5/v6 are flag+time; the */ \
38     /* without-flag forms live at v9/v10, unreachable here) */ \
39     X(21,0,ANY) X(21,1, 5) X(21,2, 3) \
40     /* g22 Counter Event (A.12: v5/v6 with-time) */ \
41     X(22,0,ANY) X(22,1, 5) X(22,2, 3) X(22,5,11) X(22,6, 9) \
42     /* g23 Frozen Counter Event (A.13: v5/v6 with-time) */ \
43     X(23,0,ANY) X(23,1, 5) X(23,2, 3) X(23,5,11) X(23,6, 9) \
44     /* g30 Analog Input (A.14) */ \
45     X(30,0,ANY) X(30,1, 5) X(30,2, 3) X(30,3, 4) X(30,4, 2) X(30,5, 5) \
46     /* g31 Frozen Analog Input (A.15) */ \
47     X(31,0,ANY) X(31,1, 5) X(31,2, 3) X(31,3,11) X(31,4, 9) \
48     /* g32 Analog Input Event (A.16) */ \
49     X(32,0,ANY) X(32,1, 5) X(32,2, 3) X(32,3,11) X(32,4, 9) X(32,5, 5) X(32,6, 9) \
50     /* g33 Frozen Analog Input Event (A.17) */ \
51     X(33,0,ANY) X(33,1, 5) X(33,2, 3) X(33,3,11) X(33,4, 9) X(33,5, 5) X(33,6, 9) \
52     /* g40 Analog Output Status (A.19) */ \
53     X(40,0,ANY) X(40,1, 5) X(40,2, 3) X(40,3, 5) \
54     /* g41 Analog Output Command (A.20) */ \
55     X(41,1, 5) X(41,2, 3) X(41,3, 5) \
56     /* g42 Analog Output Event (A.21) */ \
57     X(42,0,ANY) X(42,1, 5) X(42,2, 3) X(42,3,11) X(42,4, 9) X(42,5, 5) X(42,6, 9) \
58     /* g50 Time and Date (A.23) */ \
59     X(50,1, 6) X(50,2,10) X(50,3, 6) X(50,4,11) \
60     /* g51 Time and Date CTO (A.24) */ \
61     X(51,1, 6) X(51,2, 6) \
62     /* g52 Time Delay (A.25) */ \
63     X(52,1, 2) X(52,2, 2) \
64     /* g60 Class Data (A.26: carries no data) */ \
65     X(60,1,ANY) X(60,2,ANY) X(60,3,ANY) X(60,4,ANY) \
66     /* g80 Internal Indications (A.28) */ \

```

```

67 |     X(80,1,PKD)
68 |
69 | static const u8 DNP3_POINT_SIZE[81][7] = {
70 | #define X(g, v, sz)      [g][v] = (u8)(sz),
71 |     DNP3_POINTS(X)
72 | #undef X
73 | };
74 |
75 | #undef PKD
76 | #undef ANY

```

DNP3\_POINT\_SIZE[group][variation] gives the size in bytes of one data point for that group/variation combination. Three sentinel values encode special cases:

- DNP3\_VARIATION\_INVALID (0x00): not permitted; this is also the default for any (group, variation) not listed in DNP3\_POINTS. The filter rejects with DNP3\_REJECT\_VARIATION.
- PKD / DNP3\_VARIATION\_PACKED (0xFF): packed bit-string. Data bytes =  $\lceil \text{count}/8 \rceil$ . Used for Group 1 Var 1 (packed binary input), Group 3 Var 1 (packed double-bit binary input), Group 10 Var 1 (packed binary output), and Group 80 Var 1 (packed internal indications).
- ANY / DNP3\_VARIATION\_ANY (0xFD): no fixed size. Variation 0 (and class objects) means “any variation” and appears in read requests where the responder chooses the variation.

As a concrete example, Group 30 (Analog Input) has: Var 0 = any (ANY), Var 1 = 32-bit with flags (5 bytes), Var 2 = 16-bit with flags (3), Var 3 = 32-bit no flags (4), Var 4 = 16-bit no flags (2), Var 5 = float with flags (5); Var 6 is not listed, so it defaults to DNP3\_VARIATION\_INVALID (double-precision float is not accepted).

### 10.7.1 How each size is derived

Each DNP3\_POINT\_SIZE entry is the sum of the octet fields in that variation’s “Formal structure” definition in IEEE Std 1815-2012 Annex A. The building blocks are:

- **flag** = a 1-octet flag/quality octet (BSTR8); for the double-bit groups (g3, g4) the flags and 2-bit state pack into the same single octet.
- **INT16** / **UINT16** = 2 octets; **INT32** / **UINT32** / **FLT32** = 4 octets; **FLT64** = 8 octets.
- **time** = a 6-octet DNP3 Time value (UINT48).

The table below decomposes every accepted fixed-size (group, variation). The PKD entries (g1/g3/g10/g80 Var 1) carry  $\lceil \text{count}/8 \rceil$  packed bits rather than whole-octet points, and the ANY entries (Var 0 and the g60 class objects) carry no data, so neither appears here.

Table 10: DNP3\_POINT\_SIZE entries derived from IEEE Std 1815-2012 Annex A field layouts

| g / v                                                                         | Field composition                                                                | Octets                 |
|-------------------------------------------------------------------------------|----------------------------------------------------------------------------------|------------------------|
| <i>g1 Binary Input, g2 Binary Input Event (A.2–A.3)</i>                       |                                                                                  |                        |
| 1 / 2                                                                         | flag                                                                             | 1 = 1                  |
| 2 / 1                                                                         | flag                                                                             | 1 = 1                  |
| 2 / 2                                                                         | flag + time                                                                      | 1 + 6 = 7              |
| 2 / 3                                                                         | flag + relative time (UINT16)                                                    | 1 + 2 = 3              |
| <i>g3 Double-bit Binary Input, g4 Double-bit Binary Input Event (A.4–A.5)</i> |                                                                                  |                        |
| 3 / 2                                                                         | flags + 2-bit state (one octet)                                                  | 1 = 1                  |
| 4 / 1                                                                         | flags + 2-bit state (one octet)                                                  | 1 = 1                  |
| 4 / 2                                                                         | flags+state + time                                                               | 1 + 6 = 7              |
| 4 / 3                                                                         | flags+state + relative time (UINT16)                                             | 1 + 2 = 3              |
| <i>g10 Binary Output, g11 Binary Output Event (A.6–A.7)</i>                   |                                                                                  |                        |
| 10 / 2                                                                        | flag                                                                             | 1 = 1                  |
| 11 / 1                                                                        | flag                                                                             | 1 = 1                  |
| 11 / 2                                                                        | flag + time                                                                      | 1 + 6 = 7              |
| <i>g12 Control Relay Output Block / CROB (A.8.1)</i>                          |                                                                                  |                        |
| 12 / 1                                                                        | control(1) + count(UINT8,1) + on-time(UINT32,4) + off-time(UINT32,4) + status(1) | 1 + 1 + 4 + 4 + 1 = 11 |
| <i>g20 Counter, g21 Frozen Counter (A.10–A.11)</i>                            |                                                                                  |                        |
| 20 / 1                                                                        | flag + count(UINT32)                                                             | 1 + 4 = 5              |
| 20 / 2                                                                        | flag + count(UINT16)                                                             | 1 + 2 = 3              |
| 20 / 5                                                                        | count(UINT32), no flag                                                           | 4 = 4                  |
| 20 / 6                                                                        | count(UINT16), no flag                                                           | 2 = 2                  |
| 21 / 1                                                                        | flag + count(UINT32)                                                             | 1 + 4 = 5              |
| 21 / 2                                                                        | flag + count(UINT16)                                                             | 1 + 2 = 3              |
| <i>g22 Counter Event, g23 Frozen Counter Event (A.12–A.13)</i>                |                                                                                  |                        |
| 22 / 1                                                                        | flag + count(UINT32)                                                             | 1 + 4 = 5              |

| <b>g / v</b>                                                                                    | <b>Field composition</b>                              | <b>Octets</b>  |
|-------------------------------------------------------------------------------------------------|-------------------------------------------------------|----------------|
| 22 / 2                                                                                          | flag + count(UINT16)                                  | 1 + 2 = 3      |
| 22 / 5                                                                                          | flag + count(UINT32) + time                           | 1 + 4 + 6 = 11 |
| 22 / 6                                                                                          | flag + count(UINT16) + time                           | 1 + 2 + 6 = 9  |
| 23 / 1                                                                                          | flag + count(UINT32)                                  | 1 + 4 = 5      |
| 23 / 2                                                                                          | flag + count(UINT16)                                  | 1 + 2 = 3      |
| 23 / 5                                                                                          | flag + count(UINT32) + time                           | 1 + 4 + 6 = 11 |
| 23 / 6                                                                                          | flag + count(UINT16) + time                           | 1 + 2 + 6 = 9  |
| <i>g30 Analog Input, g31 Frozen Analog Input (A.14–A.15)</i>                                    |                                                       |                |
| 30 / 1                                                                                          | flag + value(INT32)                                   | 1 + 4 = 5      |
| 30 / 2                                                                                          | flag + value(INT16)                                   | 1 + 2 = 3      |
| 30 / 3                                                                                          | value(INT32), no flag                                 | 4 = 4          |
| 30 / 4                                                                                          | value(INT16), no flag                                 | 2 = 2          |
| 30 / 5                                                                                          | flag + value(FLT32)                                   | 1 + 4 = 5      |
| 31 / 1                                                                                          | flag + value(INT32)                                   | 1 + 4 = 5      |
| 31 / 2                                                                                          | flag + value(INT16)                                   | 1 + 2 = 3      |
| 31 / 3                                                                                          | flag + value(INT32) + time                            | 1 + 4 + 6 = 11 |
| 31 / 4                                                                                          | flag + value(INT16) + time                            | 1 + 2 + 6 = 9  |
| <i>g32 Analog Input Event, g33 Frozen Analog Input Event (A.16–A.17)</i>                        |                                                       |                |
| 32 / 1                                                                                          | flag + value(INT32)                                   | 1 + 4 = 5      |
| 32 / 2                                                                                          | flag + value(INT16)                                   | 1 + 2 = 3      |
| 32 / 3                                                                                          | flag + value(INT32) + time                            | 1 + 4 + 6 = 11 |
| 32 / 4                                                                                          | flag + value(INT16) + time                            | 1 + 2 + 6 = 9  |
| 32 / 5                                                                                          | flag + value(FLT32)                                   | 1 + 4 = 5      |
| 32 / 6                                                                                          | flag + value(FLT64)                                   | 1 + 8 = 9      |
| 33 / 1                                                                                          | flag + value(INT32)                                   | 1 + 4 = 5      |
| 33 / 2                                                                                          | flag + value(INT16)                                   | 1 + 2 = 3      |
| 33 / 3                                                                                          | flag + value(INT32) + time                            | 1 + 4 + 6 = 11 |
| 33 / 4                                                                                          | flag + value(INT16) + time                            | 1 + 2 + 6 = 9  |
| 33 / 5                                                                                          | flag + value(FLT32)                                   | 1 + 4 = 5      |
| 33 / 6                                                                                          | flag + value(FLT64)                                   | 1 + 8 = 9      |
| <i>g40 Analog Output Status, g41 Analog Output Command, g42 Analog Output Event (A.19–A.21)</i> |                                                       |                |
| 40 / 1                                                                                          | flag + value(INT32)                                   | 1 + 4 = 5      |
| 40 / 2                                                                                          | flag + value(INT16)                                   | 1 + 2 = 3      |
| 40 / 3                                                                                          | flag + value(FLT32)                                   | 1 + 4 = 5      |
| 41 / 1                                                                                          | value(INT32) + control status(UINT8)                  | 4 + 1 = 5      |
| 41 / 2                                                                                          | value(INT16) + control status(UINT8)                  | 2 + 1 = 3      |
| 41 / 3                                                                                          | value(FLT32) + control status(UINT8)                  | 4 + 1 = 5      |
| 42 / 1                                                                                          | flag + value(INT32)                                   | 1 + 4 = 5      |
| 42 / 2                                                                                          | flag + value(INT16)                                   | 1 + 2 = 3      |
| 42 / 3                                                                                          | flag + value(INT32) + time                            | 1 + 4 + 6 = 11 |
| 42 / 4                                                                                          | flag + value(INT16) + time                            | 1 + 2 + 6 = 9  |
| 42 / 5                                                                                          | flag + value(FLT32)                                   | 1 + 4 = 5      |
| 42 / 6                                                                                          | flag + value(FLT64)                                   | 1 + 8 = 9      |
| <i>g50 Time and Date, g51 Common Time-of-Occurrence, g52 Time Delay (A.23–A.25)</i>             |                                                       |                |
| 50 / 1                                                                                          | time                                                  | 6 = 6          |
| 50 / 2                                                                                          | time + interval(UINT32)                               | 6 + 4 = 10     |
| 50 / 3                                                                                          | time                                                  | 6 = 6          |
| 50 / 4                                                                                          | time + interval count(UINT32) + interval units(UINT8) | 6 + 4 + 1 = 11 |
| 51 / 1                                                                                          | time                                                  | 6 = 6          |
| 51 / 2                                                                                          | time                                                  | 6 = 6          |
| 52 / 1                                                                                          | delay seconds(UINT16)                                 | 2 = 2          |
| 52 / 2                                                                                          | delay milliseconds(UINT16)                            | 2 = 2          |

## 10.8 DNP3\_RANGE\_SIZE – Qualifier Range Size

Listing 31: DNP3\_RANGE\_SIZE – range field size by qualifier code

```

1 static const u8 DNP3_RANGE_SIZE[16] = {
2     0x02, 0x04, 0x08,
3     0xFF, 0xFF, 0xFF,
4     0x00, 0x01, 0x02,
5     0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF,

```

The low nibble of the qualifier byte (`qual & 0x0FU`) selects the range specifier format. This table gives the number of bytes in the range field. A value of `0xFF` means the qualifier code is rejected with `DNP3_REJECT_QUALIFIER`.

Table 11: DNP3 qualifier range-field sizes (`DNP3_RANGE_SIZE`)

| Code | Bytes | Range Format                          |
|------|-------|---------------------------------------|
| 0x00 | 2     | 1-byte start index, 1-byte stop index |
| 0x01 | 4     | 2-byte start index, 2-byte stop index |
| 0x02 | 8     | 4-byte start index, 4-byte stop index |
| 0x06 | 0     | All objects (no range field)          |
| 0x07 | 1     | 1-byte count                          |
| 0x08 | 2     | 2-byte count                          |

## 10.9 DNP3\_PREFIX\_SIZE – Object Prefix Size

Listing 32: `DNP3_PREFIX_SIZE` – prefix size by qualifier high nibble

```

1 static const u8 DNP3_PREFIX_SIZE[8] = {
2     0x00, 0x01, 0x02, 0x04, 0xFF, 0xFF, 0xFF, 0xFF
3 };

```

Bits 4–6 of the qualifier byte (extracted as `(qual >> 4) & 0x07U`) select the object prefix size. Each data point can be preceded by an index prefix of 0, 1, 2, or 4 bytes. Values 4–7 are rejected.

Table 12: DNP3 object prefix sizes (`DNP3_PREFIX_SIZE`)

| Code | Bytes | Prefix Type  |
|------|-------|--------------|
| 0    | 0     | No prefix    |
| 1    | 1     | 1-byte index |
| 2    | 2     | 2-byte index |
| 3    | 4     | 4-byte index |

## 10.10 Disabled DNP3 function codes: rationale

The table below explains why each conditionally disabled or fully rejected DNP3 function code is blocked by default.

Table 13: DNP3 function codes disabled by default and rationale

| FC        | Name                  | Enable flag                                  | Reason disabled                                                                                                                                                                                                              |
|-----------|-----------------------|----------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 0x05      | Direct Operate        | <code>SAFE_ENABLE_DNP3_DIRECT_OPERATE</code> | Single-command direct control with no two-phase confirmation. Bypasses Select-Before-Operate (SBO, FC 0x03/0x04), allowing a single packet to actuate a breaker. Enable only after IEC 62351-8 access control authorisation. |
| 0x06      | Direct Operate No Ack | (same flag as 0x05)                          | Same as Direct Operate but suppresses the acknowledgement response, removing the audit trail.                                                                                                                                |
| 0x07–0x0C | Counter Freeze        | <code>SAFE_ENABLE_DNP3_COUNTER_FREEZE</code> | Freezes and clears metering counters. Can silently reset energy values used for billing. Enable only for metering outstations.                                                                                               |
| 0x0D      | Cold Restart          | <code>SAFE_ENABLE_DNP3_COLD_RESTART</code>   | Hard reboot: outstation goes offline, loses event buffer, drops in-flight confirmations. Denial-of-service risk.                                                                                                             |

| FC             | Name                  | Enable flag                          | Reason disabled                                                                                                  |
|----------------|-----------------------|--------------------------------------|------------------------------------------------------------------------------------------------------------------|
| 0x0E           | Warm Restart          | SAFE_ENABLE_DNP3_WARM_RESTART        | Application-layer restart. Same DoS risk as Cold Restart.                                                        |
| 0x15           | Disable Unsolicited   | SAFE_ENABLE_DNP3_DISABLE_UNSOLICITED | Silences spontaneous event reports. An attacker can use it to suppress alarms during an incident.                |
| 0x16           | Assign Class          | SAFE_ENABLE_DNP3_ASSIGN_CLASS        | Reassigns data points to lower-priority event classes, hiding critical alarms from the master.                   |
| 0x19–0x1E      | File Transfer         | SAFE_ENABLE_DNP3_FILE_TRANSFER       | Read/write outstation filesystem (Group 70). Primary firmware and configuration injection path in DNP3.          |
| Groups 120–122 | Secure Authentication | SAFE_ENABLE_DNP3_SECURE_AUTH         | Variable-length challenge-response objects expand parser attack surface. Prefer IEC 62351 TLS transport instead. |

## 10.11 safe\_dnp3\_filter – Entry Point

### 10.11.1 ACSL Contract

Listing 33: safe\_dnp3\_filter ACSL contract

```

1  /*@
2  requires length <= 65535;
3  requires length == 0 || \valid_read(apdu + (0 .. length - 1));
4  assigns \nothing;
5
6  ensures apdu != \null &&
7  (length < SAFE_DNP3_MIN_APP || length > SAFE_DNP3_MAX_APP)
8  ==> \result.error == DNP3_REJECT_SIZE;
9
10 ensures apdu != \null &&
11 length >= SAFE_DNP3_MIN_APP &&
12 length <= SAFE_DNP3_MAX_APP &&
13 apdu[1] < sizeof(DNP3_FUNCTION_CODE_WHITELIST) &&
14 DNP3_FUNCTION_CODE_WHITELIST[apdu[1]] == 0
15 ==> \result.error == DNP3_REJECT_FUNCTION_CODE;
16
17 ensures \result.error == DNP3_OK
18 ==> \result.object_count <= SAFE_DNP3_MAX_OBJECTS;
19 */
20 SAFE_PURE SafeDnp3Result safe_dnp3_filter(
21     const u8 *const apdu,
22     const u32 length)

```

Three functional correctness guarantees:

- Messages outside the [DNP3\_MIN\_APP, DNP3\_MAX\_APP] range always yield DNP3\_REJECT\_SIZE.
- If the function code byte is within the whitelist array range but maps to 0, the result is guaranteed to be DNP3\_REJECT\_FUNCTION\_CODE.
- On success, the object count is bounded: object\_count <= SAFE\_DNP3\_MAX\_OBJECTS (64).

### 10.11.2 Algorithm

1. **Size guard.** Reject if length < DNP3\_MIN\_APP (2) or length > DNP3\_MAX\_APP (2048).
2. **Function code.** Read buffer[1] (byte 0 is the application control byte), store it in out.function\_code, and check against DNP3\_FUNCTION\_CODE\_WHITELIST.
3. **Short-circuit.** Confirm (0x00) and Delay Measurement (0x17) have no object headers; return OK immediately.
4. **Response IIN.** If the function code is ≥ 0x80 (Response or Unsolicited Response), skip 2 bytes of Internal Indications.
5. **Object header loop.** The main loop has these ACSL annotations:

Listing 34: Object header loop annotations

```

1  /*@
2  loop invariant 0 <= object_count <= SAFE_DNP3_MAX_OBJECTS;
3  loop invariant offset <= length;
4  loop assigns object_count, offset, out;
5  loop variant (int)(SAFE_DNP3_MAX_OBJECTS - object_count);
6  */
7  while (offset < length)

```

Each iteration processes one object header:



- (a) If `object_count`  $\geq$  `DNP3_MAX_OBJECTS` (64), reject with `DNP3_REJECT_OBJ_COUNT`.
- (b) Read 3 bytes: `group`, `var` (variation), `qual` (qualifier).
- (c) Check group against `DNP3_GROUP_WHITELIST`. Reject if entry is 0.
- (d) Check variation: reject if `var`  $\geq$  7U (out of table column bounds). Look up `DNP3_POINT_SIZE[group][var]`. Reject if `DNP3_VARIATION_INVALID`.
- (e) Decode qualifier: `range_size` from `DNP3_RANGE_SIZE[qual & 0x0FU]`, `prefix_size` from `DNP3_PREFIX_SIZE[(qual >> 4) & 0x07U]`. Reject either if `0xFF`.
- (f) Read range bytes. The range is stored in a local pointer `range_buffer = &buffer[offset]`, then `offset` is advanced by `range_size`.
- (g) Compute point count based on qualifier type (a switch on `qual & 0x0FU`):
  - `0x00`: `count = range_buffer[1] - range_buffer[0] + 1` (1-byte start-stop indices).
  - `0x01`: count from 2-byte little-endian start-stop indices.
  - `0x06`: `count = 0` (all objects, no data follows).
  - `0x07`: `count = range_buffer[0]` (1-byte count).
  - `0x08`: count from 2-byte little-endian count field.
- (h) **Data skip.** For function codes that carry data (not read `0x01`, not enable/disable unsolicited `0x14/0x15`, not assign-class `0x16`), compute the data block size and advance past it. The `is_read` flag determines this:

Listing 35: Read-like function code detection

```

1  const int is_read = (fc == 0x01U || fc == 0x16U ||
2                        fc == 0x14U || fc == 0x15U);

```

If not a read-like FC, and `point_size`  $\neq$  `DNP3_VARIATION_ANY`, and `count > 0U`:

- Packed bits (PKD): `data_bytes = (count + 7U) / 8U`
- Normal: `data_bytes = count * (point_size + prefix_size)`

- (i) Increment `object_count`.

6. Store `object_count` in `out.object_count` and return OK.

## 10.12 Worked Examples: Object Size Counting

The entire reason for the `DNP3_POINT_SIZE`, `DNP3_RANGE_SIZE`, and `DNP3_PREFIX_SIZE` tables is to let the object loop compute, for each object, exactly how many bytes it occupies so it can advance to the next object header. If that count is wrong by even one byte, every object after it is parsed at the wrong offset (a “desync”), and an attacker can use the gap to smuggle bytes the filter never inspects. The Read request in Section 10.2 carries no data, so it never exercises this arithmetic. The three examples below do.

The size of one object is:

$$\text{object\_size} = \underbrace{3}_{g + v + q} + \text{range\_size} + \text{data\_bytes}, \quad \text{data\_bytes} = \text{count} \times (\text{point\_size} + \text{prefix\_size})$$

with `count` derived from the qualifier low nibble, `prefix_size` from its high nibble, and `point_size` from `DNP3_POINT_SIZE[group]`. Packed bit-strings override `data_bytes` with  $\lceil \text{count}/8 \rceil$ .

### 10.12.1 Example 1 – analog input (g30 v1)

1E 01 00 03 06 ...

1. `0x1E` = group 30 (Analog Input); `0x01` = variation 1 (32-bit with flag).
2. `0x00` = qualifier: high nibble 0 (`prefix_size` = 0), low nibble 0 (`range_size` = 2, 1-byte start/stop indices).
3. Range field 03 06: start = 3, stop = 6, so `count` = 6 - 3 + 1 = 4 points.
4. `DNP3_POINT_SIZE[30][1]` = 5 (1 flag byte + 4-byte value).
5. `data_bytes` = 4 \* (5 + 0) = 20.

Object size = 3 + 2 + 20 = 25 bytes; the loop advances 25 bytes to the next header. If `point_size` were wrongly 3 instead of 5, the loop would land 8 bytes early, inside the data, and treat data bytes as the next object header.

### 10.12.2 Example 2 – packed binary input (g1 v1)

01 01 00 00 0C ...

1. `0x01` = group 1 (Binary Input); `0x01` = variation 1 (packed, 1 *bit* per point).
2. `0x00` = qualifier: `prefix_size` = 0, `range_size` = 2.
3. Range field 00 0C: start = 0, stop = 12, so `count` = 12 - 0 + 1 = 13 points.
4. `DNP3_POINT_SIZE[1][1]` = PKD (packed; the per-point byte size is meaningless here).

The generic formula would give `count * (point_size + prefix_size)` = 13 \* (0 + 0) = 0 data bytes, which is exactly the trap: a packed object whose `point_size` is 0 would appear to carry no data, and the following bits would be read as a fresh object header. The packed override rescues it: `data_bytes` = (count + 7U) / 8U = (13 + 7) / 8 = 2 bytes (13 bits packed eight-to-a-byte need  $\lceil 13/8 \rceil$  = 2 bytes; the + 7 is the round-up). Object size = 3+2+2 = 7 bytes.

### 10.12.3 Example 3 – time and interval (g50 v2), a corrected cell

32 02 07 01 ...

1. 0x32 = group 50 (Time and Date); 0x02 = variation 2 (absolute time *and* interval).
2. 0x07 = qualifier: `prefix_size` = 0, low nibble 7 (`range_size` = 1, the range field is a 1-byte count).
3. Range field 01: `count` = 1 (the byte value itself).
4. `DNP3_POINT_SIZE[50][2]` = 10 (DNP3TIME = 6 bytes + UINT32 interval = 4 bytes).
5. `data_bytes` = 1 \* (10 + 0) = 10.

Object size = 3 + 1 + 10 = 14 bytes. This cell was previously 6: it counted only the 6-byte timestamp and omitted the 4-byte interval. With the wrong value, `data_bytes` = 1 \* (6 + 0) = 6, the loop consumes only the timestamp, and the 4 leftover interval bytes are read as the start of the next object header, desyncing every object after it. Correcting the entry to 10 makes the byte count match the on-wire layout in IEEE Std 1815-2012 Annex A, so the loop lands exactly on the next object boundary. Every entry in `DNP3_POINTS` is verified against that Annex for the same reason.

## 10.13 All 7 validation checks: what, why, and what error code

### 10.13.1 Check 1 – Application layer size (DNP3\_REJECT\_SIZE)

**What is checked.** The application-layer fragment length must be in `[DNP3_MIN_APP, DNP3_MAX_APP]` = [2, 2048] bytes.

**Why.** IEEE 1815 (the DNP3 standard) caps the application-layer fragment at 2048 bytes. A fragment shorter than 2 bytes cannot contain the mandatory application control byte plus function code. Both bounds are checked before any byte is read beyond the header.

**Attack prevented.** Sending a 1-byte fragment to crash the function-code read (`buffer[1]`) is blocked here. Sending an oversized fragment to exhaust the parser's iteration limit is also blocked.

### 10.13.2 Check 2 – Function code whitelist (DNP3\_REJECT\_FUNCTION\_CODE)

**What is checked.** `buffer[1]` (the function code byte) is looked up in `DNP3_FUNCTION_CODE_WHITELIST`. Codes not in the whitelist are rejected.

**Why.** DNP3 defines function codes from 0x00 through 0x1F plus responses (0x81–0x83). Operational IEC 61850 and SCADA deployments use a small subset: read (0x01), write (0x02), select-before-operate (0x03/0x04), direct operate (0x05/0x06 – disabled by default), confirm (0x00), and responses. Dangerous codes such as cold restart (0x0D) and warm restart (0x0E) are disabled by default in `safe_config.h` and rejected here unless explicitly re-enabled.

**Attack prevented.** An attacker who sends a cold restart request (0x0D) to knock an RTU offline is rejected unless the operator has explicitly enabled `SAFE_ENABLE_DNP3_COLD_RESTART`.

### 10.13.3 Check 3 – Object group whitelist (DNP3\_REJECT\_GROUP)

**What is checked.** Each object header's group byte is looked up in `DNP3_GROUP_WHITELIST`. Groups not in the whitelist are rejected.

**Why.** DNP3 defines object groups 0–122 plus secure-auth groups 120–122. In practice, a substation automation deployment uses analog inputs (Group 30), binary inputs (Group 1), counters (Group 20), binary outputs (Group 10), and a handful of others. Secure-authentication groups (120–122) are disabled by default because they expand parser attack surface and are superseded by TLS transport.

**Attack prevented.** An injected packet referencing Group 70 (file transfer) to upload malicious firmware is blocked unless `SAFE_ENABLE_DNP3_FILE_TRANSFER` is set.

### 10.13.4 Check 4 – Variation validity (DNP3\_REJECT\_VARIATION)

**What is checked.** The variation byte is checked against the `DNP3_POINT_SIZE` 2-D lookup table for the given group. If the variation index is out of table bounds or maps to `DNP3_VARIATION_INVALID`, the frame is rejected.

**Why.** Each DNP3 group supports a small set of variations (e.g., Group 30 has variations 1–6 for different analog data widths). Variation 0 means “any,” while other values outside the table indicate a malformed or probing packet.

**Attack prevented.** Sending Group 30 Variation 255 to confuse the point-size calculation – which directly feeds into the data-skip arithmetic – is blocked at this check.

### 10.13.5 Check 5 – Qualifier code validity (DNP3\_REJECT\_QUALIFIER)

**What is checked.** The qualifier byte is split into range code (low nibble, via `DNP3_RANGE_SIZE`) and prefix code (bits 6–4, via `DNP3_PREFIX_SIZE`); bit 7 is reserved (IEEE 1815) and must be zero. Either lookup resolving to 0xFF (“invalid”), or a set reserved bit 7, causes rejection (`qualifier & 0x80`).

**Why.** The qualifier determines how object addresses and counts are encoded in the following bytes (e.g., start-stop 1-byte indices vs. 2-byte indices vs. all-objects). An invalid qualifier code means the parser cannot determine how many range bytes follow, making all subsequent offset arithmetic undefined.

**Attack prevented.** Sending a malformed qualifier byte (e.g., range code 0x0C) to make the parser skip an unexpected number of range bytes, advancing the offset to attacker-controlled territory, is blocked here.

### 10.13.6 Check 6 – Object header count (DNP3\_REJECT\_OBJ\_COUNT)

**What is checked.** If the count of object headers seen so far reaches DNP3\_MAX\_OBJECTS (64), the frame is rejected.

**Why.** Legitimate DNP3 read requests use fewer than 10 object headers. The limit of 64 provides generous headroom while bounding the number of iterations in the main parsing loop, guaranteeing termination within a fixed number of steps independent of the input.

**Attack prevented.** Sending a fragment with 200 tiny object headers to exhaust CPU time in the parsing loop is blocked at the 64-header limit.

### 10.13.7 Check 7 – Truncation (DNP3\_REJECT\_TRUNCATE)

**What is checked.** Before reading range bytes and data bytes, the filter verifies that `offset + range_size <= length` and that the computed data block does not extend past the end of the buffer.

**Why.** An object header can claim a range or data size larger than the remaining buffer. Without this check, the offset arithmetic would reference bytes past the end of the input.

**Attack prevented.** A READ request with a start-stop range of 0–9999 (implying 10 000 points of data) inside a 100-byte fragment is caught here when the computed data-block size exceeds the buffer.

## 11 SafeIEC104 – APCI Header + TypeID/VSQ Whitelists

### 11.1 What is IEC 60870-5-104?

IEC 60870-5-104 (commonly shortened to IEC 104 or T104) is the TCP/IP variant of the IEC 60870-5 telecontrol standard. It maps the well-known IEC 60870-5-101 serial ASDU format onto a fixed 6-byte Application Protocol Control Information (APCI) header carried directly over TCP.

Key concepts:

- **APCI:** The 6-byte envelope around every IEC 104 message. Byte 0 is always 0x68 (start); Byte 1 is the length of the remaining bytes; Bytes 2–5 are four control-field bytes that identify the frame type and carry sequence numbers.
- **Frame types:** Three types distinguished by the low bits of control field 1 (CF1):
  - **I-frame** (CF1 bit 0 = 0): Information frame. Carries an ASDU with TypeID, VSQ, COT, CA, and information objects. Used for all data transfer.
  - **S-frame** (CF1 = 0x01): Supervisory frame. Carries only receive sequence numbers (acknowledgement). No ASDU.
  - **U-frame** (CF1 bit 0 = 1, bit 1 = 1): Unnumbered frame. Connection management (STARTDT, STOPDT, TESTFR act/con). No ASDU.
- **TypeID:** A 1-byte field in the ASDU that identifies the data type of all information objects in the message. TypeID names follow a four-part convention:

```
M  _  EI  _  NA  _  1
|   |   |   |   |
|   |   |   |   +--- Version (always 1)
|   |   |   |
|   |   |   +--- Time-tag qualifier
|   |   |       NA = no time tag
|   |   |       TA = CP24Time2a (3 bytes)
|   |   |       TB = CP56Time2a (7 bytes)
|   |   |
|   +--- Information type
|       SP = Single Point (binary status)
|       DP = Double Point (binary status)
|       ME = Measured Value
|       BO = Bitstring of 32 bits
|       EI = End of Initialisation
|       IC = Interrogation Command
|       CS = Clock Synchronisation
|       SC = Single Command
|
+--- Direction
    M = Monitored   (outstation -> control centre)
    C = Control     (control centre -> outstation)
    P = Parameter
    F = File transfer
```

Common examples: M.SP.NA.1 (TypeID 1) = monitored single-point, no time tag; M.EI.NA.1 (TypeID 70) = end of initialisation; M.ME.TF.1 (TypeID 36) = measured float with CP56Time2a; C.SC.NA.1 (TypeID 45) = single-point command; C.IC.NA.1 (TypeID 100) = interrogation command.

- **VSQ** (Variable Structure Qualifier): A 1-byte field immediately after TypeID. Bit 7 is the SQ (sequence) flag; bits 6:0 give the number of information objects (max 127). When SQ=0 each object has its own 3-byte IOA; when SQ=1 only the first IOA is present and subsequent objects are consecutive.
- **Information objects**: After the 12-byte ASDU header (TypeID 1 + VSQ 1 + COT 2 + CA 2 = 6 bytes in the ASDU, preceded by 6-byte APCI) comes a sequence of information objects, each of the form: 3-byte IOA followed by IEC104\_ELEMENT\_SIZE[TypeID] bytes of element data.

The filter operates on the raw APDU (the 6-byte APCI header plus any ASDU that follows), exactly as received from the TCP stream after TPDU reassembly.

## 11.2 Worked Example: IEC 104 I-frame (M\_SP\_NA\_1)

A minimal I-frame carrying one single-point status object:

68 0E 00 00 00 00 01 01 14 00 01 00 01 05 00 00

1. **Byte 0**: Start byte = 0x68. Accepted.
2. **Byte 1**: L = 0x0E = 14. Total APDU = 1 + 1 + 14 = 16 bytes ( $\leq 255$ ). Size check passed.
3. **Bytes 2–5**: CF1=0x00, CF2=0x00, CF3=0x00, CF4=0x00. CF1 bit 0 = 0  $\Rightarrow$  I-frame. SSN = 0, RSN = 0.
4. **Byte 6**: TypeID = 0x01 (M\_SP\_NA\_1 – single-point information without time tag). Whitelist: IEC104\_TYPEID\_WHITELIST == 1  $\Rightarrow$  accepted. Element size: IEC104\_ELEMENT\_SIZE[0x01] == 1 byte.
5. **Byte 7**: VSQ = 0x01: SQ=0, N=1.  $N = 1 \leq 127 \Rightarrow$  accepted.
6. **Bytes 8–9**: COT = 0x14 0x00 = 20 (spontaneous). Accepted.
7. **Bytes 10–11**: CA = 0x01 0x00 = 1. Common address.
8. **Bytes 12–14**: IOA = 0x05 0x00 0x00 = 5. 3-byte little-endian information object address.
9. **Byte 15**: Element = 0x00. Single-point value: OFF, valid. One element of IEC104\_ELEMENT\_SIZE[1] = 1 byte.
10. **Body size check**:  $N \times (\text{IOA\_LEN} + \text{elem\_size}) = 1 \times (3 + 1) = 4$  bytes. Bytes 12–15 = 4 bytes available.  $L - 6 = 14 - 6 = 8$  bytes for ASDU; header 6 bytes (TypeID+VSQ+COT+CA) + body 4 bytes = 10 bytes  $\leq 14 - 0 = \text{OK}$ .
11. **Result**: IEC104\_OK, frame\_type = 0 (I-frame), object\_count = 1.

## 11.3 Implementation

The IEC 104 filter validates the APCI header, identifies the frame type, and for I-frames validates the ASDU TypeID, VSQ, and information object body size. For U-frames the control field is checked against a whitelist. S-frames require only the APCI structure check. This section covers all three lookup tables and the main filter function.

## 11.4 IEC 60870-5-104 APDU Wire Format

**APCI** (Application Protocol Control Information, always 6 bytes):

| B 0           | B 1                | B 2 (CF1)            | B 3 (CF2)                              | B 4 (CF3) | B 5 (CF4) |
|---------------|--------------------|----------------------|----------------------------------------|-----------|-----------|
| Start<br>0x68 | Length<br>APDU len | Frame type<br>I/S/U? | Sequence numbers<br>(send/receive SNs) |           |           |

**CF1 frame-type encoding** (bits 1–0 of byte 2):

- bit0=0: **I-frame** (data, carries ASDU)
- CF1=0x01: **S-frame** (supervisory / ACK only)
- bit0=1, bit1=1: **U-frame** (connection management: STARTDT / STOPDT / TESTFR)

**I-frame ASDU** (bytes 6+ when CF1 bit0=0):

| B 6    | B 7 (VSQ) | B 8–9 (COT) | B 10–11 (CA) | B 12–14 (IOA) | B 15+ (data)                                                          |
|--------|-----------|-------------|--------------|---------------|-----------------------------------------------------------------------|
| TypeID | SQ + N    | Cause       | Station      | Object addr   | $N \text{ objects} \times \text{ELEM\_SIZE}[\text{TypeID}] \text{ B}$ |

**VSQ byte**: bit 7 = SQ (0 = each object has its own IOA; 1 = only first IOA present, objects contiguous). Bits 6:0 = N (object count, 1–127). COT and CA are little-endian 2-byte fields. IOA is a little-endian 3-byte address.

## 11.5 IEC104\_ELEMENT\_SIZE – TypeID Element Size Array

IEC104\_ELEMENT\_SIZE and IEC104\_TYPEID\_WHITELIST are both generated from a single set of X-macro lists (IEC104\_TYPES\_BASE / \_COMMANDS / \_CLOCK), so the two tables can never drift apart. This is the X-macro pattern of Section 10.7, expanded twice: once as [id] = (u8) (sz) to build the size table and once as [id] = 1U to build the permission table. Two tables are needed here, rather than the single zero-default test used for DNP3, because one permitted TypeID (C\_RD\_NA\_1, read command) has a genuine element size of zero; a separate permission table keeps that legitimate zero from being read as “unlisted”.

Listing 36: IEC104\_ELEMENT\_SIZE – 256-entry element size lookup, generated from the TypeID lists

```

1 #define IEC104_TYPES_BASE(X) \
2 X( 1, 1) /* M_SP_NA_1 single-point, no time */ \
3 X( 3, 1) /* M_DP_NA_1 double-point, no time */ \
4 X( 5, 2) /* M_ST_NA_1 step position */ \
5 X( 7, 5) /* M_BO_NA_1 bitstring */ \
6 X( 9, 3) /* M_ME_NA_1 meas. NVA */ \
7 X(11, 3) /* M_ME_NB_1 meas. SVA */ \
8 X(13, 5) /* M_ME_NC_1 meas. short float */ \
9 X(15, 5) /* M_IT_NA_1 integrated totals */ \
10 X(21, 2) /* M_ME_ND_1 meas. NVA, no QDS */ \
11 X(30, 8) /* M_SP_TB_1 single-point + time */ \
12 X(31, 8) /* M_DP_TB_1 double-point + time */ \
13 X(32, 9) /* M_ST_TB_1 step position + time */ \
14 X(33,12) /* M_BO_TB_1 bitstring + time */ \
15 X(34,10) /* M_ME_TD_1 NVA + time */ \
16 X(35,10) /* M_ME_TE_1 SVA + time */ \
17 X(36,12) /* M_ME_TF_1 float + time */ \
18 X(37,12) /* M_IT_TB_1 integ. totals + time */ \
19 X(38,10) /* M_EP_TD_1 prot. event + time */ \
20 X(39,11) /* M_EP_TE_1 packed start evt + time */ \
21 X(40,11) /* M_EP_TF_1 packed out ckt + time */ \
22 X(70, 1) /* M_EI_NA_1 end of initialisation */ \
23 X(100, 1) /* C_IC_NA_1 interrogation command */ \
24 X(101, 1) /* C_CI_NA_1 counter interrogation */ \
25 X(102, 0) /* C_RD_NA_1 read (IOA only) */ \
26
27 #define IEC104_TYPES_COMMANDS(X) \
28 X(45, 1) /* C_SC_NA_1 single command */ \
29 X(46, 1) /* C_DC_NA_1 double command */ \
30 X(47, 1) /* C_RC_NA_1 regulating step */ \
31 X(48, 3) /* C_SE_NA_1 setpoint NVA */ \
32 X(49, 3) /* C_SE_NB_1 setpoint SVA */ \
33 X(50, 5) /* C_SE_NC_1 setpoint float */ \
34 X(51, 4) /* C_BO_NA_1 bitstring command */ \
35 X(58, 8) /* C_SC_TA_1 single cmd + time */ \
36 X(59, 8) /* C_DC_TA_1 double cmd + time */ \
37 X(60, 8) /* C_RC_TA_1 step cmd + time */ \
38 X(61,10) /* C_SE_TA_1 setpoint NVA + time */ \
39 X(62,10) /* C_SE_TB_1 setpoint SVA + time */ \
40 X(63,12) /* C_SE_TC_1 setpoint float + time */ \
41 X(64,11) /* C_BO_TA_1 bitstring cmd + time */ \
42
43 #define IEC104_TYPES_CLOCK(X) \
44 X(103, 7) /* C_CS_NA_1 clock synchronisation */ \
45
46 static const u8 IEC104_ELEMENT_SIZE[256] = {
47 #define X(id, sz) [id] = (u8)(sz),
48 IEC104_TYPES_BASE(X)
49 #ifdef SAFE_ENABLE_IEC104_COMMANDS
50 IEC104_TYPES_COMMANDS(X)
51 #endif
52 #ifdef SAFE_ENABLE_IEC104_CLOCK_SYNC
53 IEC104_TYPES_CLOCK(X)
54 #endif
55 #undef X
56 };

```

IEC104\_ELEMENT\_SIZE[typeID] gives the number of element bytes per information object (after the 3-byte IOA). The body size of an I-frame ASDU is computed as  $N \times (3 + \text{IEC104\_ELEMENT\_SIZE}[\text{TypeID}])$  where  $N$  is the object count from VSQ. TypeID 102 (C\_RD\_NA\_1) has a zero element size, which is why a separate IEC104\_TYPEID\_WHITELIST array is needed (a zero entry in IEC104\_ELEMENT\_SIZE alone cannot distinguish “permitted with 0 bytes” from “not permitted”).

Table 14: IEC 104 permitted TypeIDs and per-element data sizes (IEC104\_ELEMENT\_SIZE)

| TypeID | Mnemonic      | Description                   | Elem bytes |
|--------|---------------|-------------------------------|------------|
| 1      | M_SP_NA_1     | Single-point information      | 1          |
| 3      | M_DP_NA_1     | Double-point information      | 1          |
| 5      | M_ST_NA_1     | Step position                 | 2          |
| 7      | M_BO_NA_1     | Bitstring of 32 bits          | 5          |
| 9      | M_ME_NA_1     | Measured value, normalised    | 3          |
| 11     | M_ME_NB_1     | Measured value, scaled        | 3          |
| 13     | M_ME_NC_1     | Measured value, short float   | 5          |
| 15     | M_IT_NA_1     | Integrated totals             | 5          |
| 21     | M_ME_ND_1     | Normalised value (no quality) | 2          |
| 30–40  | M_SP_TB_1 ... | With CP56Time2a timestamp     | 8–12       |
| 70     | M_EI_NA_1     | End of initialisation         | 1          |
| 45–51  | C_SC_NA_1 ... | Commands (optional)           | 1–5        |
| 58–64  | C_SC_TA_1 ... | Commands with time (optional) | 8–12       |

| TypeID  | Mnemonic       | Description                      | Elem bytes |
|---------|----------------|----------------------------------|------------|
| 100–102 | C_IC/C_CI/C_RD | Interrogation (optional)         | 0–1        |
| 103     | C_CS_NA_1      | Clock synchronisation (optional) | 7          |

## 11.6 IEC104.TYPEID\_WHITELIST – TypeID Permission Array

Listing 37: IEC104.TYPEID\_WHITELIST – 256-entry TypeID whitelist, generated from the same TypeID lists

```

1 static const u8 IEC104_TYPEID_WHITELIST[256] = {
2 #define X(id, sz) [id] = 1U,
3 IEC104_TYPES_BASE(X)
4 #ifdef SAFE_ENABLE_IEC104_COMMANDS
5 IEC104_TYPES_COMMANDS(X)
6 #endif
7 #ifdef SAFE_ENABLE_IEC104_CLOCK_SYNC
8 IEC104_TYPES_CLOCK(X)
9 #endif
10 #undef X
11 };

```

This is a 256-entry boolean whitelist. A zero entry means the TypeID is rejected regardless of any value in IEC104\_ELEMENT\_SIZE. The separation between IEC104\_ELEMENT\_SIZE and IEC104.TYPEID\_WHITELIST is intentional: TypeID 102 (C\_RD\_NA\_1) carries no element bytes (size 0) but is nonetheless a valid TypeID when interrogation is enabled. A check of IEC104\_ELEMENT\_SIZE[102] != 0 would incorrectly reject it.

## 11.7 IEC104.UFRAME\_WHITELIST – U-Frame Control Field Whitelist

Listing 38: IEC104.UFRAME\_WHITELIST – 256-entry U-frame whitelist

```

1 static const u8 IEC104_UFRAME_WHITELIST[256] = {
2 [0x07] = 1, /* STARTDT act */
3 [0x0B] = 1, /* STARTDT con */
4 [0x13] = 1, /* STOPDT act */
5 [0x23] = 1, /* STOPDT con */
6 [0x43] = 1, /* TESTFR act */
7 [0x83] = 1, /* TESTFR con */
8 };

```

For U-frames, the APCI carries the function in CF1 rather than in an ASDU. Only six values are valid per the IEC 60870-5-104 standard: STARTDT activate (0x07) and confirm (0x0B), STOPDT activate (0x13) and confirm (0x23), and TESTFR activate (0x43) and confirm (0x83). Any other CF1 value for a U-frame is rejected.

**What each command does.**

- STARTDT** Activates the data transfer channel. The master sends STARTDT activate; the outstation replies with STARTDT confirm. I-frames (data) may not flow until this handshake completes. Reconnecting after a TCP restart always begins with STARTDT.
- STOPDT** Suspends data transfer. The master sends STOPDT activate; the outstation confirms. Used during maintenance or controlled shutdown. After STOPDT, the TCP connection stays open but I-frames are blocked.
- TESTFR** Keepalive heartbeat. Either side may send TESTFR activate; the other must reply with TESTFR confirm within the inactivity timeout T3 (default 20s). Prevents the TCP connection from being silently dropped by NAT devices during quiet periods. Carries no data.

Table 15: IEC 104 U-frame CF1 whitelist

| CF1  | Name             | Direction           |
|------|------------------|---------------------|
| 0x07 | STARTDT activate | Master → Outstation |
| 0x0B | STARTDT confirm  | Outstation → Master |
| 0x13 | STOPDT activate  | Master → Outstation |
| 0x23 | STOPDT confirm   | Outstation → Master |
| 0x43 | TESTFR activate  | Either              |
| 0x83 | TESTFR confirm   | Either              |

## 11.8 Disabled IEC 104 TypeIDs: rationale

The table below explains why each rejected or conditionally disabled TypeID group is blocked by default.

Table 16: IEC 104 excluded TypeID groups and rationale

| TypeIDs                       | Group                      | Reason disabled                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |
|-------------------------------|----------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 2, 4, 6, 8, 10, 12, 14, 16–20 | Legacy CP24Time2a variants | These TypeIDs carry a 3-byte CP24Time2a timestamp. The CP24Time2a field is variable in interpretation (it encodes only minutes and milliseconds, losing the day/month/year), creating parser ambiguity compared to the full CP56Time2a. IEC 60870-5-104 over TCP/IP mandates CP56Time2a; the corresponding TypeIDs with CP56Time2a timestamps are TypeIDs 30–40. Accepting the CP24 variants alongside the CP56 variants doubles the number of monitoring TypeIDs the parser must handle for no operational benefit. Use TypeIDs 30–40 instead.           |
| 45–51, 58–64                  | Commands (optional)        | IEC 104 commands (single-point trip/close, double-point, step regulation, setpoint normalised/scaled/float, bitstring) directly actuate substation equipment: they open and close circuit breakers, change protection relay setpoints, and adjust tap changers. Enabling command TypeIDs requires explicit operational authorisation per IEC 62351-8 role-based access control. The monitoring-only default prevents a compromised HMI or SCADA server from issuing commands through the filter. Enable via <code>SAFE_ENABLE_IEC104_COMMANDS</code> .    |
| 103                           | Clock synchronisation      | <code>C_CS_NA.1</code> is a legitimate operational message, but clock injection attacks can desynchronise protection relays: a fabricated clock-sync message shifting the relay’s time reference causes CP56Time2a timestamps in GOOSE and SV frames to become incorrect, corrupting disturbance records and event logs. The preferred approach is NTP or GPS/PPS at the RTU level. Enable via <code>SAFE_ENABLE_IEC104_CLOCK_SYNC</code> only if the RTU has no independent time source.                                                                 |
| 104–106                       | Test / reset / delay       | <code>C_TS_NA.1</code> (test), <code>C_RP_NA.1</code> (reset process), <code>C_CD_NA.1</code> (delay acquisition) create availability risk: the reset command restarts the outstation process and the delay-acquisition command changes the outstation’s internal time delay. None of these are needed in normal monitoring operation. Blocked unconditionally (no compile guard).                                                                                                                                                                        |
| 110–113                       | Parameter services         | <code>P_ME_NA.1</code> through <code>P_AC_NA.1</code> allow the master to program measurement parameters (deadbands, thresholds, activation flags) directly on the RTU at runtime. Changing protection relay measurement parameters via IEC 104 is a high operational risk: an attacker could widen deadbands to suppress alarm reporting or change threshold values. Parameter programming should go through engineering tools (IEDScout, SCL import) via dedicated commissioning sessions, not through the live SCADA channel. Blocked unconditionally. |
| 120–126                       | File services              | <code>F_FR_NA.1</code> through <code>F_SG_NA.1</code> provide firmware upload, sector read/write, and configuration download to the RTU filesystem. This is the direct firmware injection path: an attacker with access to the IEC 104 session can upload a modified firmware image or replace protection settings. The same reasoning applies as for DNP3 file transfer groups and MMS file services. Blocked unconditionally.                                                                                                                           |

## 11.9 safe\_iec104\_filter – Entry Point

### 11.9.1 ACSL Contract

Listing 39: `safe_iec104_filter` ACSL contract

```

1 /*@
2  requires length <= 65535;
3  requires length == 0 || \valid_read(apdu + (0 .. length - 1));
4  assigns \nothing;
5  ensures apdu != \null &&

```



```

6 |         (length < IEC104_MIN_APDU || length > SAFE_IEC104_MAX_APDU)
7 |         ==> \result.error == IEC104_REJECT_SIZE;
8 |     ensures apdu != \null &&
9 |           length >= IEC104_MIN_APDU && length <= SAFE_IEC104_MAX_APDU &&
10 |          apdu[0] != IEC104_START_BYTE
11 |         ==> \result.error == IEC104_REJECT_START;
12 |     ensures \result.error == IEC104_OK && \result.frame_type == 0U
13 |         ==> \result.object_count <= SAFE_IEC104_MAX_OBJECTS;
14 | */
15 | SAFE_PURE SafeIec104Result safe_iec104_filter(
16 |     const u8 *const apdu,
17 |     const u32 length)

```

Three functional correctness guarantees:

- Messages shorter than 6 bytes or longer than 255 bytes always yield `IEC104_REJECT_SIZE`.
- Messages of a valid size that do not start with `0x68` always yield `IEC104_REJECT_START`.
- On success with an I-frame (`frame_type == 0`), the object count is bounded: `object_count <= 127`.

## 11.9.2 Algorithm

1. **Null and size guard.** Reject with `IEC104_REJECT_SIZE` if `apdu == NULL` or if `length < IEC104_MIN_APDU` (6) or `length > SAFE_IEC104_MAX_APDU` (255).
2. **Start byte.** Check `apdu[0] == 0x68`. Reject with `IEC104_REJECT_START` if not.
3. **Length field.** Reject with `IEC104_REJECT_LEN` if `apdu[1] + 2 > length` (claimed APDU exceeds buffer) or if `apdu[1] < 4` (minimum four control-field bytes required).
4. **Frame type.** Decode `CF1` (`apdu[2]`) bits [1:0]:
  - bit 0 = 0  $\Rightarrow$  I-frame. Proceed to ASDU checks (step 6).
  - `CF1 == 0x01`  $\Rightarrow$  S-frame. Check `L = 4` (exactly four control-field bytes; anything else is `IEC104_REJECT_LEN`). Store `frame_type = 1`. No ASDU. Return OK.
  - Otherwise (bit 0 = 1, `CF1  $\neq$  0x01`)  $\Rightarrow$  U-frame. Check `L = 4` (same constraint). Reject with `IEC104_REJECT_LEN` if not. Check `CF1` against `IEC104_UFRAME_WHITELIST`. Reject with `IEC104_REJECT_UFRAME` if not whitelisted. Store `frame_type = 2`. Return OK.
5. **I-frame ASDU minimum size.** Reject with `IEC104_REJECT_LEN` if `L` field  $< 10$ : the ASDU requires at least CF (4) + TypeID (1) + VSQ (1) + COT (2) + CA (2) = 10 bytes within the L-counted region.
6. **TypeID check.** Read `typeID = apdu[6]`. Check `IEC104_TYPEID_WHITELIST[typeID] == 1`. Reject with `IEC104_REJECT_TYPEID` if not.
7. **VSQ check.** Read `vsq = apdu[7]`. Extract `N = vsq & 0x7FU` (bits 6:0). Reject with `IEC104_REJECT_OBJECTS` if `N == 0` or `N > IEC104_MAX_OBJECTS` (127).
8. **Body size check.** Compute `actual_body = apdu[1] - 10`. Compute `expected_body`: if `SQ=0`,  $N \times (\text{IOA\_LEN} + \text{ELEM\_SIZE}[\text{TypeID}])$ ; if `SQ=1`,  $\text{IOA\_LEN} + N \times \text{ELEM\_SIZE}[\text{TypeID}]$  (where `IOA_LEN = 3`). Reject with `IEC104_REJECT_TRUNCATE` if `actual_body  $\neq$  expected_body`.
9. Store `frame_type = 0`, `object_count = N`. Return `IEC104_OK`.

## 11.10 All 8 validation checks: what, why, and what error code

### 11.10.1 Check 1 – APDU size (`IEC104_REJECT_SIZE`)

**What is checked.** The APDU length must be in [6, 255] bytes. 6 is the minimum (6-byte APCI header only, for S/U-frames). 255 is the IEC 60870-5-104 protocol maximum (`L` field is one byte; max value 253 + 2 start/length bytes = 255 total).

**Why.** The `L` field in the APCI is an 8-bit unsigned integer counting the bytes *after* byte 1 (i.e., the four control-field bytes `CF1`–`CF4` plus the ASDU), but *not* counting the start byte (byte 0, always `0x68`) or `L` itself (byte 1). Total frame length = `L` + 2. The maximum `L` value of 253 gives the maximum APDU of 253 + 2 = 255 bytes. A fragment shorter than 6 bytes cannot contain the 4-byte control field. Both bounds are checked before any byte beyond byte 0 is read.

**Attack prevented.** Sending a 2-byte APDU to trigger an out-of-bounds read of the control fields (`apdu[2]`) is blocked here. Sending an oversized APDU to overflow the body size arithmetic is also blocked.

### 11.10.2 Check 2 – Start byte (`IEC104_REJECT_START`)

**What is checked.** Byte 0 of the APDU must be exactly `0x68`.

**Why.** IEC 60870-5-104 mandates `0x68` as the unique start byte. Every valid IEC 104 APDU begins with this sentinel. Its absence means the buffer does not contain an IEC 104 frame.

**Attack prevented.** Cross-protocol injection (e.g., a DNP3 application layer buffer passed to the IEC 104 filter) is rejected here before any further parsing.

### 11.10.3 Check 3 – `L` field consistency (`IEC104_REJECT_LEN`)

**What is checked.** The declared length field `apdu[1]` (`L`) must satisfy  $L + 2 \leq \text{length}$  and  $L \geq 4$  (minimum four control-field bytes). The `L` field encodes the number of bytes from byte 2 to the end of the APDU.

**Why.** An inconsistency between the declared `L` field and the buffer length indicates either a fragmented TCP delivery (frame not yet complete) or a deliberate length forgery. The filter must see a complete, self-consistent APDU.

**Attack prevented.** Sending an APDU with `L = 200` in a 50-byte buffer (which would cause body size calculations to trust a length larger than the available data) is caught here.

#### 11.10.4 Check 4 – Frame type (IEC104\_REJECT\_UFRAME)

**What is checked.** For U-frames, CF1 (byte 2) is checked against IEC104\_UFRAME\_WHITELIST. Only 6 values are valid (STARTDT act/con, STOPDT act/con, TESTFR act/con).

**Why.** The IEC 60870-5-104 standard defines exactly three U-frame functions, each with an activate and a confirm variant. Any other CF1 value for a U-frame is undefined by the standard and indicates a malformed or injected frame.

**Attack prevented.** A synthetic U-frame with CF1 = 0xFF would not match any of the six valid values and is rejected rather than silently accepted as a no-op.

#### 11.10.5 Check 5 – TypeID whitelist (IEC104\_REJECT\_TYPEID)

**What is checked.** Byte 6 of the APDU (TypeID) is looked up in IEC104\_TYPEID\_WHITELIST. TypeIDs not in the whitelist are rejected.

**Why.** IEC 60870-5-104 defines TypeIDs from 1 to 126. In practice a monitoring-only deployment uses around 22 TypeIDs. Legacy CP24Time2a variants, parameter services, and file services are rejected by default to prevent the filter from having to know the element sizes of dozens of rarely-used or hazardous TypeIDs.

**Attack prevented.** Injecting a TypeID 110 (P\_ME\_NA\_1, parameter programming) frame to change protection relay measurement parameters is blocked unless SAFE\_ENABLE\_IEC104\_COMMANDS is set. TypeID 104 (C\_TS\_NA\_1, test command) is blocked unconditionally; this intercepts GHSA-75pr-rr3v-j6px (null-pointer dereference in *lib60870-C* when a test command with a malformed ASDU is received) and Crash B (TestCommand\_getFromBuffer, *lib60870-C* v2.4.0), both of which require TypeID 104 to reach the vulnerable parsing code.

#### 11.10.6 Check 6 – VSQ validity (IEC104\_REJECT\_OBJECTS)

**What is checked.** The object count  $N$  (bits 6:0 of VSQ) must be in [1, 127].

**Why.** A VSQ of zero (no objects) would make the ASDU body empty but syntactically present, creating ambiguity. The maximum of 127 follows directly from the 7-bit field width in the IEC 60870-5-104 standard.

**Attack prevented.** A VSQ claiming 0 objects would cause the body check to compute a zero body size, allowing a crafted ASDU with a forged TypeID but no objects to pass silently.

#### 11.10.7 Check 7 – Body size (IEC104\_REJECT\_TRUNCATE)

**What is checked.** The computed body size  $N \times (\text{IOA\_LEN} + \text{IEC104\_ELEMENT\_SIZE}[\text{TypeID}])$  must not exceed the remaining buffer after the ASDU header.

**Why.** Each information object is exactly  $\text{IOA\_LEN} + \text{IEC104\_ELEMENT\_SIZE}[\text{TypeID}]$  bytes wide. If the declared object count  $N$  implies more bytes than are present in the APDU, the frame is either truncated or deliberately crafted to claim more objects than exist. In *lib60870-C*, `InformationObject_ParseObjectAddress` and approximately 150 `*getFromBuffer` functions carry no buffer-size argument and cannot perform bounds checking themselves; this check is the only line of defence before the overread.

**Attack prevented.** An I-frame with TypeID 13 (5-byte float) and VSQ claiming 100 objects in a 30-byte APDU would require  $100 \times 8 = 800$  body bytes; the mismatch is caught here before any per-object processing. Crash A (`InformationObject_ParseObjectAddress` OOB read, *lib60870-C* v2.4.0): TypeID 17 (M\_EP\_TA\_1) with VSQ SQ=1 and inflated object count triggers the overread; the body size check intercepts this before the vulnerable parsing code is reached.

#### 11.10.8 Check 8 – Minimum ASDU size (IEC104\_REJECT\_LEN)

**What is checked.** For I-frames, the L field must be at least 10 (4 CF bytes + 6 ASDU header bytes). This guarantees the ASDU header fields TypeID (1) + VSQ (1) + COT (2) + CA (2) are present.

**Why.** Without this check, an I-frame of exactly 6 bytes (APCI header only) would have CF1 bit 0 = 0 (I-frame), but reading TypeID at byte 6 would be an out-of-bounds access. This check closes the gap between the size guard (minimum 6) and the byte 6 read.

**Attack prevented.** A 6-byte packet with CF1 bit 0 = 0 (I-frame flag) but no ASDU bytes cannot reach the TypeID check.

### 11.11 Error Code Reference

Table 17: IEC 104 rejection codes

| Code | Constant               | Meaning                                                 |
|------|------------------------|---------------------------------------------------------|
| 0    | IEC104_OK              | APDU is valid                                           |
| 1    | IEC104_REJECT_SIZE     | Length outside [6, 255]                                 |
| 2    | IEC104_REJECT_START    | First byte is not 0x68                                  |
| 3    | IEC104_REJECT_LEN      | L field inconsistent with buffer length                 |
| 4    | IEC104_REJECT_UFRAME   | U-frame CF1 not in whitelist                            |
| 5    | IEC104_REJECT_TYPEID   | TypeID not in IEC104_TYPEID_WHITELIST or ASDU too short |
| 6    | IEC104_REJECT_OBJECTS  | Object count $N$ is 0 or $> 127$                        |
| 7    | IEC104_REJECT_TRUNCATE | Body bytes implied by $N$ do not match available bytes  |

## 12 SafeGOOSE – State Machine + Data Value Validation

### 12.1 What is GOOSE?

GOOSE (Generic Object Oriented Substation Event) is the fastest protocol in IEC 61850. When a protection relay detects a fault (e.g., overcurrent on a feeder), it must signal other relays and circuit breakers within **4 milliseconds**. GOOSE achieves this by sending raw Ethernet frames (Layer 2 multicast, EtherType 0x88B8) with no TCP/IP overhead.

**Why Layer 2 (no TCP/IP)?** TCP adds acknowledgement, retransmission timers, and per-connection state; even a single 200  $\mu$ s retransmission would blow the 3 ms latency budget. By skipping the IP and TCP layers, GOOSE frames travel directly from the relay’s Ethernet port to every subscribed device on the process-bus switch. There is no connection setup, no SYN/ACK, and no routing. The consequence for security is that GOOSE is unauthenticated and unencrypted by default — any device on the process-bus VLAN can inject a GOOSE frame. SafeGOOSE provides structural filtering but cannot verify the sender’s identity.

A GOOSE message contains 12 fields in strict order:

1. **gocbRef** – name of the GOOSE control block (identifies the sender)
2. **timeAllowedToLive** – timeout in milliseconds
3. **datSet** – dataset name
4. **goID** – message ID (**optional** – the only optional GOOSE field)
5. **t** – UTC timestamp (8 bytes)
6. **stNum** – state number: incremented by the publisher each time the dataset values change (e.g., a breaker trips). Subscribers use **stNum** to distinguish a new event from a retransmission of the same event: if **stNum** is the same as the last received frame, the frame is a retransmission.
7. **sqNum** – sequence number within one event: resets to 0 whenever **stNum** increments, then counts up with each retransmission of that event (0 = first transmission, 1 = first retry, etc.). A gap in consecutive **sqNum** values means one or more retransmissions were lost.
8. **simulation** – boolean: real or test message?
9. **confRev** – configuration revision
10. **ndsCom** – needs commissioning?
11. **numDataSetEntries** – declared count of data values; must equal the actual number of entries in **allData** per IEC 61850-8-1 §9.2.1 (enforced; mismatch → GOOSE\_REJECT\_DATA\_VALUE\_COUNT)
12. **allData** – the actual values (max 64 entries)

The filter enforces this strict ordering using a **state machine**: the first field must be **gocbRef** (tag 0x80), the second must be **timeAllowedToLive** (tag 0x81), and so on. Any out-of-order or missing field causes immediate rejection.

### 12.2 Worked Example: GOOSE Frame

A minimal GOOSE frame (Ethernet header + APDU with 2 data values):

01 02 ... 88 B8 ... 61 2F 80 04 74 65 73 74 81 02 03 E8 ...

1. **Bytes 0–11:** Destination MAC (6B) + Source MAC (6B).
2. **Bytes 12–13:** EtherType = 0x88B8 → GOOSE. Accepted.
3. **Bytes 14–21:** GOOSE header (AppID, Length, Reserved fields).
4. **Byte 22:** Outer tag = 0x61 (APPLICATION 1) → expected. Accepted.
5. **Byte 23:** APDU length.
6. **State machine walk:**
  - State 1: expect tag 0x80 (gocbRef). Got 0x80 → transition to state 2. Content: “test” (4 bytes). Size 4  $\in$  [1, 129] → accepted.
  - State 2: expect tag 0x81 (timeAllowedToLive). Got 0x81 → state 3. Content: 0x03E8 = 1000ms. Size 2  $\in$  [1, 5] → accepted.
  - ... (states 3–12 check remaining fields)
  - State 13: ACCEPT. Count data values in **allData**  $\leq$  64 → OK.
7. **Result:** GOOSE\_OK, data\_value\_count = 2.

### 12.3 Implementation

The GOOSE filter validates IEC 61850 GOOSE (Generic Object Oriented Substation Event) frames, which are Ethernet-layer multicast messages used for fast protection signaling in substations. The filter uses a state machine to enforce the strict field ordering of the GOOSE PDU.

## 12.4 GOOSE Ethernet Frame Wire Format

GOOSE is a Layer 2 protocol transmitted as raw Ethernet multicast (EtherType 0x88B8), with no IP or TCP involvement.

**GOOSE Ethernet frame layout:**

| Ethernet header           |         | GOOSE header        |         |         |         |         |               | APDU                            |
|---------------------------|---------|---------------------|---------|---------|---------|---------|---------------|---------------------------------|
| B 0–5                     | B 6–11  | B 12–13             | B 14–15 | B 16–17 | B 18–19 | B 20–21 | B 22          | B 23+                           |
| Dst MAC<br>01:0C:CD:01:.. | Src MAC | EtherType<br>0x88B8 | AppID   | Length  | Res1    | Res2    | Outer<br>0x61 | 12 fields<br>state machine walk |

**APDU fields** (state machine enforces this exact order):

| Tag  | Field             | Description                                                                                   |
|------|-------------------|-----------------------------------------------------------------------------------------------|
| 0x80 | gocbRef           | GOOSE control block reference (sender identity)                                               |
| 0x81 | timeAllowedToLive | Timeout in ms before receiver treats data as stale                                            |
| 0x82 | datSet            | Dataset name                                                                                  |
| 0x83 | goID              | Message ID ( <b>optional</b> — the only optional GOOSE field)                                 |
| 0x84 | t                 | UTC timestamp (8 bytes)                                                                       |
| 0x85 | stNum             | State number, incremented on each value change                                                |
| 0x86 | sqNum             | Sequence number for loss detection                                                            |
| 0x87 | simulation        | BER BOOLEAN: real (0x00) or test (0x01+). <b>Test frames always rejected</b> (unconditional). |
| 0x88 | confRev           | Configuration revision                                                                        |
| 0x89 | ndsCom            | Needs commissioning flag                                                                      |
| 0x8A | numDatSetEntries  | Count of data values that follow                                                              |
| 0xAB | allData           | The actual values (max 64 entries)                                                            |

## 12.5 GOOSE\_NEXT\_STATE – State Transition Table

Listing 40: GOOSE\_NEXT – state machine transition table

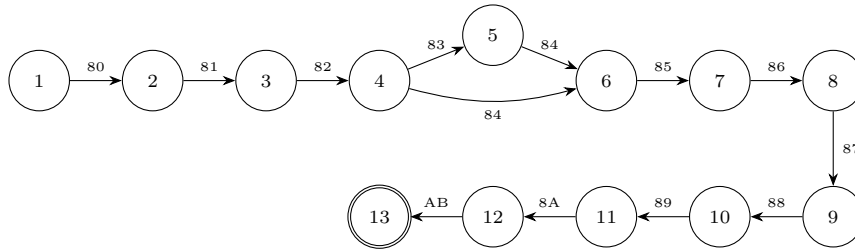
```

1 #define GOOSE_STATE_REJECT 0U
2 #define GOOSE_STATE_ACCEPT 13U
3 #define GOOSE_STATE_COUNT 14U
4
5 static const u8 GOOSE_NEXT_STATE[GOOSE_STATE_COUNT][256] = {
6     [ 1] = { [0x80] = 2 },
7     [ 2] = { [0x81] = 3 },
8     [ 3] = { [0x82] = 4 },
9     [ 4] = { [0x83] = 5, [0x84] = 6 }, /* goID optional */
10    [ 5] = { [0x84] = 6 },
11    [ 6] = { [0x85] = 7 },
12    [ 7] = { [0x86] = 8 },
13    [ 8] = { [0x87] = 9 },
14    [ 9] = { [0x88] = 10 },
15    [10] = { [0x89] = 11 },
16    [11] = { [0x8A] = 12 },
17    [12] = { [0xAB] = GOOSE_STATE_ACCEPT },
18 };

```

The GOOSE PDU body contains 12 context-tagged fields that must appear in a specific order. The state machine starts in state 1. At each field, `GOOSE_NEXT[state][tag]` gives the next state. If the result is 0 (**G-REJECT**), the tag is unexpected for this position. If the machine reaches state 13 (**G-ACCEPT**), all required fields are present and in order.

State 4 has two outgoing transitions: tag 0x83 leads to state 5, and tag 0x84 leads to state 6. This makes field 3 (**goID**, tag 0x83) **optional**: the machine can skip directly from state 4 to state 6 when it sees tag 0x84 (**t**, the UTC timestamp).



The GOOSE fields in order:

Table 18: GOOSE state machine transitions and field names

| Transition | Tag  | Field             | IEC 61850 Name    |
|------------|------|-------------------|-------------------|
| 1 → 2      | 0x80 | gocbRef           | VISIBLE-STRING    |
| 2 → 3      | 0x81 | timeAllowedToLive | INTEGER           |
| 3 → 4      | 0x82 | datSet            | VISIBLE-STRING    |
| 4 → 5      | 0x83 | goID (optional)   | VISIBLE-STRING    |
| 4/5 → 6    | 0x84 | t (timestamp)     | UtcTime (8 bytes) |
| 6 → 7      | 0x85 | stNum             | INTEGER           |
| 7 → 8      | 0x86 | sqNum             | INTEGER           |
| 8 → 9      | 0x87 | simulation        | BOOLEAN           |
| 9 → 10     | 0x88 | confRev           | INTEGER           |
| 10 → 11    | 0x89 | ndsCom            | BOOLEAN           |
| 11 → 12    | 0x8A | numDatSetEntries  | INTEGER           |
| 12 → 13    | 0xAB | allData           | SEQUENCE          |

## 12.6 GOOSE\_FIELD\_BOUNDS – Field Length Bounds

Listing 41: GOOSE field length bounds (struct array indexed by tag)

```

1 typedef struct {
2     u16 min;
3     u16 max;
4 } GooseFieldBounds;
5
6 static const GooseFieldBounds GOOSE_FIELD_BOUNDS[256] = {
7     /*      min      max      */
8     [0x80] = { 1, 129 }, /* gocbRef */
9     [0x81] = { 1, 5 }, /* timeAllowedToLive */
10    [0x82] = { 1, 129 }, /* datSet */
11    [0x83] = { 0, 129 }, /* goID (optional) */
12    [0x84] = { 8, 8 }, /* t (UTC time) */
13    [0x85] = { 1, 5 }, /* stNum */
14    [0x86] = { 1, 5 }, /* sqNum */
15    [0x87] = { 1, 1 }, /* simulation */
16    [0x88] = { 1, 5 }, /* confRev */
17    [0x89] = { 1, 1 }, /* ndsCom */
18    [0x8A] = { 1, 5 }, /* numDatSetEntries */
19    [0xAB] = { 0, 1400 }, /* allData */
20 };

```

For each GOOSE field tag, this 256-entry struct array gives the minimum and maximum allowed content length. The array is indexed directly by the tag byte (no search or computation needed). Notable constraints:

- Timestamp (0x84): exactly 8 bytes (min = max = 8).
- Boolean fields (0x87 simulation, 0x89 ndsCom): exactly 1 byte. For **simulation** (0x87): if the byte value is non-zero (**TRUE**), the frame is in test/simulation mode and is *always* rejected with **GOOSE\_REJECT\_TEST** (unconditional; no override compile flag exists). Test frames are not required in normal operation; protection relays must ignore them per IEC 61850-8-1.
- Integer fields (0x81, 0x85, 0x86, 0x88, 0x8A): 1–5 bytes (up to 40-bit integer).
- String fields (0x80 gocbRef, 0x82 datSet, 0x83 goID): up to 129 bytes.
- The allData field (0xAB): up to 1400 bytes. Its inner contents are validated separately by `goose_validate_alldata`.
- `GOOSE_FIELD_BOUNDS[0x83].min = 0` because goID is optional (length 0 is accepted if present).

## 12.7 GOOSE\_DATA\_VALUE\_WHITELIST – Data Value Tag Whitelist Array

Listing 42: GOOSE\_DATA\_VALUE\_WHITELIST – 168-entry whitelist array

```

1 static const u8 GOOSE_DATA_VALUE_WHITELIST[168] = {
2     [0x83] = 1, /* boolean */
3     [0x84] = 1, /* bit-string */
4     [0x85] = 1, /* integer */
5     [0x86] = 1, /* unsigned */
6     [0x87] = 1, /* floating-point */

```

```

7      [0x89] = 1, /* octet-string */
8      [0x8A] = 1, /* visible-string */
9      #ifdef SAFE_ENABLE_GOOSE_MMS_STRING
10     [0x90] = 1, /* mms-string */
11     #endif
12     [0x91] = 1, /* utc-time */
13     [0xA1] = 1, /* array */
14     [0xA2] = 1, /* structure */
15 };

```

This array determines which tags are allowed inside the `allData` SEQUENCE. The accepted tags correspond to IEC 61850 MMS data types encoded in GOOSE. The `mms-string` tag (0x90, `GOOSE_DATA_MMS_STRING_TAG`) is accepted only when `SAFE_ENABLE_GOOSE_MMS_STRING` is defined (off by default), so ten tags are accepted in the default build:

Table 19: GOOSE `allData` accepted MMS data type tags

| Tag  | MMS Data Type                                                   |
|------|-----------------------------------------------------------------|
| 0x83 | boolean                                                         |
| 0x84 | bit-string                                                      |
| 0x85 | integer                                                         |
| 0x86 | unsigned                                                        |
| 0x87 | floating-point                                                  |
| 0x89 | octet-string                                                    |
| 0x8A | visible-string                                                  |
| 0x90 | mms-string (only if <code>SAFE_ENABLE_GOOSE_MMS_STRING</code> ) |
| 0x91 | utc-time                                                        |
| 0xA1 | array                                                           |
| 0xA2 | structure                                                       |

## 12.8 Disabled GOOSE data types: rationale

The accepted tags in `GOOSE_DATA_VALUE_WHITELIST` (ten by default, eleven with `SAFE_ENABLE_GOOSE_MMS_STRING`) cover every IEC 61850 data type that appears in GOOSE `allData`. The table below explains why the remaining ASN.1 tag space is blocked.

Table 20: GOOSE `allData` blocked tag ranges and rationale

| Tag range  | ASN.1 type                           | class       | / | Reason blocked                                                                                                                                                                                                                                                                                     |
|------------|--------------------------------------|-------------|---|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 0x00–0x82  | Universal tags, unrecognised context |             |   | No IEC 61850 GOOSE data value uses universal-class ASN.1 tags. A frame containing a UNIVERSAL INTEGER (0x02) or UNIVERSAL SEQUENCE (0x30) in the <code>allData</code> field is either a protocol mismatch or an injection attempt targeting an ASN.1 decoder that processes universal types.       |
| 0x88 (gap) | Unassigned tag                       | context     |   | Tag 0x88 falls between floating-point (0x87) and octet-string (0x89) but is unassigned in IEC 61850-8-1. Accepting it would require the filter to know its element size, which is undefined, opening a size-computation gap.                                                                       |
| 0x8B–0x8F  | Unassigned tags                      | context     |   | IEC 61850-8-1 Annex A assigns GOOSE data value tags only up to 0x91 (utc-time). Tags in the 0x8B–0x8F range are not assigned and have no defined element size in the standard.                                                                                                                     |
| 0x92–0xA0  | Unassigned tags                      | or extended |   | Beyond 0x91 (utc-time) and before the array (0xA1) and structure (0xA2) constructed tags, no IEC 61850 data type is defined. Accepting these tags would require knowing their length from a future or proprietary extension, which the filter cannot verify.                                       |
| 0xA3–0xFF  | Higher tags                          | constructed |   | Constructed context tags above 0xA2 are not defined in IEC 61850-8-1. A frame embedding a deeply nested constructed type here would force the <code>allData</code> validator to recurse into attacker-controlled structure, potentially exhausting the recursion depth limit in an unexpected way. |

The two constructed tags that *are* accepted (0xA1 array, 0xA2 structure) allow nested datasets, which IEC 61850 does use (e.g., a GOOSE dataset containing a structure of boolean and floating-point values). Nesting depth is bounded by `SAFE_GOOSE_MAX_DATA_DEPTH` (4) to prevent stack exhaustion.

## 12.9 goose\_validate\_alldata – Data Value Validator

### 12.9.1 ACSL Contract

Listing 43: goose\_validate\_alldata ACSL contract

```
1  /*@
2   requires length == 0 ||
3           \valid_read(content + (0 .. length - 1));
4   requires \valid(reject_offset);
5   requires \valid(data_value_count_output);
6   requires \separated(reject_offset, data_value_count_output);
7   requires \separated(reject_offset,
8                       content + (0 .. length - 1));
9   requires \separated(data_value_count_output,
10                  content + (0 .. length - 1));
11  requires length <= SAFE_GOOSE_MAX_APDU;
12  requires depth <= SAFE_GOOSE_MAX_DATA_DEPTH;
13  assigns *reject_offset, *data_value_count_output;
14  ensures *data_value_count_output <= SAFE_GOOSE_MAX_DATA_VALUES;
15  */
16  static u32 goose_validate_alldata(
17      const u8 *const content,
18      const u32 length,
19      u32 *const reject_offset,
20      u32 *const data_value_count_output,
21      const u32 depth)
```

Let us read each clause:

- **requires** `\valid(reject_offset)` and **requires** `\valid(data_value_count_output)`: both output pointers are writable.
- Three **\separated** clauses ensure that the two output pointers do not alias each other and do not alias any byte of the content buffer. This is essential for the prover to reason about writes to `*reject_offset` not corrupting `*data_value_count_output` or the content.
- **requires** `length <= GOOSE_MAX_APDU` – the allData content cannot exceed 1400 bytes.
- **requires** `depth <= SAFE_GOOSE_MAX_DATA_DEPTH` – recursion depth for nested array (tag 0xA1) and structure (tag 0xA2) data values. The filter recurses into nested arrays and structures but the depth is bounded by `SAFE_GOOSE_MAX_DATA_DEPTH` (4), preventing stack overflow. The top-level call passes depth 0; each recursive call increments it and returns `GOOSE_REJECT_DATA_VALUE_TAG` if the limit is reached.
- **assigns** `*reject_offset, *data_value_count_output` – only these two output locations are modified.
- **ensures** `*data_value_count_output <= GOOSE_MAX_DATA_VALUES` – bounded output count. This is the key guarantee that callers rely on.

### 12.9.2 Main Loop Annotations

Listing 44: goose\_validate\_alldata loop annotations

```
1  /*@
2   loop invariant offset <= length;
3   loop invariant count <= SAFE_GOOSE_MAX_DATA_VALUES;
4   loop assigns offset, count,
5               *reject_offset, *data_value_count_output;
6   loop variant length - offset;
7  */
8  while (offset < length)
9  {
10     //@ ghost u32 offset_top = offset;
11     ...
12     //@ assert offset > offset_top; */
13     count++;
14 }
```

The loop invariant bounds both `offset` (stays within the buffer) and `count` (never exceeds `SAFE_GOOSE_MAX_DATA_VALUES`, default 64). The ghost variable `offset_top` and the assertion `offset > offset_top` prove strict forward progress, which the prover uses to verify the loop variant `length - offset`.

### 12.9.3 Algorithm

This function iterates through the `allData` content, which is a flat sequence of TLV-encoded data values:

1. Check that the count has not exceeded 64. Reject with `GOOSE_REJECT_DATA_VALUE_COUNT` if it has.
2. Read the tag byte. Check against `GOOSE_DATA_VALUE_WHITELIST`. Reject with `GOOSE_REJECT_DATA_VALUE_TAG` if not found.
3. Decode BER length inline (same short/long-form logic as `read_length`, but embedded directly to keep the ACSL annotations local). The inline long-form decoding loop has its own annotations:

Listing 45: Inline BER length loop in goose\_validate\_alldata

```
1  /*@
2   loop invariant 0 <= i <= num_length_bytes;
3   loop invariant offset <= length;
4   loop invariant offset > offset_top;
5   loop assigns i, value_len, offset;
6   loop variant num_length_bytes - i;
7  */
```



```

8         for (u32 i = 0;
9             (i < num_length_bytes) && (offset < length);
10            ++i)
11         {
12             value_len = (value_len << 8U) |
13                         (u32)content[offset];
14             offset++;
15         }

```

4. Bounds-check the decoded length against the remaining buffer.
5. Advance past the value content (`offset += value_len`). Increment count.

After the loop, if `count < 1U` (empty `allData`), reject with `GOOSE_REJECT_DATA_VALUE_COUNT`. Otherwise, store the count in `*data_value_count_output` and return 0 (success).

## 12.10 safe\_goose\_filter – Entry Point

### 12.10.1 ACSL Contract

Listing 46: safe\_goose\_filter ACSL contract

```

1  /*@
2  requires length <= 65535;
3  requires length == 0 ||
4      \valid_read(ethernet_frame + (0 .. length - 1));
5  assigns \nothing;
6
7  ensures \result.error == GOOSE_OK
8      ==> \result.data_value_count <= SAFE_GOOSE_MAX_DATA_VALUES;
9
10 behavior undersized:
11     assumes length < GOOSE_MIN_FRAME;
12     ensures \result.error == GOOSE_REJECT_SIZE;
13
14 behavior without_vlan:
15     assumes length >= GOOSE_MIN_FRAME;
16     assumes length < GOOSE_MIN_FRAME + 4 ||
17         ethernet_frame[ETH_ETHERTYPE_OFFSET] != VLAN_TPID_HI ||
18         ethernet_frame[ETH_ETHERTYPE_OFFSET + 1] != VLAN_TPID_LO;
19     ensures
20         (ethernet_frame[ETH_ETHERTYPE_OFFSET] != GOOSE_ETHERTYPE_HI ||
21          ethernet_frame[ETH_ETHERTYPE_OFFSET + 1] != GOOSE_ETHERTYPE_LO)
22         ==> \result.error == GOOSE_REJECT_ETH_TYPE;
23
24 behavior with_vlan:
25     assumes length >= GOOSE_MIN_FRAME + 4;
26     assumes ethernet_frame[ETH_ETHERTYPE_OFFSET] == VLAN_TPID_HI;
27     assumes ethernet_frame[ETH_ETHERTYPE_OFFSET + 1] == VLAN_TPID_LO;
28     ensures
29         (ethernet_frame[ETH_ETHERTYPE_OFFSET + 4] != GOOSE_ETHERTYPE_HI ||
30          ethernet_frame[ETH_ETHERTYPE_OFFSET + 5] != GOOSE_ETHERTYPE_LO)
31         ==> \result.error == GOOSE_REJECT_ETH_TYPE;
32
33 complete behaviors undersized, without_vlan, with_vlan;
34 disjoint behaviors undersized, without_vlan, with_vlan;
35 */
36 SAFE_PURE SafeGooseResult safe_goose_filter(
37     const u8 * const ethernet_frame,
38     const u32 length)

```

Four machine-checked guarantees:

- Undersized frames (`< GOOSE_MIN_FRAME`) always yield `GOOSE_REJECT_SIZE`.
- Non-VLAN frames with wrong EtherType always yield `GOOSE_REJECT_ETH_TYPE`.
- IEEE 802.1Q VLAN-tagged frames check the inner EtherType at offset +4.
- On success, `data_value_count <= SAFE_GOOSE_MAX_DATA_VALUES`.

### 12.10.2 Algorithm

1. **Minimum size.** Reject if `length < GOOSE_MIN_FRAME` (22).
2. **EtherType.** Check bytes at offset 12–13 for 0x88B8 (GOOSE).
3. **Header length.** Read the 2-byte big-endian length at offset `PROTOCOL_HEADER_LENGTH_OFFSET` (16). Verify that `ETH_HEADER_LENGTH + goose_len` does not exceed the frame.
4. **APDU size.** Compute `apdu_len = length - APDU_OFFSET`. Reject if `> GOOSE_MAX_APDU` (1400).
5. **Outer tag.** The first byte of the APDU must be 0x61 (`GOOSE_OUTER_TAG`, APPLICATION 1, constructed).
6. **Body length.** Decode the body length using `read_length`. Inline assertions help the prover establish separation and bounds:

```

1  /*@ assert offset <= apdu_len; */
2  /*@ assert \separated(&offset, apdu + (0 .. apdu_len - 1)); */
3  const u32 body_len = read_length(apdu, apdu_len, &offset);
4  /*@ assert offset <= apdu_len; */

```

7. **State machine walk.** Initialize `state = 1U`. The main loop:

Listing 47: GOOSE state machine loop annotations

```

1  /*@

```

```

2      loop invariant offset <= apdu_len;
3      loop invariant body_end <= apdu_len;
4      loop invariant 0 <= state <= GOOSE_STATE_ACCEPT;
5      loop invariant data_value_count <= SAFE_GOOSE_MAX_DATA_VALUES;
6      loop assigns    offset, state, data_value_count, out;
7      loop variant    body_end - offset;
8
9  */
  while (offset < body_end && state != GOOSE_STATE_ACCEPT) {

```

For each field in the body:

- (a) Read the tag byte.
  - (b) Look up `GOOSE_NEXT[state][tag]`. If 0, reject with `GOOSE_REJECT_FIELD_TAG`.
  - (c) Decode field length using `read_length`.
  - (d) If this is the `allData` field (`state == 12U`), call `goose_validate_alldata` on the field content. If validation fails, propagate the rejection code and offset.
  - (e) Otherwise, check field length against `GOOSE_FIELD_BOUNDS[tag].min` and `GOOSE_FIELD_BOUNDS[tag].max`. Reject with `GOOSE_REJECT_FIELD_SIZE` if out of bounds.
  - (f) Advance past the field. Update state.
8. **Accept check.** If the state machine did not reach `G_ACCEPT` (13), the message is truncated; return `GOOSE_REJECT_TRUNCATE`.
  9. Store `data_value_count` in `out.data_value_count` and return OK.

## 12.11 All 14 validation checks: what, why, and what error code

### 12.11.1 Check 1 – Frame size (`GOOSE_REJECT_SIZE`)

**What is checked.** The frame length must be  $\geq$  `GOOSE_MIN_FRAME` (22 bytes). The APDU length (after the 14-byte Ethernet header and 8-byte GOOSE header) must also not exceed `GOOSE_MAX_APDU` (1 400 bytes).

**Why.** A frame shorter than 22 bytes cannot contain both the 14-byte Ethernet header and the 8-byte GOOSE-specific header (AppID, Length, Res1, Res2). Any GOOSE APDU larger than 1 400 bytes violates the IEC 61850-8-1 Ethernet MTU limit.

**Attack prevented.** A 10-byte synthetic frame that makes the EtherType check (`buffer[12]`) an out-of-bounds access is blocked here.

### 12.11.2 Check 2 – EtherType (`GOOSE_REJECT_ETH_TYPE`)

**What is checked.** Bytes 12–13 must be exactly `0x88 0xB8`.

**Why.** Network infrastructure is expected to route only EtherType `0x88B8` frames to the GOOSE filter. The explicit check makes the filter self-contained: even if a misconfigured switch forwards an SV frame (`0x88BA`) into the GOOSE path, it is rejected immediately rather than being silently misparse as GOOSE.

**Attack prevented.** Cross-protocol injection (sending an SV frame to the GOOSE filter) is rejected.

### 12.11.3 Check 3 – Header length consistency (`GOOSE_REJECT_HDR_LEN`)

**What is checked.** The 2-byte big-endian length field at bytes 16–17 (the GOOSE header length field) must not exceed the full frame length minus the 14-byte Ethernet header.

**Why.** The GOOSE header length field is used to derive the APDU start offset. If the declared header length is larger than the remaining frame, the computed APDU pointer would lie outside the buffer.

**Attack prevented.** A frame with a header-length field of 60 000 in a 200-byte frame is rejected before any pointer arithmetic uses the invalid value.

### 12.11.4 Check 4 – Outer APPLICATION 1 tag (`GOOSE_REJECT_OUTER_TAG`)

**What is checked.** The first byte of the APDU must be `0x61` (`GOOSE_OUTER_TAG: APPLICATION` class, constructed, tag number 1).

**Why.** IEC 61850-8-1 mandates that every GOOSE APDU starts with APPLICATION 1 (`0x61`). This is the mandatory APDU wrapper tag. Its absence means the frame is either not a GOOSE PDU or is corrupted.

**Attack prevented.** A frame with a valid EtherType but an incorrect outer tag (e.g., a GOOSE-like replay with a `0x60` SV wrapper tag) is rejected here.

### 12.11.5 Check 5 – Field tag state machine (`GOOSE_REJECT_FIELD_TAG`)

**What is checked.** Each field's tag byte is validated against the `GOOSE_NEXT_STATE` transition table for the current state. Any tag that does not match the expected transition (or the allowed optional-skip) is rejected.

**Why.** IEC 61850-8-1 specifies a strict field ordering for the 12 GOOSE fields (`gocbRef`, `timeAllowedToLive`, `datSet`, ..., `allData`). Enforcing this order with a state machine means each field is verified against its expected tag at exactly its expected position in the PDU. An out-of-order field cannot be accepted.

**Attack prevented.** An attacker who injects a frame with the fields in a different order (e.g., putting `allData` before the mandatory header fields) is rejected at the first out-of-state tag. Similarly, a frame with a fabricated extra field after the standard 12 is rejected.

### 12.11.6 Check 6 – Field length bounds (`GOOSE_REJECT_FIELD_SIZE`)

**What is checked.** Each field's decoded length is checked against the `GOOSE_FIELD_BOUNDS[tag].min` and `GOOSE_FIELD_BOUNDS[tag].max` constants. Fields outside these bounds are rejected.

**Why.** Each GOOSE field has a well-defined wire size defined by IEC 61850-8-1: for example, `t` (UTC timestamp) is always 8 bytes. Accepting a `t` field of length 0 or length 100 would indicate a malformed frame. The bounds check ensures every field's content is the expected size before the content is accessed.

**Attack prevented.** A frame with a `stNum` field of length 0 (which would make the incremented state number undefined) is rejected here.

#### 12.11.7 Check 7 – Field boundary / truncation (GOOSE\_REJECT\_TRUNCATE)

**What is checked.** The state machine rejects the frame if it reaches the end of the APDU body without reaching the `G_ACCEPT` terminal state (all 12 mandatory fields consumed).

**Why.** If any field's declared length causes the offset to advance past the end of the declared body length, or if the APDU runs out of bytes before all 12 fields are seen, the frame is truncated. The truncation check is also applied per-field during the `read_length` decode of each field's length.

**Attack prevented.** A deliberately short frame with only 5 of the 12 required fields is rejected here rather than being partially accepted.

#### 12.11.8 Check 8 – Data value tag whitelist (GOOSE\_REJECT\_DATA\_VALUE\_TAG)

**What is checked.** Every tag byte in the `allData` array is looked up in `GOOSE_DATA_VALUE_WHITELIST`. Tags not in the list (entry 0) are rejected. For nested `array` (0xA1) and `structure` (0xA2) tags, the validator recurses up to `SAFE_GOOSE_MAX_DATA_DEPTH` levels deep.

**Why.** The IEC 61850 data model for GOOSE `allData` is limited to a small set of ASN.1 primitive types (BOOLEAN, INTEGER, FLOAT, OCTET STRING, etc.) plus array and structure containers. Any tag outside this set is either a protocol mismatch or an injection attempt targeting the data decoder.

**Attack prevented.** Embedding an arbitrary BER stream inside the `allData` field (e.g., a SEQUENCE OF UTCTime) is rejected at the data value whitelist check.

#### 12.11.9 Check 9 – Data value count (GOOSE\_REJECT\_DATA\_VALUE\_COUNT)

**What is checked.** The total count of top-level data values in `allData` must be in `[1, SAFE_GOOSE_MAX_DATA_VALUES]` (1–64).

**Why.** Production SCL files define at most 40 data values per GOOSE dataset; the limit of 64 provides generous headroom. Requiring at least 1 value ensures the `allData` field is non-empty. Bounding the count at 64 prevents a maliciously large `allData` field from causing a long parsing loop.

**Attack prevented.** A synthetic GOOSE frame with 1 000 Boolean data values (inflating the `allData` loop count) is rejected when the 64-value limit is reached.

#### 12.11.10 Check 10 – Bit-string padding byte (GOOSE\_REJECT\_DATA\_VALUE\_PADDING)

**What is checked.** When a data value inside `allData` has tag 0x84 (bit-string), the first byte of its content is the *unused-bits* count (how many low-order bits of the last byte are padding). The BER validity constraint is that this byte must be 0–7; a value greater than 7 is impossible (an octet has only 8 bits) and is rejected with `GOOSE_REJECT_DATA_VALUE_PADDING`.

**Why.** IEC 61850 GOOSE bit-string values (e.g., DBPOS double-point position) use whole-octet lengths, so conformant publishers always set this byte to 0. The filter enforces the BER structural constraint ( $\text{unused-bits} \leq 7$ ): any value exceeding 7 is structurally impossible and indicates a malformed or crafted frame.

**Attack prevented.** A crafted bit-string with an impossible unused-bits count ( $> 7$ ) — which could trigger a decoder bug in downstream relay firmware that misinterprets the padding count — is rejected before the application processes the data.

#### 12.11.11 Check 11 – Simulation / test frame (GOOSE\_REJECT\_TEST)

**What is checked.** The `simulation` field (tag 0x87) carries a BER BOOLEAN value. Any non-zero content byte means TRUE per X.690 §11.1. Frames with `simulation = TRUE` are always rejected immediately when the `simulation` field is reached in the state-machine walk. This rejection is unconditional; there is no compile flag to override it.

**Why.** IEC 61850-8-1 specifies that devices in normal operation must ignore GOOSE frames with `simulation = TRUE`. Test frames are injected only during secondary injection tests and commissioning; passing them to operational relay logic is a safety defect — the relay would act on non-operational data. Filtering is always active because test traffic must never reach a deployed filter. Commissioning environments should use a dedicated test network segment or disable this filter entirely rather than modifying the production filter binary.

**Attack prevented.** An attacker replaying captured test-mode GOOSE frames into an operational network is rejected at the `simulation` field before relay protection logic sees the frame.

#### 12.11.12 Check 12 – allData integer and unsigned widths (GOOSE\_REJECT\_DATA\_VALUE\_LEN)

**What is checked.** Inside `goose_validate_alldata`, after the tag is confirmed against `GOOSE_DATA_VALUE_WHITELIST`, two additional width constraints are enforced:

- **integer** (tag 0x85): content length must be 1, 2, 4, or 8 bytes.
- **unsigned** (tag 0x86): content length must be 1, 2, or 4 bytes.

Any other length returns `GOOSE_REJECT_DATA_VALUE_LEN` (12).

**Why.** IEC 61850-8-1 Table 33 specifies that INTEGER and Unsigned values map to machine-word-width types (8, 16, 32, or 64-bit integers; unsigned has no 64-bit variant). Accepting a 3-byte or 5-byte integer would pass structural

checks but produce an unrepresentable value on any real-world IED implementation. Rejecting non-machine-word widths enforces the standard’s type-width invariant and prevents crafted frames from reaching IED application code with semantically invalid data.

**Attack prevented.** A GOOSE frame carrying an `integer` data value with claimed length 100 would normally pass tag-whitelist and per-field length checks (the global length bound allows up to 5 bytes for integer fields in `GOOSE_FIELD_BOUNDS`); this check intercepts it before the value reaches relay logic.

#### 12.11.13 Check 13 – Unsigned32 header field value range (`GOOSE_REJECT_FIELD_VALUE`)

**What is checked.** The GOOSE header fields encoded as IEC 61850-7-2 INT32U – `timeAllowedToLive` (0x81), `stNum` (0x85), `sqNum` (0x86), `confRev` (0x88), and `numDatSetEntries` (0x8A) – must encode a value in  $[0, 2^{32} - 1]$ . The size bounds accept byte-lengths 1–5; this check tightens the value: a 5-byte BER integer must lead with 0x00 (non-negative and capped at 0xFFFFFFFF), and any shorter encoding with the BER sign bit set is rejected. Failure returns `GOOSE_REJECT_FIELD_VALUE` (13). Minimal-encoding is not enforced (zero-padding is tolerated for IED interop).

**Why.** These fields are unsigned 32-bit on every conformant IED. A 5-byte BER integer can encode a value larger than 0xFFFFFFFF or, with the high bit set, a negative number; either would be silently truncated or sign-extended by downstream code, desynchronising the subscriber’s `stNum`/`sqNum` tracking.

#### 12.11.14 Check 14 – allData unsigned value range (`GOOSE_REJECT_DATA_VALUE_RANGE`)

**What is checked.** Inside `goose_validate_alldata`, an unsigned data value (tag 0x86) that passed the width check (1–5 bytes, Check 12) is additionally constrained to the INT32U subset  $[0, 2^{32} - 1]$ : a 5-byte encoding must lead with 0x00, and any shorter encoding with the BER sign bit set is rejected. Failure returns `GOOSE_REJECT_DATA_VALUE_RANGE` (14).

**Why.** Unsigned in IEC 61850 has no 64-bit variant; the value must fit in 32 unsigned bits. This is the `allData` analogue of Check 13: it prevents a structurally valid 5-byte integer from carrying a magnitude or sign that no conformant publisher would emit.

## 13 SafeSV – Sampled Values Filter

### 13.1 What are Sampled Values, and why does security matter?

To protect an electrical power grid, substation equipment must react to faults (short circuits, overloads, broken cables) in milliseconds. The first step is measurement. Instrument transformers on high-voltage busbars convert dangerous kV/kA signals into safe millivolt/milliamp signals that electronics can read. A device called a **merging unit** (MU) digitises these signals at up to 4,000 samples per second (80 samples per 50 Hz power cycle for European grids) and broadcasts the results over Ethernet so that protection relays and other IEDs can use them for fast calculations.

That broadcast is the **Sampled Values (SV)** stream, standardised by IEC 61850-9-2. Each Ethernet frame carries one or more **ASDUs** (Application Service Data Units), each representing one instant in time: typically the digitised current and voltage on all three phases of the busbar. Protection relays consume these ASDUs continuously and feed them into algorithms that decide whether to trip a circuit breaker.

**The security consequence.** If a malformed SV frame reaches a relay’s parser, two bad outcomes are possible.

**Crash or memory corruption.** A parser bug (buffer overread, integer overflow) triggered by crafted field lengths or counts can crash the relay firmware, causing a denial-of-service: the relay stops protecting the grid.

**Silent mis-operation.** A frame with plausible but incorrect measurement values (crafted `data` field) is structurally valid and will pass through SafeSV unchanged. SafeSV eliminates the *structural* attack surface: malformed encodings, out-of-range enumerations, inconsistent metadata, and frames that do not come from a conformant SV publisher (wrong APPID range).

SafeSV is a whitelist filter, not a cryptographic authenticator. It cannot verify the origin of a frame. What it does is ensure that every frame reaching the relay parser is structurally sound, within defined bounds, and free of the encoding anomalies that have historically been used to exploit parser bugs.

### 13.2 SV on the wire: the complete frame layout

SV is a **Layer 2** (Ethernet) protocol with EtherType 0x88BA. There is no IP or TCP/UDP header. Here is the complete layout, byte by byte.

**SV Ethernet frame layout:**

| Ethernet header            |         | SV header           |                        |         |         |         | savPDU           |
|----------------------------|---------|---------------------|------------------------|---------|---------|---------|------------------|
| B 0–5                      | B 6–11  | B 12–13             | B 14–15                | B 16–17 | B 18–19 | B 20–21 | B 22+            |
| Dst MAC<br>01:0C:CD:04:... | Src MAC | EtherType<br>0x88BA | APPID<br>0x4000–0x7FFF | Length  | Res1    | Res2    | tag 0x60 + ASDUs |

**ASDU structure** (tag 0x30; up to 8 ASDUs per frame):

| Tag   | 0x80 | 0x81*  | 0x82   | 0x83    | 0x84*  | 0x85*    | 0x86*   | 0x87 |
|-------|------|--------|--------|---------|--------|----------|---------|------|
| Field | svID | datSet | smpCnt | confRev | refrTm | smpSynch | smpRate | data |

\* = optional field. Two further optional fields may follow **data**: *smpMod* (tag 0x88) and the vendor extension *gmidData* (tag 0x89). Fields must appear in this order; the filter rejects any frame where they appear out of sequence.

#### Field-by-field explanation:

**Bytes 0–5: Dst MAC** The multicast address of the SV subscriber group. Typically 01:0C:CD:04:xx:xx for SV (GOOSE uses 01:0C:CD:01:xx:xx). SafeSV does not validate the MAC (enforced by Ethernet switch multicast filters).

**Bytes 6–11: Src MAC** The merging unit’s MAC address. Not validated by SafeSV.

**Bytes 12–13: EtherType = 0x88BA** Identifies the frame type. This is the first validation point. If these bytes are not 0x88 0xBA, the frame is not an SV frame and is rejected immediately (SV\_REJECT\_ETH\_TYPE). The value 0x88B8 identifies GOOSE; 0x0800 identifies IPv4.

**Bytes 14–15: APPID** A 16-bit Application Process IDentifier assigned by the substation engineer to each SV publisher. IEC 61850-9-2 Annex A reserves 0x4000–0x7FFF for SV and 0x0000–0x3FFF for GOOSE. SafeSV rejects any APPID outside the SV range (SV\_REJECT\_APPID).

**Bytes 16–17: Length** A big-endian 16-bit field counting bytes from byte 14 to the end of the frame (APPID + Length + Res1 + Res2 + APDU). SafeSV checks consistency with the actual Ethernet frame length. (SV\_REJECT\_HDR\_LEN on mismatch.)

**Bytes 18–21: Reserved1 and Reserved2** Always zero in conformant equipment. SafeSV does not validate these.

**Byte 22: savPDU outer tag = 0x60** The BER tag for the savPDU APPLICATION SEQUENCE. Must be exactly 0x60. Anything else is rejected (SV\_REJECT\_OUTER.TAG).

**savPDU length (variable)** A BER-encoded length following the 0x60 tag. SafeSV decodes this and checks that the declared PDU length fits within the frame.

**Tag 0x80: noASDU** A 1-byte field declaring how many ASDUs follow. SafeSV reads this value and later cross-checks it against the actual ASDU count found inside seqASDU (SV\_REJECT\_NOASDU on mismatch).

**Tag 0xA2: seqASDU** A BER SEQUENCE containing the actual ASDUs. Each inner entry is tagged 0x30. SafeSV iterates over these entries, calling `sv_validate_asdu` on each one.

**Tag 0x30: ASDU** One sampling instant. Contains the 10 ASDU fields described in Section 13.4.

### 13.3 All 13 validation checks: what, why, and what error code

The SV filter applies checks in a specific order, from cheapest (byte count) to most expensive (semantic content). The first failing check determines the error code; subsequent checks are skipped.

#### 13.3.1 Check 1 – Frame size (SV\_REJECT\_SIZE)

Listing 48: Size guard in `safe_sv_filter`

```
1 if (length > SAFE_SV_MAX_FRAME || length < SAFE_SV_MIN_FRAME) {
2     out.error = SV_REJECT_SIZE;
3     return out;
4 }
```

**What is checked.** The frame length must be in [22, 1,600] bytes.

**Why.** The minimum of 22 ensures the mandatory Ethernet header (14 bytes) plus the SV-specific header (8 bytes) are both present before any field is read. Without this check, accessing `buffer[12]` on a 10-byte frame would be an out-of-bounds read. The maximum of 1,600 is the standard SV frame ceiling derived from the Ethernet MTU; a frame larger than this is a jumbo frame not used by IEC 61850-9-2 equipment.

**Attack prevented.** An attacker who sends a 1-byte frame hoping to crash the parser when it reads bytes 12–13 (EtherType) is blocked here, before any byte past the first is accessed.

### 13.3.2 Check 2 – EtherType (SV\_REJECT\_ETH\_TYPE)

Listing 49: EtherType check

```
1 if (buffer[ETH_ETHERTYPE_OFFSET] != 0x88U ||
2     buffer[ETH_ETHERTYPE_OFFSET + 1] != 0xBAU) {
3     out.error = SV_REJECT_ETH_TYPE;
4     return out;
5 }
```

**What is checked.** Bytes 12–13 must be exactly 0x88 0xBA.

**Why.** In practice, a network filter rule routes only EtherType 0x88BA frames to the SV parser. But filters must be self-contained and not rely on infrastructure correctness. If a misconfigured switch passes a GOOSE frame (0x88B8) into the SV processing path, the EtherType check stops it immediately. Without this guard, the GOOSE payload would be parsed as if it were an SV frame, reading GOOSE fields as SV fields with unpredictable results.

**What ETH\_ETHERTYPE\_OFFSET is.** This is the constant 12, defined in `safe.h`. It is the byte offset of the 2-byte EtherType field in an Ethernet II frame.

### 13.3.3 Check 3 – APPID range (SV\_REJECT\_APPID)

Listing 50: APPID range check

```
1 const u32 appid =
2     ((u32)buffer[ETH_HEADER_LENGTH] << 8) |
3     (u32)buffer[ETH_HEADER_LENGTH + 1];
4 if (appid < 0x4000U || appid > 0xFFFFU) {
5     out.error = SV_REJECT_APPID;
6     out.reject_at = ETH_HEADER_LENGTH;
7     return out;
8 }
```

**What is checked.** Bytes 14–15 (the APPID, immediately after the Ethernet header) must lie in the SV-allocated range 0x4000–0xFFFF.

**The IEC 61850 APPID space partition.** IEC 61850-9-2 Annex A divides the 16-bit APPID space as follows:

| Range         | Assignment     |
|---------------|----------------|
| 0x0000–0x3FFF | GOOSE          |
| 0x4000–0x7FFF | Sampled Values |
| 0x8000–0xBFFF | Reserved       |
| 0xC000–0xFFFF | Reserved       |

**Why the check matters.** An SV frame with an APPID outside the SV band is not conformant. Several concrete scenarios produce out-of-range APPIDs:

- **Misconfigured publisher.** A merging unit misconfigured with a GOOSE APPID broadcasts SV frames with an APPID in 0x0000–0x3FFF. The receiving relay would look up the stream identifier in its SV subscription table and find no match, but the filter catches this earlier.
- **Cross-protocol injection.** An attacker replays a captured GOOSE frame with the EtherType changed to 0x88BA. The APPID is in the GOOSE range. The APPID check rejects it.
- **Crafted frame.** A frame with APPID 0x0000 (default “all-zeros” value on improperly initialised hardware) is rejected because 0x0000 is in the GOOSE range.

**Why APPID alone is not authentication.** An attacker who knows a valid APPID value can spoof it. APPID validation closes the easy cases (wrong range, GOOSE masquerading as SV) but does not replace cryptographic authentication. IEC 62351-6 defines TLS-based authentication for SV if the deployment requires it.

### 13.3.4 Check 4 – Header length consistency (SV\_REJECT\_HDR\_LEN)

Listing 51: Header length check

```
1 const u32 sv_len =
2     ((u32)buffer[PROTOCOL_HEADER_LENGTH_OFFSET] << 8) |
3     (u32)buffer[PROTOCOL_HEADER_LENGTH_OFFSET + 1];
4 if ((ETH_HEADER_LENGTH + sv_len) > length) {
5     out.error = SV_REJECT_HDR_LEN;
6     return out;
7 }
```

**What is checked.** Bytes 16–17 form a big-endian 16-bit Length field that declares the number of bytes from byte 14 (APPID) to the end of the frame. The check verifies that 14 + `sv_len` does not exceed the actual Ethernet frame length.

**Why.** If the Length field is forged to be larger than the actual frame, a parser that trusts it and allocates memory accordingly will attempt to read past the end of the frame buffer. This is the setup for a classic buffer overread. Checking before parsing prevents it.



### 13.3.5 Check 5 – Outer savPDU tag (SV\_REJECT\_OUTER\_TAG)

**What is checked.** Byte 22 (APDU\_OFFSET) must be 0x60: the BER APPLICATION 0 CONSTRUCTED tag used for the savPDU wrapper.

**Why.** In BER encoding, the tag byte encodes three things: the class (bits 7–6), whether the content is primitive or constructed (bit 5), and the tag number (bits 4–0). For savPDU: class = APPLICATION (10), constructed = 1 (1), tag = 0 (00000), giving 0x60. Any other tag means the payload is not a savPDU. Without this check, a parser expecting savPDU structure would misinterpret the entire payload.

### 13.3.6 Check 6 – Truncation (throughout: SV\_REJECT\_TRUNCATE)

**What is checked.** At every point where a BER length field is decoded, the filter checks that `offset + declared_length <= frame_length` before advancing. This applies to the savPDU length, the seqASDU length, each ASDU length, and each ASDU field length.

**Why truncation checks are so important.** BER is a length-delimited encoding: every TLV element declares its own length. An attacker can craft a frame where a declared length is larger than the actual bytes present. Without a truncation check at each level, the parser reads beyond the frame buffer. This is the root cause of many real-world vulnerabilities in protocol parsers, including CVE-2016-2108 (OpenSSL heap corruption via ASN.1 overread).

SafeSV performs truncation checks at every BER length decode, at every TLV advance, and before every ASDU iteration. If any check fails, SV\_REJECT\_TRUNCATE is returned with the byte offset where the problem was detected, making it easy to diagnose.

### 13.3.7 Check 7 – ASDU sequence tag (SV\_REJECT\_ASDU\_TAG)

**What is checked.** Each entry inside the seqASDU (tag 0xA2) must begin with tag 0x30 (BER UNIVERSAL SEQUENCE), which is the IEC 61850-9-2 encoding for one ASDU.

**Why.** The seqASDU is a sequence of identically-typed ASDUs. Each must start with 0x30. Any other tag means the frame is structurally broken (the ASDU boundaries were miscalculated, or the frame is crafted). Without this check, the parser could attempt to read ASDU field tags from arbitrary bytes.

### 13.3.8 Check 8 – ASDU count limit (SV\_REJECT\_ASDU\_COUNT)

Listing 52: ASDU count guard in the seqASDU loop

```
1 if (out.asdu_count >= SAFE_SV_MAX_ASDU) { /* default: 8 */
2     out.error = SV_REJECT_ASDU_COUNT;
3     out.reject_at = asdu_offset;
4     return out;
5 }
```

**What is checked.** The number of ASDUs in the frame must not exceed SAFE\_SV\_MAX\_ASDU (default 8, configurable in `safe_config.h`).

**Why.** IEC 61850-9-2LE (the “Light Edition” profile deployed in most substation equipment) specifies 1 or 2 ASDUs per frame. The limit of 8 provides 4× headroom while bounding the loop iteration count. Without this bound, a malformed frame with `noASDU = 255` or a very long seqASDU length field would force the parser to run 255 ASDU parse iterations, consuming significant CPU time on a time-critical relay (SV latency budget: 250 μs). In the worst case, it could cause stack overflow in parsers that process ASDUs recursively.

### 13.3.9 Checks 9–10 – ASDU field tag and size (SV\_REJECT\_FIELD\_TAG, SV\_REJECT\_FIELD\_SIZE)

These checks are applied inside `sv_validate_asdu` for each of the 10 ASDU fields. They are described in detail in Section 13.4.

### 13.3.10 Check 11 – smpSynch and smpMod content (SV\_REJECT\_CONTENT)

Listing 53: smpSynch and smpMod content checks in `sv_validate_asdu`

```
1 /* smpSynch (0x85): INT8U; valid: 0,1,2,5-254; reserved (do not use): 3,4,255 */
2 if (SV_FIELDS[field_index].tag == SV_SMP_SYNC_TAG && field_len == 1U &&
3     (content[offset] == 3U || content[offset] == 4U || content[offset] == 255U)) {
4     *reject_offset = offset;
5     return SV_REJECT_CONTENT;
6 }
7
8 /* smpMod (0x88): INT16U big-endian, SIZE(2); valid values 0-2 */
9 if (SV_FIELDS[field_index].tag == SV_SMP_MOD_TAG && field_len == 2U &&
10     (content[offset] != 0U || content[offset + 1U] > 2U)) {
11     *reject_offset = offset;
12     return SV_REJECT_CONTENT;
13 }
```

**What is checked.** When the optional field `smpSynch` (tag 0x85) is present and exactly 1 byte (INT8U), the reserved values 3, 4, and 255 are rejected; 0, 1, 2, and 5–254 are accepted. `smpMod` (tag 0x88) is a 2-byte big-endian INT16U whose value must be 0, 1, or 2 (high byte 0, low byte ≤ 2).

**What do these values mean in the real world?**



| Value | smpSynch                               | smpMod             |
|-------|----------------------------------------|--------------------|
| 0     | Not synchronised                       | Samples per period |
| 1     | Synchronised to local clock            | Samples per second |
| 2     | Synchronised to global clock (GPS/PTP) | Seconds per sample |

*smpSynch: 3, 4, 255 reserved (rejected); 5–254 accepted.*  
*smpMod: only 0–2 valid.*

**Why validate the content and not just the length?** A length check only confirms that the field occupies the right number of bytes; it says nothing about the byte’s value. IEC 61850-9-2 defines `smpSynch` (INT8U) with reserved values 3, 4, and 255 that must not be used, and `smpMod` (INT16U) with only the values 0, 1, and 2 defined. A relay’s firmware uses `smpSynch` to select a synchronisation algorithm, typically via a switch statement or a lookup table. A reserved or out-of-range value has no matching case: the default branch executes, or behaviour is undefined. If the firmware uses the value directly as an array index without a bounds check, a crafted `smpSynch` becomes an attacker-controlled memory offset.

**Why these fields in particular?** `smpSynch` and `smpMod` are small (1 byte) and appear in every SV frame from equipment that includes them. They are routinely overlooked during protocol stack review because “it is just an enumeration.” This invisibility makes them reliable candidates for carrying an invalid value past a parser that checks tag and length but not content.

**Note on the size bounds in SV\_FIELDS.** The `SV_FIELDS` entry for `smpSynch` has `max = 1`: the field is at most 1 byte long. This is a *length* bound. The content check above is a *value* bound. Both are enforced.

### 13.3.11 Check 12 – noASDU consistency (SV\_REJECT\_NOASDU)

Listing 54: noASDU consistency: read declared count then cross-check

```

1  /* Inside the outer TLV loop: read declared count from noASDU field */
2  if (tag == SV_NOASDU_TAG && tlv_len == 1U && offset < length)
3      declared_asdu_count = (u32)buffer[offset];
4
5  /* After iterating all ASDUs: cross-check */
6  if (declared_asdu_count != 0U &&
7      declared_asdu_count != out.asdu_count) {
8      out.error = SV_REJECT_NOASDU;
9      return out;
10 }
```

**What is checked.** The `noASDU` field (tag 0x80) declares how many ASDUs the publisher says are in the frame. The filter counts ASDUs independently while iterating `seqASDU`. After the iteration, the declared count and the actual count must agree.

**Why a mismatch is suspicious.** Consider the two possible disagreements:

#### noASDU claims more ASDUs than are present.

A parser that iterates `noASDU` times reading ASDUs (rather than stopping when the `seqASDU` sequence ends) will run past the end of the sequence data, reading whatever follows as if it were an ASDU. This is a buffer-overread setup: the attacker appends crafted bytes after the last legitimate ASDU and the parser reads them as a new ASDU.

#### noASDU claims fewer ASDUs than are present.

A parser that stops after `noASDU` iterations will silently ignore the extra ASDUs at the end of the sequence. An attacker who knows the parser is count-limited can hide a crafted ASDU as the last entry, which legitimate parsers skip but vulnerable ones process.

A well-configured merging unit always sets `noASDU` equal to the number of ASDUs in `seqASDU`. A mismatch is a strong indicator that the frame was not generated by conformant equipment; it is characteristic of replayed-and-modified or hand-crafted frames.

**The zero exception.** If `noASDU` is 0 or absent (some older equipment omits the field), the consistency check is skipped (`declared_asdu_count != 0U` guards the cross-check). This prevents false positives on legacy equipment that does not include `noASDU`.

### 13.3.12 Check 13 – Trailing bytes (SV\_REJECT\_TRAILING)

Listing 55: Trailing bytes check after the outer TLV loop

```

1  if (offset != pdu_end) {
2      out.error = SV_REJECT_TRAILING;
3      out.reject_at = offset;
4      return out;
5  }
```

**What is checked.** After consuming all TLV elements inside the `savPDU`, the parser’s byte position must equal `pdu_end` exactly. Any bytes between the last consumed element and `pdu_end` are rejected.

**Why.** IEC 61850-9-2 is a strictly delimited protocol: the `savPDU` length field defines the exact size of the PDU. If bytes remain inside the PDU boundary after all recognised TLVs are processed, one of the following has occurred:

- A TLV with an unknown tag was skipped. SafeSV rejects unknown tags earlier (the outer loop only accepts 0x80 and 0xA2), so this case cannot arise in normal execution.
- The frame was crafted with extra bytes appended after the last legitimate TLV. These bytes could be a second payload intended for a parser that does not enforce strict length-based termination.

**The technique being prevented: trailing data injection.** This is sometimes called “content smuggling” or “superfluous trailing data.” In TLS and ASN.1 attacks, extra data appended after a legitimate structure is interpreted as a second message or command by a poorly written parser. The Heartbleed vulnerability (CVE-2014-0160) exploited a related pattern: the parser’s response size was derived from an attacker-controlled length field rather than the actual amount of data present. The trailing bytes check ensures the filter accounts for every byte inside the savPDU and rejects any frame that cannot be fully explained by recognised TLVs.

## 13.4 The SV field descriptor array and sequential validation

### 13.4.1 Fixed-position array versus dynamic tag dispatch

General-purpose ASN.1 parsers discover field tags dynamically and dispatch to type-specific handlers based on the tag value. This flexibility creates two weaknesses: unexpected tags can reach handlers not designed for them, and field ordering is not enforced. SafeSV uses a different approach.

SafeSV uses a completely different approach. The 10 fields of an ASDU are described in a fixed array `SV_FIELDS[10]`, indexed by *position* (0–9), not by tag value. The parser walks this array sequentially. Each entry says: “at this position I expect tag X, with content length between *min* and *max* bytes. If the field is optional and the tag does not match, skip to the next position.”

This design enforces three properties that general-purpose parsers do not guarantee:

- **Unknown tags are never accepted.** A tag not expected at the current position always causes a rejection.
- **Field order is enforced.** IEC 61850-9-2 specifies a fixed field ordering; the filter enforces it.
- **No dynamic dispatch.** There is no function pointer, no switch on a tag value, no hash table lookup.

Listing 56: The `SV_FIELDS[10]` descriptor array

```

1 static const SvField SV_FIELDS[10] = {
2     /* tag      max    min  optional */
3     { 0x80,    65,    1,    0 }, /* svID */
4     { 0x81,    65,    0,    1 }, /* dataSet (optional) */
5     { 0x82,     2,    2,    0 }, /* smpCnt */
6     { 0x83,     4,    4,    0 }, /* confRev */
7     { 0x84,     8,    1,    1 }, /* refrTm (optional, SIZE(8) when present) */
8     { 0x85,     1,    1,    1 }, /* smpSynch (optional) */
9     { 0x86,     2,    2,    1 }, /* smpRate (optional, SIZE(2) INT16U) */
10    { 0x87,  4096,    1,    0 }, /* data */
11    { 0x88,     2,    2,    1 }, /* smpMod (optional, SIZE(2) INT16U) */
12    { 0x89,     8,    8,    1 }, /* gmidData (optional vendor extension; Wireshark SV dissector GmidData ::=
13    OCTET STRING; absent from IEC 61850-9-2; observed SIZE(8)) */
14 };

```

### 13.4.2 Why these specific bounds?

#### svID: 1–65 bytes

IEC 61850-9-2 defines `svID` as a visible string of at most 64 characters plus a BER length byte. The minimum of 1 ensures the identifier is non-empty: a merging unit with no identifier cannot be matched to a subscription entry.

#### dataSet: 0–65 bytes

Dataset name, same maximum as `svID`. Minimum is 0 because the field is optional and may be absent (0-length presence).

#### smpCnt: exactly 2 bytes

A 16-bit counter incremented by the merging unit for each sample it publishes. At 4000 Sa/s (IEC 61869-9 profile), `smpCnt` counts 0–3999 per second, rolling over at 65535. Subscribers detect dropped samples by checking that `smpCnt` advances by exactly 1 (or wraps). It must be exactly 2 bytes on the wire.

#### confRev: exactly 4 bytes

Configuration revision is a 32-bit unsigned integer. Always 4 bytes.

#### refrTm: exactly 8 bytes when present

An optional reference timestamp for the first sample in this ASDU, encoded as an 8-byte UTCTime value (4-byte seconds since 1970-01-01, 3-byte fractional second in nanoseconds, 1-byte quality flags). The field is optional (merging units without IEEE 1588 PTP synchronisation often omit it), but when present it must be exactly 8 bytes (min = max = 8).

#### smpSynch: exactly 1 byte when present; reserved values rejected

When present, exactly 1 byte. Reserved values 3, 4, and 255 are rejected with `SV_REJECT_CONTENT` (see Check 11 above); 0, 1, 2, and 5–254 are accepted. Absent from equipment that does not report synchronisation status.

**smpRate: exactly 2 bytes when present**

The sampling rate in samples per second, when present, as a 16-bit unsigned integer (e.g., 4000 for the IEC 61869-9 profile at 50 Hz). Absent when not reported by the merging unit; receivers then use the profile default.

**data: 1–4096 bytes**

The actual measurement bytes. The maximum of 4,096 accommodates the largest standard configuration: 80 samples/ASDU  $\times$  8 channels  $\times$  6 bytes/sample = 3,840 bytes, with margin. The minimum of 1 ensures the data field is non-empty.

**smpMod: exactly 2 bytes when present; value 0–2**

When present, a 2-byte big-endian INT16U whose value must be 0, 1, or 2 (high byte must be 0 and low byte  $\leq$  2); see Check 11 above.

**gmidData: exactly 8 bytes when present**

An optional vendor extension (tag 0x89) defined in the Wireshark SV dissector grammar as `GmidData ::= OCTET STRING` but absent from IEC 61850-9-2, observed at SIZE(8). Accepted as the tenth field when present; otherwise skipped.

**13.4.3 How the sequential walk handles optional fields**

Consider an ASDU containing only the four mandatory fields (`svID`, `smpCnt`, `confRev`, `data`), with all six optional fields absent. The parser encounters:

1. Position 0 (`svID`, tag 0x80): tag matches; consume and advance `field_index` to 1.
2. Position 1 (`datSet`, optional, tag 0x81): current byte is 0x82. Tag does not match, but the field is optional; skip (`field_index` = 2).
3. Position 2 (`smpCnt`, mandatory, tag 0x82): tag matches; consume.
4. Position 3 (`confRev`, mandatory, tag 0x83): tag matches; consume.
5. Positions 4–6 (optional): current tag is 0x87; each optional position is skipped in turn.
6. Position 7 (`data`, mandatory, tag 0x87): tag matches; consume.
7. Positions 8–9 (`smpMod`, `gmidData`, optional): content exhausted; loop exits.
8. Post-loop check: positions 8–9 are optional; all remaining positions are optional. Return success.

If a mandatory field’s tag does not match at any step, `SV_REJECT_FIELD_TAG` is returned with the byte offset of the unexpected byte.

**13.4.4 ACSL annotations for `sv_validate_asdu`**

To prove the loop terminates, Frama-C/WP requires a *loop variant*: an expression that strictly decreases with each iteration and is bounded below by zero. The loop condition is `offset < length && field_index < 10`. On every iteration, `field_index` increases by exactly 1 (whether the current field is consumed or skipped as optional), so `10 - field_index` is a valid variant: it decreases by 1 each time and reaches 0 when `field_index` reaches 10, terminating the loop after at most 10 iterations.

Listing 57: `sv_validate_asdu` main loop ACSL annotations

```

1  /*@
2   loop invariant  offset <= length;
3   loop invariant  field_index <= 9;
4   loop assigns    offset, field_index, *reject_offset;
5   loop variant    9 - field_index;
6  */
7  while ((offset < length) && (field_index < 9U)) {

```

**13.5 Complete error code reference**

Table 21: SV rejection codes

| Code | Constant                          | Meaning and trigger location                                          |
|------|-----------------------------------|-----------------------------------------------------------------------|
| 0    | <code>SV_OK</code>                | Frame accepted; <code>asdu_count</code> is valid.                     |
| 1    | <code>SV_REJECT_SIZE</code>       | <code>length</code> < 22 or > 1,600 bytes.                            |
| 2    | <code>SV_REJECT_ETH_TYPE</code>   | Bytes 12–13 $\neq$ 0x88 0xBA.                                         |
| 3    | <code>SV_REJECT_HDR_LEN</code>    | Length field (bytes 16–17) > frame size.                              |
| 4    | <code>SV_REJECT_OUTER_TAG</code>  | Byte 22 $\neq$ 0x60 (savPDU tag).                                     |
| 5    | <code>SV_REJECT_TRUNCATE</code>   | Declared TLV length exceeds available bytes.                          |
| 6    | <code>SV_REJECT_FIELD_TAG</code>  | ASDU field tag unexpected at its position, or mandatory field absent. |
| 7    | <code>SV_REJECT_FIELD_SIZE</code> | ASDU field length outside [min, max] for that field.                  |
| 8    | <code>SV_REJECT_ASDU_TAG</code>   | An entry in <code>seqASDU</code> does not begin with 0x30.            |
| 9    | <code>SV_REJECT_ASDU_COUNT</code> | More than <code>SAFE_SV_MAX_ASDU</code> (default 8) ASDUs.            |

Table 21: SV rejection codes (continued)

| Code | Constant           | Meaning and trigger location                        |
|------|--------------------|-----------------------------------------------------|
| 10   | SV_REJECT_APPID    | APPID (bytes 14–15) outside 0x4000–0x7FFF.          |
| 11   | SV_REJECT_CONTENT  | smpSynch or smpMod value > 2.                       |
| 12   | SV_REJECT_NOASDU   | Declared noASDU count $\neq$ actual ASDU count.     |
| 13   | SV_REJECT_TRAILING | Bytes remain inside savPDU after all TLVs consumed. |

### 13.6 ACSL contract for safe\_sv\_filter

Listing 58: safe\_sv\_filter ACSL contract

```

1  /*@
2  requires length <= 65535;
3  requires length == 0 ||
4  \valid_read(ethernet_frame + (0 .. length - 1));
5  assigns \nothing;
6
7  ensures \result.error == SV_OK
8  ==> \result.asdu_count <= SAFE_SV_MAX_ASDU;
9
10 behavior bad_size:
11   assumes length < SAFE_SV_MIN_FRAME || length > SAFE_SV_MAX_FRAME;
12   ensures \result.error == SV_REJECT_SIZE;
13
14 behavior without_vlan:
15   assumes length >= SAFE_SV_MIN_FRAME;
16   assumes length <= SAFE_SV_MAX_FRAME;
17   assumes length < SAFE_SV_MIN_FRAME + 4 ||
18   ethernet_frame[ETH_ETHERTYPE_OFFSET] != VLAN_TPID_HI ||
19   ethernet_frame[ETH_ETHERTYPE_OFFSET + 1] != VLAN_TPID_LO;
20   ensures
21   (ethernet_frame[ETH_ETHERTYPE_OFFSET] != SV_ETHERTYPE_HI ||
22   ethernet_frame[ETH_ETHERTYPE_OFFSET + 1] != SV_ETHERTYPE_LO)
23   ==> \result.error == SV_REJECT_ETH_TYPE;
24
25 behavior with_vlan:
26   assumes length >= SAFE_SV_MIN_FRAME + 4;
27   assumes length <= SAFE_SV_MAX_FRAME;
28   assumes ethernet_frame[ETH_ETHERTYPE_OFFSET] == VLAN_TPID_HI;
29   assumes ethernet_frame[ETH_ETHERTYPE_OFFSET + 1] == VLAN_TPID_LO;
30   ensures
31   (ethernet_frame[ETH_ETHERTYPE_OFFSET + 4] != SV_ETHERTYPE_HI ||
32   ethernet_frame[ETH_ETHERTYPE_OFFSET + 5] != SV_ETHERTYPE_LO)
33   ==> \result.error == SV_REJECT_ETH_TYPE;
34
35 complete behaviors bad_size, without_vlan, with_vlan;
36 disjoint behaviors bad_size, without_vlan, with_vlan;
37 */
38 SAFE_PURE SafeSvResult safe_sv_filter(
39   const u8 * const ethernet_frame,
40   const u32 length)

```

The contract makes four machine-checked guarantees, proved by Frama-C/WP:

- **Size rejection is total.** Any frame outside [SAFE\_SV\_MIN\_FRAME, SAFE\_SV\_MAX\_FRAME] returns SV\_REJECT\_SIZE without accessing any field.
- **EtherType rejection is total (non-VLAN).** A valid-sized frame without an IEEE 802.1Q VLAN tag and the wrong SV EtherType always returns SV\_REJECT\_ETH\_TYPE.
- **EtherType rejection is total (VLAN).** A valid-sized frame carrying an IEEE 802.1Q VLAN tag checks the inner EtherType at offset +4; wrong inner EtherType returns SV\_REJECT\_ETH\_TYPE.
- **ASDU count is bounded on acceptance.** If the filter returns SV\_OK, then asdu\_count  $\leq$  SAFE\_SV\_MAX\_ASDU (default 8). This prevents the returned count from being used as an unchecked array index by the caller.

The five semantic checks (APPID range, smpSynch/smpMod content, noASDU consistency, trailing bytes) are validated by 26 targeted regression tests in test\_sv\_restrictions.c and CBMC bounded model checking over the full 96-byte symbolic input space (0 counter-examples, all assertions proved).

### 13.7 Worked example: annotated step-by-step validation

Below is a minimal but complete SV frame with one ASDU containing svID, smpCnt, confRev, and data. Optional fields are omitted.

| Byte offset | Hex   | Meaning                                                      |
|-------------|-------|--------------------------------------------------------------|
| 0 .. 5      | (6B)  | Dst MAC 01:0C:CD:04:00:01 (SV multicast)                     |
| 6 .. 11     | (6B)  | Src MAC 00:1A:8C:12:34:56 (merging unit)                     |
| 12 .. 13    | 88 BA | EtherType = SV                                               |
| 14 .. 15    | 40 01 | APPID = {\tt 0x4001} (in SV range {\tt 0x4000}–{\tt 0x7FFF}) |

```

16 .. 17    00 26  Length = 38 (bytes 14..51 inclusive)
18 .. 21    00 00 00 00  Reserved1, Reserved2
22          60      savPDU tag = {\tt 0x60}
23          1E      savPDU length = 30 bytes (bytes 24..53)
24          80      noASDU tag
25          01      noASDU length = 1
26          01      noASDU value = 1 (declaring 1 ASDU)
27          A2      seqASDU tag = {\tt 0xA2}
28          17      seqASDU length = 23 bytes
29          30      ASDU[0] tag = {\tt 0x30}
30          15      ASDU[0] length = 21 bytes
31          80      svID tag
32          04      svID length = 4
33..36      73 76 30 31  svID = "sv01"
37          82      smpCnt tag
38          02      smpCnt length = 2
39..40      00 0A      smpCnt = 10
41          83      confRev tag
42          04      confRev length = 4
43..46      00 00 00 01  confRev = 1
47          87      data tag
48          05      data length = 5
49..53      A1 B2 C3 D4 E5  data (5 bytes of samples)

```

#### Validation trace:

1. **Size:** 54 bytes.  $22 \leq 54 \leq 1,600$ . **Pass.**
2. **EtherType:** bytes 12–13 = 88 BA. **Pass.**
3. **APPID:** bytes 14–15 = 40 01 = 0x4001.  $0x4000 \leq 0x4001 \leq 0x7FFF$ . **Pass.**
4. **Header length:** Length field = 38;  $14 + 38 = 52 \leq 54$ . **Pass.**
5. **Outer tag:** byte 22 = 0x60. **Pass.**
6. **PDU length:** savPDU length = 30;  $24 + 30 = 54 \leq 54$ . **Pass.** pdu\_end = 54.
7. **Outer loop, iteration 1:** tag = 0x80 (noASDU), length = 1. declared\_asdu\_count = 1.
8. **Outer loop, iteration 2:** tag = 0xA2 (seqASDU), length = 23. Enter ASDU loop.
9. **ASDU loop, iteration 1:**
  - asdu\_count (0) < 8. Continue.
  - Tag = 0x30. **Pass.**
  - ASDU length = 21.
  - Call sv\_validate\_asdu on 21 bytes:
    - Position 0 (svID, 0x80): tag matches, length  $4 \in [1, 65]$ . **Pass.**
    - Position 1 (datSet, optional, 0x81): next tag is 0x82; field is optional; skip.
    - Position 2 (smpCnt, 0x82): tag matches, length  $2 == 2$ . **Pass.**
    - Position 3 (confRev, 0x83): tag matches, length  $4 == 4$ . **Pass.**
    - Positions 4–6 (optional): next tag is 0x87; each skipped.
    - Position 7 (data, 0x87): tag matches, length  $5 \in [1, 4096]$ . **Pass.**
    - Position 8 (smpMod, optional): content exhausted.
    - Post-loop: all remaining positions are optional. Return 0 (success).
  - asdu\_count incremented to 1.
10. **Trailing bytes:** offset = 54 = pdu\_end. **Pass.**
11. **noASDU consistency:** declared = 1, actual = 1. **Pass.**
12. **Result:** SV\_OK, asdu\_count = 1.

## 14 ACSL Annotations – Complete Reference

Every function in `safe.c` carries ACSL (ANSI/ISO C Specification Language) annotations for verification with Frama-C/WP. This section explains every ACSL construct used in the codebase.

### 14.1 Function Contracts

Listing 59: Anatomy of an ACSL function contract

```

1  /*@
2  requires \valid_read(buffer + (0 .. length - 1));
3  requires *offset <= length;
4  assigns *offset;
5  ensures *offset <= length;
6  ensures *offset >= \old(*offset);
7  */

```

- **requires** – A precondition. The caller must ensure this holds before calling the function. If violated, the behavior is undefined (from the prover’s perspective). Multiple **requires** clauses are conjoined (all must hold).
- **\valid\_read(p + (a .. b))** – Memory from **p+a** through **p+b** is allocated and readable. Used for input buffers that the function reads but does not modify.
- **\valid(p)** – Memory at **p** is allocated, readable, and writable. Used for output parameters that the function writes through.
- **\separated(p, q + (a .. b))** – The pointer **p** does not alias any byte in **q[a..b]**. This is crucial for reasoning about functions that take both a buffer and an output pointer. Without it, writing to **\*p** could change the buffer contents, and the prover could not reason about buffer reads after pointer writes.
- **assigns X** – The function modifies **only X**. Everything else in memory is unchanged. **assigns \nothing** means the function is pure: it modifies no memory at all. This is the most powerful annotation for compositional reasoning: callers know exactly what may have changed.
- **ensures** – A postcondition. The function guarantees this holds when it returns, provided all preconditions were met. Multiple **ensures** clauses are conjoined.
- **\old(E)** – The value of expression **E** at function entry. Used in postconditions to relate output to input. For example, **\*offset >= \old(\*offset)** says the offset never decreased.
- **\result** – The return value of the function. Used in postconditions like **\result.error == ASN\_REJECT\_SIZE** to specify which return value is produced under which conditions.
- **==>** – Logical implication. **A ==> B** means “if **A** holds, then **B** must hold.” Used in postconditions to express conditional guarantees, for example: “if the length exceeds the maximum, then the result code must be **REJECT\_SIZE**.”

## 14.2 Loop Annotations

Listing 60: Anatomy of an ACSL loop annotation

```

1  /*@
2     loop invariant offset <= length;
3     loop invariant count <= 64;
4     loop assigns offset, count, *reject_offset;
5     loop variant length - offset;
6  */
7  while (offset < length) { ... }
```

- **loop invariant** – Must hold before the first iteration and after every iteration. The prover checks two things: (1) the invariant holds on loop entry (“establishment”), and (2) if the invariant holds at the start of an iteration, it still holds at the end (“preservation”).
- **loop assigns** – The loop modifies only these variables. This helps the prover reason about what is preserved across iterations. Without it, the prover would have to assume that any variable might have changed.
- **loop variant** – A non-negative integer expression that strictly decreases with each iteration. This proves the loop terminates. Every loop in the codebase has a proved variant.

## 14.3 Ghost Variables

Listing 61: Ghost variable declaration and use

```

1  /*@ ghost u32 offset_top = offset;
2  /* ... loop body ... */
3  /*@ assert offset > offset_top; */
```

Ghost variables exist only in the specification world; they are not compiled into the binary. They are used to capture intermediate values that the prover needs for reasoning. In the codebase, ghost variables serve two purposes:

1. **offset\_entry** (at function entry) – captures the initial offset so that loop invariants can express **offset >= offset\_entry**.
2. **offset\_top** (at loop iteration start) – captures the offset at the top of each iteration so that an assertion can prove **offset > offset\_top** (strict forward progress).

## 14.4 Inline Assertions

Listing 62: Inline assert

```

1  /*@ assert offset + length <= buffer_length; */
```

The **assert** directive asks the prover to verify that the condition holds at that program point. These assertions serve two purposes:

1. **Proof hints.** They break complex proofs into smaller steps. The SMT solvers often need intermediate facts to complete a chain of reasoning. For example, after **read\_length** returns, asserting **offset <= length** reminds the prover of the postcondition it just established.
2. **Documentation.** They make the programmer’s reasoning visible. A reader can follow the logical chain from precondition through assertions to postcondition.



## 14.5 The decreases Clause (Recursive Termination)

Listing 63: Recursive termination measure

```
1 /*@
2   decreases end - offset;
3 */
4 static SafeAsnResult asn_filter_recursive(
5   ..., u32 offset, const u32 end, ...)
```

The `decreases` clause is for recursive functions. It specifies a non-negative integer expression that must strictly decrease with each recursive call. In the ASN.1 walker, the recursive call into a `SEQUENCE` passes (`offset`, `offset+length`) where `offset+length ≤ end`, so `(offset+length) - offset < end - offset`. Combined with the fact that `offset+length ≤ end` ensures `end - offset` is non-negative, this proves termination.

Only `asn_filter_recursive` uses `decreases` in this codebase: it is the only function that recurses into itself. (`asn_parse_recursive` delegates to `asn_filter_recursive` in the Frama-C proof model and does not carry a `decreases` clause.)

## 14.6 The \separated Clause in Detail

The `\separated` annotation appears in every function that takes both an input buffer and one or more output pointers. The reason it matters is straightforward. Consider `read_length`:

```
1 requires \separated(offset, buffer + (0 .. buffer_length - 1));
```

Without this, the prover must consider the possibility that `offset` points into the middle of `buffer`. Then `++(*offset)` could change `buffer[k]` for some `k`, and the prover could not assume `buffer` is unchanged after the write. With separation, the prover knows that writing to `*offset` leaves the buffer untouched.

In `goose_validate_alldata`, three separation clauses are needed because there are two output pointers (`reject_offset` and `data_value_count_output`) plus the input buffer. Each pair must be declared separated.

## 15 Frama-C Verification

### 15.1 Why Formal Verification Matters

Normal testing checks specific inputs: “does the filter reject this one malformed packet?” But an attacker can craft any input. Testing can never cover all possible inputs.

**Formal verification** proves properties for *all* possible inputs. Instead of “we tested 17 billion packets and found no crashes,” formal verification says “for every possible byte sequence of every possible length, the filter will never read out of bounds, never overflow an integer, and always terminate.”

This library proves three categories of properties:

1. **Memory safety:** Every array access is within bounds. Every pointer dereference is valid. This is proved automatically by the `-wp-rte` flag, which inserts a proof obligation for every potentially dangerous operation.
2. **Termination:** Every loop terminates. The recursive ASN.1 walker terminates because the walk region shrinks on each call (`decreases end - offset`). A malicious input can make the filter reject, but never hang.
3. **Functional correctness** (selected properties): If the first tag byte is not in the whitelist, the filter returns a rejection code. If the frame exceeds the maximum size, the filter returns `REJECT_SIZE`. These are machine-checked postconditions.

The result: **2798/2798 proof goals discharged (100%)** using two SMT solvers (Alt-Ergo and Z3). This means no memory-safety bug exists in the filter code, regardless of what input an attacker provides.

### 15.2 Running the Verification

All functions in `safe.c` are verified with Frama-C/WP:

```
bash run_frama_parallel.sh
```

**Result: 2798/2798 goals proved (100%).**

The script discovers and runs all functions in parallel, using per-function timeouts (90–300s) tuned to goal complexity. For a quick single-command sanity check, the following also works but requires a longer timeout for `asn_parse_recursive`:

```
frama-c -wp -wp-rte -wp-timeout 300 -wp-prover alt-ergo,z3 safe.c
```



### 15.3 What the Command-Line Flags Mean

- `-wp` – enable the Weakest Precondition plugin, which generates proof obligations from ACSL annotations.
- `-wp-rte` – generate Runtime Error assertions. This inserts checks for every pointer dereference (is the pointer valid?), every array access (is the index in bounds?), every signed arithmetic operation (does it overflow?), and every division (is the divisor nonzero?). These generated assertions become additional proof goals.
- `-wp-timeout 90` – default per-goal time limit. Most goals discharge in under 1s; `asn_parse_recursive` requires up to 300s for its loop-invariant preservation goals. `run_frama_parallel.sh` applies the correct per-function timeout automatically.
- `-wp-prover alt-ergo,z3` – use two SMT solvers. A goal is proved if either solver discharges it. Using two solvers improves coverage because each has different strengths.

### 15.4 What the Verification Covers

- **Memory safety.** Every buffer access is within bounds. The `-wp-rte` flag generates assertions for all pointer dereferences, array accesses, signed arithmetic overflow, and division by zero.
- **Functional correctness.** Postconditions on every function ensure that specific inputs produce specific rejection codes. For example, the GOOSE filter’s contract guarantees that a wrong EtherType always yields `GOOSE_REJECT_ETH_TYPE`.
- **Termination.** Loop variants prove that every loop terminates. The `decreases` clause on `asn_filter_recursive` proves termination across recursive calls.
- **Bound guarantees.** Output counts are bounded: `data_value_count <= 64` (GOOSE), `asdu_count <= 8` (SV), `object_count <= 64` (DNP3).
- **Offset monotonicity.** `read_length` never decreases its offset parameter, preventing infinite loops in callers.

## 16 Lookup Table Catalog

For reference, here is the complete catalog of all 16 lookup tables in `safe.c`:

Table 22: Complete catalog of lookup tables in `safe.c`

| Table                        | Size   | Purpose                                                                           |
|------------------------------|--------|-----------------------------------------------------------------------------------|
| ASN_TAG_WHITELIST            | 192    | Accept array for ASN.1 tag bytes                                                  |
| ASN_TAG_WHITELIST_RELAXED    | 192    | Extra tags accepted by <code>safe_asn_filter_relaxed</code> (X.509/OCSP/Kerberos) |
| MMS_PDU_WHITELIST            | 176    | Accept array for MMS PDU type tags                                                |
| MMS_SERVICE_WHITELIST        | 80     | Accept array for MMS service tags                                                 |
| DNP3_FUNCTION_CODE_WHITELIST | 136    | Accept array for DNP3 function codes                                              |
| DNP3_GROUP_WHITELIST         | 123    | Accept array for DNP3 object groups                                               |
| DNP3_POINT_SIZE              | 81×7   | 2D point size by group and variation                                              |
| DNP3_RANGE_SIZE              | 16     | Range field size by qualifier low nibble                                          |
| DNP3_PREFIX_SIZE             | 8      | Object prefix size by qualifier bits 4–6                                          |
| IEC104_ELEMENT_SIZE          | 256    | Element bytes per information object by TypeID                                    |
| IEC104_TYPEID_WHITELIST      | 256    | Accept array for IEC 104 TypeIDs                                                  |
| IEC104_UFRAME_WHITELIST      | 256    | Accept array for U-frame CF1 values                                               |
| GOOSE_NEXT_STATE             | 14×256 | State transition table for field ordering                                         |
| GOOSE_FIELD_BOUNDS           | 256    | Min/max field length by tag (struct array)                                        |
| GOOSE_DATA_VALUE_WHITELIST   | 168    | Accept array for allData value tags                                               |
| SV_FIELDS                    | 10     | Field descriptors: tag, min, max, optional (struct array)                         |

The IEC 104 element-size and permitted-TypeID tables are cross-referenced against the lib60870 (MZ Automation) TypeID enumeration to ensure correctness.

## 17 Summary of All Functions

Table 23: Public API function summary

| Function                          | Visibility          | Purpose                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |
|-----------------------------------|---------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>read_length</code>          | <code>static</code> | Decode BER definite-length field; advance <code>*offset</code> . Shared by all BER-based filters (ASN.1, GOOSE, SV, MMS). Returns the decoded length as <code>uint32_t</code> . Postcondition guarantees offset monotonicity.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
| <code>asn_filter_recursive</code> | <code>static</code> | Recursive TLV walker for ASN.1 DER. Checks each tag against <code>ASN_TAG_WHITELIST</code> , decodes BER lengths, validates INTEGER minimal encoding, recurses into SEQUENCE children. Bounded to caller-supplied <code>max_depth</code> (8 for headers, 32 for MMS bodies). <code>decreases end - offset</code> proves termination.                                                                                                                                                                                                                                                                                                                                                                                                                       |
| <code>asn_filter_top</code>       | <code>static</code> | Shared pre-check entry point used by both <code>safe_asn_filter</code> and <code>safe_mms_filter</code> . Validates null pointer, message size, and first-tag whitelist membership, then delegates to <code>asn_filter_recursive</code> with the caller-supplied <code>max_depth</code> . Replaces the former separate <code>safe_asn_filter_body</code> wrapper.                                                                                                                                                                                                                                                                                                                                                                                          |
| <code>safe_asn_filter</code>      | <code>public</code> | Entry point for ASN.1 DER structural validation (protocol headers). Thin wrapper: calls <code>asn_filter_top</code> with <code>max_depth = SAFE_ASN_MAX_DEPTH = 8</code> .                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
| <code>safe_mms_filter</code>      | <code>public</code> | Entry point for MMS PDU validation. Checks PDU tag against <code>MMS_PDU_WHITELIST</code> . For confirmed PDUs (0xA0/0xA1), reads <code>invokeId</code> INTEGER directly after the PDU length (no inner SEQUENCE; [0] IMPLICIT SEQUENCE per ISO 9506-2), validates <code>invokeId</code> (max 4 bytes), and extracts service tag (with base-128 high-tag decoding) checked against <code>MMS_SERVICE_WHITELIST</code> . For unconfirmed PDUs (0xA3), expects inner SEQUENCE (0x30); only service 0 (informationReport) is accepted. For accepted services, calls <code>asn_filter_top</code> with <code>SAFE_ASN_MMS_BODY_MAX_DEPTH = 32</code> on the service body content; returns <code>MMS_REJECT_BODY</code> if the body fails structural validation. |
| <code>safe_dnp3_filter</code>     | <code>public</code> | Entry point for DNP3 application layer validation. Checks function code against <code>DNP3_FUNCTION_CODE_WHITELIST</code> . Short-circuits for confirm/delay-measure. Skips IIN for responses. Parses up to 64 object headers, validating group ( <code>DNP3_GROUP_WHITELIST</code> ), variation ( <code>DNP3_POINT_SIZE</code> ), and qualifier ( <code>DNP3_RANGE_SIZE</code> , <code>DNP3_PREFIX_SIZE</code> ). Computes data block sizes for non-read function codes.                                                                                                                                                                                                                                                                                  |
| <code>safe_iec104_filter</code>   | <code>public</code> | Entry point for IEC 60870-5-104 APDU validation. Checks start byte (0x68), L field consistency, and frame type (I/S/U). For U-frames, checks CF1 against <code>IEC104_UFRAME_WHITELIST</code> . For I-frames, checks <code>TypeID</code> against <code>IEC104_TYPEID_WHITELIST</code> , validates VSQ object count (1–127), looks up element size from <code>IEC104_ELEMENT_SIZE</code> , and verifies the body byte count fits in the APDU. Returns <code>frame_type</code> , <code>object_count</code> , and error code.                                                                                                                                                                                                                                 |

| Function                            | Visibility          | Purpose                                                                                                                                                                                                                                                                                                                                                                                  |
|-------------------------------------|---------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>goose_validate_alldata</code> | <code>static</code> | Walk GOOSE allData TLVs. Check each data value tag against <code>GOOSE_DATA_VALUE_WHITELIST</code> . Inline BER length decoding. Count must be 1–64. Returns rejection code and offset through output pointers.                                                                                                                                                                          |
| <code>safe_goose_filter</code>      | <code>public</code> | Entry point for GOOSE frame validation. Checks Ethernet header (EtherType <code>0x88B8</code> ), header length, APDU size ( $\leq 1400$ ), outer tag ( <code>0x61</code> ). Walks fields via <code>GOOSE_NEXT</code> state machine. Validates field lengths against <code>GOOSE_FIELD_MIN/GOOSE_FIELD_MAX</code> . Delegates allData validation to <code>goose_validate_alldata</code> . |
| <code>sv_validate_asdu</code>       | <code>static</code> | Walk SV ASDU fields against the <code>SV_FIELDS[10]</code> struct array (tag, min, max, optional per field). Check tag order, length bounds, and optionality. After parsing, verify that all remaining unparsed fields are optional.                                                                                                                                                     |
| <code>safe_sv_filter</code>         | <code>public</code> | Entry point for SV frame validation. Checks Ethernet header (EtherType <code>0x88BA</code> ), header length, outer tag ( <code>0x60</code> ). Walks SAVPDU top-level elements. For <code>SV_SEQASDU_TAG</code> ( <code>0xA2</code> ), iterates ASDUs (max 8), each validated by <code>sv_validate_asdu</code> .                                                                          |

All static functions are called only from the corresponding `safe*_filter` entry point. All functions are pure (no side effects beyond writing to output parameters passed by pointer or returning result structs by value). All are formally verified with Frama-C/WP: **2798/2798 goals proved (100%)**.